SENTAURUS DOCUMENTATION / LOCAL HTML EXPORT

Mystic User Guide

W-2024.09

Exported from the installed server documentation on 2026-10-22.

> This is a local PDF export of the complete User Guide chapter tree and copyright page. It is not the publisher's native PDF.
>
> Source installation: W-2024.09-SP1
> Source: manuals/tcad_spice/mystic_api/user_guide.html
>
> Chapter links are retained. Links to API pages outside this guide are shown as plain text. The original HTML, assets and API pages are preserved separately in HTML_Sources.zip.

Included sections

Mystic User Guide
Introduction to the Mystic Tool
Uploading Target Data to The Database
The Mystic GUI
SPICE Model Extraction
Specific Use Cases
Machine Learning in Mystic
Copyright

Mystic User Guide

Welcome to the Mystic User Guide! This documentation provides detailed usage information for the Mystic SPICE model extraction framework.

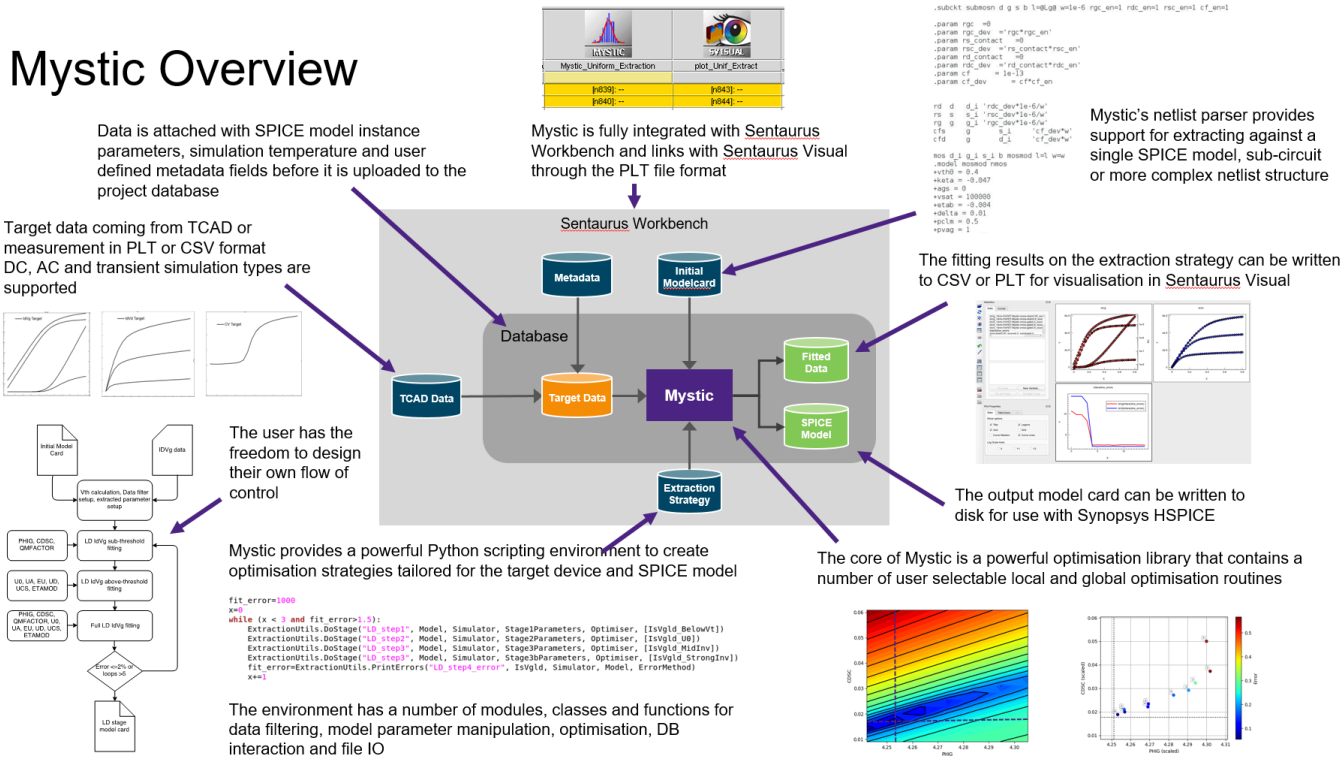
Contents:

- Introduction to the Mystic Tool
  - Installing Mystic
  - Running Mystic as a Standalone Tool
  - Running Mystic Through Sentaurus Workbench
  - Extraction Environment
- Uploading Target Data to The Database
  - The Data Class
  - Uploading PLT Files
  - Uploading CSV Files
- The Mystic GUI
  - Introduction
  - Interface Overview
  - Menus and Run Toolbar
  - Editor Panel
  - Syntax Checker
  - Console Panel
  - Plot Log Panel
  - Controlling the Execution of the Strategy
  - Doc Lookup Panel
  - Variable Explorer Panel
  - Plot Area Panel
  - Plot Parameters Panel
  - Wizards Panel
  - Usage Examples
- SPICE Model Extraction
  - Database
  - Target Data
  - Filtering Data
  - Data Masks
  - Figure of Merit Transforms
  - SPICE Models
  - Simulator
  - Parameter Space Exploration
  - Optimization Parameters
  - Optimization Algorithms
  - The Engine Module
  - Full Strategy
  - Configuring Settings
- Specific Use Cases
  - Transient Simulation Extraction
  - MOSFET Reliability (MOSRA) Extraction
  - Extraction of Variability Using Principal Component Analysis
  - RF Extraction
  - Lookup Table (LUT) Modelling
  - Josephson Junction Extraction
- Machine Learning in Mystic
  - Data Storage & Retrieval

Introduction to the Mystic Tool

Mystic is a Python-based framework designed to facilitate the extraction of SPICE compact models, using the Synopsys® PrimeSim™ HSPICE® tool. The process of extracting a SPICE model manually involves adjusting the parameter values of a SPICE compact model, running simulations with a SPICE engine, and comparing these simulation results to a set of target data curves, usually coming from measurement or TCAD data. This process is repeated as many times as necessary to obtain an accurate fit with the target data and requires a strong understanding of the underlying equations of the SPICE model and experience in setting up and running SPICE simulations. Mystic aims to limit barriers to using SPICE modeling by providing a flexible framework to assist with all parts of the procedure, from parsing input SPICE models, to automatically setting up, running, and parsing outputs from SPICE simulations based on provided target data, to optimizing the values of the SPICE model parameters using a range of sophisticated optimization algorithms.

This section looks at the standard Mystic tool flow, including the input requirements, core components, and tool outputs. The figure below shows a block-level overview of a standard Mystic tool flow.



Overview of the Mystic tool flow.

Mystic is contained within Sentaurus Workbench. This integration allows you to easily connect to TCAD Sentaurus projects that can provide target data for SPICE compact model extraction. It also helps with management of splits, such as different temperatures and process conditions, provides access to local compute clusters, and allows you to visualize SPICE model extraction results using Sentaurus Visual. Underneath Sentaurus Workbench, Mystic provides a local database for consolidating target data and simulation conditions, such as temperature and instance parameters, and transferring this information between sequential Mystic extraction stages. It can also be used to track SPICE model parameter values across stages and to integrate with the larger TCAD to SPICE tool flow.

Installing Mystic

For detailed setup and installation instructions, see the TCAD installation notes.

Running Mystic as a Standalone Tool

If you run Mystic through Sentaurus Workbench, then the execution of Mystic and the configuration of command-line options are handled automatically. For instructions on setting up Mystic, refer to the TCAD Installation Notes. After you have loaded the TCAD to SPICE tools into your environment, you can run Mystic as follows:

```
Mystic <filename> -d <string> -m <string> [other_options]
```

The mandatory file name specifies a Python file containing a structured set of commands that perform the SPICE model extraction. The *-d* option provides an optional path to the TCAD to SPICE database configuration file. If you want to use the database when running Mystic in standalone mode, then it must be configured separately. The *-m* option specifies a SPICE model compatible with PrimeSim HSPICE. For details, see the SPICE Models section. When the *-i* option is supplied the extraction strategy will launch in the Mystic GUI. A full description of the GUI can be found in the Machine Learning in Mystic section and the Mystic GUI API section. The table below gives a full list of command line arguments for Mystic.

Short Flag	Long Flag	Arguments	Description	Type
		filename	Mystic strategy to execute.	Required
-h	–help	•	Print the help message and exit.	Optional
-v	–version	•	Print the Mystic version then exit.	Optional
-d	–db	filename	Location of the TCAD to SPICE database configuration file.	Optional
-m	–model	filename	The filename of the initial model to be loaded into the extraction strategy.	Optional
-ds	–dataset	string	Name of the dataset in the DB to store the Mystic extraction results.	Optional
-o	–optimizer	string	Select the optimization algorithm to use in the strategy. See Optimization Algorithms for details.	Optional
-x	–spice-args	string	String containing extra spice args to pass to PrimeSim HSPICE back-end simulation at run-time.	Optional

Short Flag	Long Flag	Arguments	Description	Type
-l	–location	string	Optional path of the PrimeSim HSPICE executable to use. This will override the version in the system path.	
-g	–gui	•	Run Mystic in interactive mode using the Mystic GUI.	Optional
-c	–console	•	If specified, launch the Mystic GUI with a regular Python console, instead of IPython.	Optional
-sl	–gui-log	filename	Optional filename for the Mystic GUI to write the Mystic output to. If not supplied output information may be lost when running through the GUI	Optional
-ar	–auto-run	•	Automatically start the extraction strategy if you’re running interactively in the Mystic GUI	Optional
-ac	–auto-close	•	Automatically close the GUI window when the strategy execution finishes if you’re running interactively in the Mystic GUI.	Optional

When running in interactive mode using the -g/–gui flag, Mystic can be opened without a strategy filename as follows:

```
Mystic --gui
```

When the GUI opens, you can use the file browser panel to open and run a strategy in the GUI. For more information about the Mystic GUI see Machine Learning in Mystic.

Running Mystic Through Sentaurus Workbench

The most common way of running Mystic is through Sentaurus Workbench. Each Mystic tool stage can contain up to five strategies that can be configured and executed by a master gpythonsh template. The gpythonsh environment provides a *mystic* object which presents a simulation interface to Mystic. The following is an annotated template example which shows how to control and link multiple Mystic extractions:

```
# Setup some variables for the fils paths to the five post processed extraction strategies
# and the initial model card. This section will be the same in every template. The variables
# encolsed by @ symbols are SWB preprocessor variables. For more information on these please
# consult the SWB documentation.
mystic_inputfile1 = "@pwd@/@nodedir@/pp@node@_mys.py1"
mystic_inputfile2 = "@pwd@/@nodedir@/pp@node@_mys.py2"
mystic_inputfile3 = "@pwd@/@nodedir@/pp@node@_mys.py3"
mystic_inputfile4 = "@pwd@/@nodedir@/pp@node@_mys.py4"
mystic_inputfile5 = "@pwd@/@nodedir@/pp@node@_mys.py5"
initial_modelcard = "@pwd@/@nodedir@/pp@node@_mys.mod"

# Tell Mystic to wait for a simulation to finish before moving on the the next command.
mystic.wait         = True

# Setup Mystic command line arguments. Please see a description of these below.
mystic.args.gui             = False
# Set the model argument to the initial modelcard file path.
mystic.args.model   = initial_modelcard

# stage 1 (LD)
# Add the first inputfile
mystic.inputfile    = mystic_inputfile1
# Execute the first Mystic extraction strategy. The previous and dataset arguments control the
# database connections between sequential Mystic stages. In this case we are running the first
# stage so there is no previous dataset. The dataset should be aa unique name based on the current
# SWB 'node'.
mystic(previous=None,   dataset=f"{node}-low-drain")

# stage 2 (HD)
# Add the second inputfile
mystic.inputfile    = mystic_inputfile2
# Since the model is transferred through the database we no longer need a model argument here.
mystic.args.model   = None
# Execute the second Mystic extraction strategy. We can now set the previous argument to the
# first extraction stage dataset and set the dataset argument to a new dataset name.
mystic(previous=f"{node}-low-drain", dataset=f"{node}-high-drain")

# stage 3 (CV)
# Add the third inputfile
mystic.inputfile    = mystic_inputfile3
mystic.args.model   = None
# Execute the third Mystic extraction strategy. We can now set the previous argument to the
# second extraction stage dataset and set the dataset argument to a new dataset name.
mystic(previous=f"{node}-high-drain", dataset=f"{node}-cv")

# In this example the fourth and fit strategies are not used. It's common practice to retain these
# blocks in the template and uncomment them if needed.
# stage 4 (unused)
# mystic.inputfile  = mystic_inputfile4
# mystic.args.model = None
# mystic(previous=f"{node}-cv", dataset=f"{node}-temp")

# stage 5 (unused)
# mystic.inputfile  = mystic_inputfile5
# mystic.args.model = None
# mystic(previous=f"{node}-temp", dataset=f"{node}-stress")
```

The *mystic.args* attribute can be used to control the command line arguments passed to Mystic by gpythonsh when it executes the jobs. All of the arguments listed in the table above can be applied here. Dashes in argument names are replaced with underscored to conform to Python syntax rules, e.g. *–spice-args* will become *mystic.args.spiceargs*. Note that when running through SWB and gpythonsh the *–db* and

–*dataset* arguments are automatically applied for you and don't need to be configured. The most common argument used here is the *mystic.args.gui* flag which, when set to True, launches each Mystic strategy in the interactive GUI described in Machine Learning in Mystic.

Extraction Environment

The Mystic extraction environment is a collection of specific Python modules, classes, and objects that are designed to help with SPICE compact model extraction (see Table below). You have full freedom to assemble these in a strategy file to suit the specific needs of your extraction.

Variable	Description
`ExtractionUtils`	ExtractionUtils provides utility functions to help with extraction. These functions usually combine the `Model`, `Simulator`, `Optimizer` and `SimData` objects.
`Engine`	The `Engine` class provides an object oriented alternative to `ExtractionUtils`, which will be deprecated in future releases.
`OptimizationParameter`	OptimizationParameters or OptParams define spice model parameters to be extracted. These keep track of the current value of parameters throughout the extraction.
`LinkedOptimizationParameter`	A LinkedOptimizationParameter can be used to link one parameter value to another to ensure they remain consistent throughout the extraction.
`Optimizer`	The Optimizer object provides an interface to the Mystic optimization library. This features a range of sophisticated algorithms for optimizing the values of SPICE model parameters.
`mdb`	mdb provides access to the TCAD to SPICE database where the target data is stored. The can be used to load data into the environment at the start of the strategy and store fitted models and data at the end.
`Model`	The Model object allows a user to get and set SPICE model parameter values inside the strategy.
`Simulator`	The Simulator object represents the back end PrimeSim HSPICE simulator and can be used to set command line arguments, global options and simulation temperature.
`SimData`	SimData provides a container format for target and simulation data. Normally SimData objects can be loaded into the extraction strategy from the database but they can also be initialized from scratch if necessary.
`Corrector`	The correctors module provides functionality for applying small analytic corrections on top of a SPICE model to perfectly match target data. This should be used when extreme accuracy is required.

Variable	Description
`CorrelatedParameters`	The CorrelatedParameters class allows a user to apply simple parameter variations to SPICE models in Mystic itself. The resultant models can then be used in PrimeSim HSPICE directly without the need for RandomSPICE.

Objects that are passed into the environment are dependent on the command-line options passed to Mystic. For example:

- If no initial model card is passed using the -m option, then the `Model` object in the environment will default to None and must be initialized within the strategy.
- If no database configuration file is passed using the -d option, then the `mdb` object will default to None.
- When running Mystic through Sentaurus Workbench, the command-line options are handled automatically, so all objects in the extraction environment are configured correctly.

Uploading Target Data to The Database

As mentioned in the Mystic Tool Flow section, Mystic is most commonly run within Sentarus Workbench as part of a larger TCAD to SPICE tool flow. These tool flows provide a local database for propagating information throughout the flow. The most common way of preparing target data for a Mystic extraction is to upload it to the database using a TCAD to SPICE Python stage located before Mystic in the flow. This methodology allows the data to be pre-processed into the correct structure and tagged with useful metadata fields that Mystic can utilize, such as simulation temperature and SPICE model instance parameters. This section will cover the different file types that can be used as target data and how to upload these to the database using TCAD to SPICE Python. It will also describe the required metadata fields that must be attached to the target data and how these are used by Mystic.

The Data Class

The `Data` class is a custom Python class that is injected into the TCAD to SPICE Python environment at runtime. It contains functionality for reading and writing PLT and CSV files, as well as a link to the database that can be used to push and pull simulation data. In the Mystic environment the Data class is used to represent all target and simulation data. It provides functionality for filtering, resampling, applying transforms and error calculation between targets and simulations during optimization. The structure of Data objects is used to construct the backend PrimeSim HSPICE netlists that are created and simulated by Mystic during the extraction process. Enough information must be provided through the metadata dictionary for Mystic to replicate the simulation conditions of the target in PrimeSim HSPICE. Below is a representation of a standard IdVg Data object.

```
bias: {'drain': 0.05, 'source': 0.0, 'substrate': 0.0}
instances: {'l':12.5e-8, 'nfin': 1}
nodes: ['drain', 'gate', 'source', 'substrate']
temperature: 27.0
tfin: 8e-9
tspacer: 8e-9
workfn: 4.301
dtype: nMOS
eot: 1e-9
hfin: 4e-8

                    idrain idrain-fit errors
vgate
0.000000e+00 -4.654437e-09        nan    nan
2.500000e-02 -1.059546e-08        nan    nan
5.000000e-02 -2.393837e-08        nan    nan
7.500000e-02 -5.346387e-08        nan    nan
1.000000e-01 -1.171774e-07        nan    nan
```

The key value pair dictionary at the top is the metadata. This contains static bias conditions, SPICE model instance parameters, an ordered list of SPICE model instance terminals, the simulation temperature and some extra arbitrary fields that help to distinguish between Data objects when loading and filtering. These are particularly useful if you have a large DoE of data at different process conditions. Of these, bias, temperature, nodes and instances are required fields. Biases are picked up automatically from the PLT or CSV files if possible but can also be supplied explicitly if necessary. This will be shown in the next section. The nodes list must match both the terminals of the model being extracted by Mystic and the names of the biases in the Data. As we can see in the example above, drain, source and substrate are all present in the

nodes list and bias dictionary (gate voltage does not have to be present in the bias dictionary as it the independent variable being swept in the simulation). Simulation temperature should be entered into the Data class in Kelvin. This will be converted to Celsius (required by SPICE) internally by Mystic. Entries in the instances dictionary must match the instance parameters in the SPICE model. This example comes from a BSIMCMG SPICE model which takes *l* and *nfin* instance parameters. This can be left empty if the default values are to be used. The core of the data is generally a single independent variable sweep, in this case gate voltage, and a single dependent variable, in this case drain current. Further dynamic input currents or voltages can be added as stimulus columns. This is mainly used for the transient simulation type and is covered in the Transient Extraction section. The column headers determine the type of PrimeSim HSPICE simulation that Mystic will produce for the data in question. The different valid column notations are shown in the table below.

Notation	As ivar | stimulus | bias	As dvar
vcontact	Voltage source - either stationary or a sweep applied via a .DATA statement in the netlist.	A contact where a voltage r esponse. is measured.
icontact	Current source - either stationary or a sweep applied via a .DATA statement in the netlist.	A contact from which a current response is measured.
c(contact0,contact1)	n/a	Capacitance, with small signal applied to contact0 and measured at contact1.
time	Time values for transient simulations.	n/a

When the Data is created two placeholder columns are added automatically for the PrimeSim HSPICE simulation data and a pointwise error calculation respectively. These will be populated by Mystic during the extraction.

Uploading PLT Files

The most common file format for Mystic extractions is the PLT file. PLT files are produced by Sentaurus Device simulations which usually provide the target data for TCAD to SPICE flows. Below is an example of how to read in a Sentaurus Device PLT file and upload its contents to the database for use with Mystic.

```
ds = dbi.create_dataset(node_prj, "iv-data", clean=True)

metadata = {'temperature': 300,
            'instances': {'l': 27e-9, 'nfin':1},
            'nodes': ['drain','gate','source','substrate']}

Data.from_plt('idvgld.plt', metadata=metadata, upload_ds=ds)
```

First, a location in the database to store the data is created using dbi.create_dataset, where dbi is the Python interface to the TCAD to SPICE database. More information about this can be found in the TCAD to SPICE Python API documentation. We then create a metadata dictionary with the required information

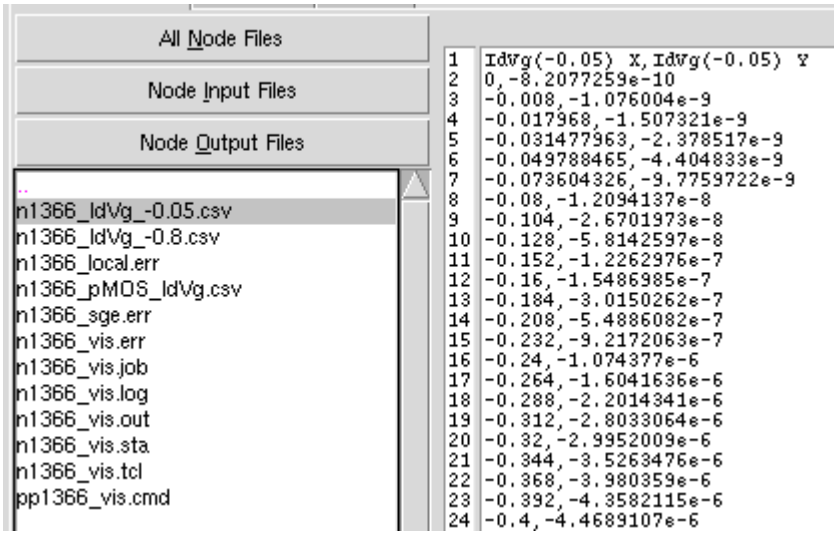
described in the previous section. Note that there is no bias dictionary defined here. Sentaurus Device PLT files contain all static and dynamic biases for the simulation, so the bias dictionary is automatically populated when the file is read. Finally, we use the Data.from_plt method to simultaneously read the information from the specified file and upload it to the database in the created location. The PLT reader automatically recognizes some of the default Sentaurus Device notation and converts it to the format required by Mystic:

- “contact OuterVoltage” is converted to “vcontact”
- “contact TotalCurrent” is converted to “icontact”

Capacitances tend to be in the correct format already and don’t need to be converted. If the column headers are not in the expected format, they can also be converted explicitly using a mapping. This will be covered in the next section. The independent and dependent variables are automatically detected as the file is read but can also be specified explicitly with ivar and dvar function arguments if required.

Uploading CSV Files

The CSV file is a very flexible format and allows a wider range of targets to be supported by Mystic. When working with CSV files the user must provide some extra information to ensure the data labels are converted into the format required by Mystic. Consider the example Id-Vg CSV file shown below:



Example Id-Vg CSV file.

In this case the columns still contain gate voltage and drain current, as in the previous example, but the column headers are not in a format that Mystic will recognize. The following example shows how to upload this file:

```
ds = dbi.create_dataset(node_prj, "iv-data", clean=True)

metadata = {'temperature': 300,
            'instances': {'l': 27e-9, 'nfin':1},
            'nodes': ['drain','gate','source','substrate'],
            'bias': {'drain': -0.05,
                     'source': 0.0,
                     'substrate': 0.0}}

column_map = {'IdVg(-0.05) X': 'vgate',
              'IdVg(-0.05) Y': 'idrain'}

Data.from_csv('n1366_IdVg_-0.05.csv', metadata=metadata,
              column_map=column_map, upload_ds=ds)
```

The first thing to note is that there is now a bias dictionary in the metadata. This CSV file contains no static biases, so values for drain, source and substrate voltage must be added manually. A column mapping dictionary can be defined to tell the Data class how to convert the column headers into the correct format. This dictionary is passed into the Data.from_csv function as a column_map argument. Once stored in the database, this data will have the exact same formatting as the PLT data in the previous section and can be used by Mystic to extract a SPICE model.

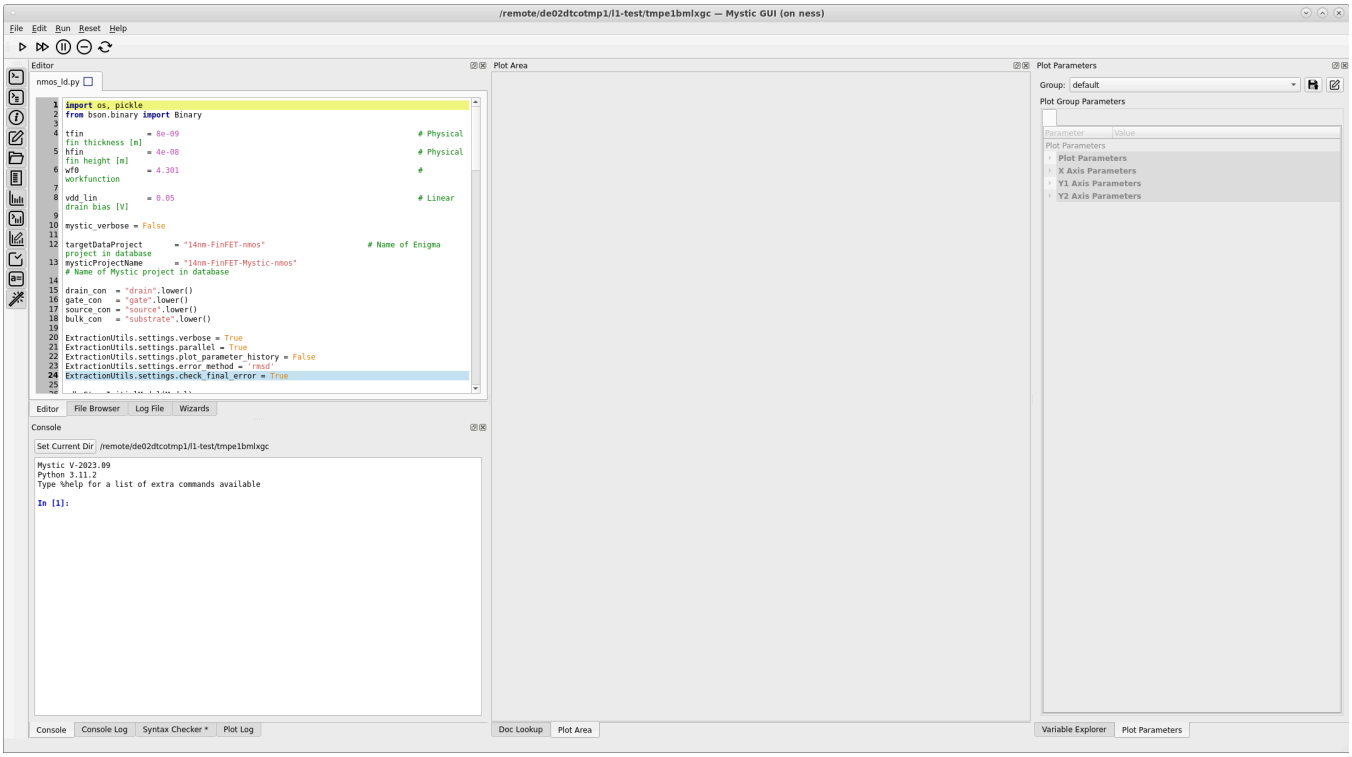
The Mystic GUI

Introduction

The Mystic Graphical User Interface (Mystic GUI) can be used to run Mystic SPICE model extraction strategies interactively. It allows you to move through existing extraction strategies step by step, analyze the outcomes of each individual command and generate plots to monitor the progress of the SPICE model fitting against the target data. The built in IPython console also allows you to build up new strategies step by step, explore parameter spaces and make manual adjustments to SPICE model parameter values. This section will give an overview of the Mystic GUI interface and give some examples of how the GUI can be used to enhance the SPICE model extraction process. For more detailed information on the GUI please consult the Mystic API documentation. For instructions on how to launch the Mystic GUI see the Running Mystic as a Standalone Tool section.

Interface Overview

The Mystic GUI is comprised of multiple dockable widgets that can be configured to suit the user's preferences. The figure below shows the default panel layout.



The Mystic GUI default layout.

The default layout splits the Mystic GUI into four main panels. Each of the elements in these panels is described in this section. The elements link together and interactive with each other to provide an environment for the interactive extraction of SPICE compact models. All panels can be shown or hidden by using the toolbar on the left side of the main window.

All widgets can be switched on or off using the View -> Panels menu from the top toolbar. Once the layout has been arranged to suit your preferences you can save this layout in the Edit -> Preferences -> Layout menu. The default layout splits the GUI into four main panels. Each of the widgets included in these panels

will be described in this section. The widgets link together and interactive with each other to provide an environment for the interactive extraction of SPICE compact models.

Menus and Run Toolbar

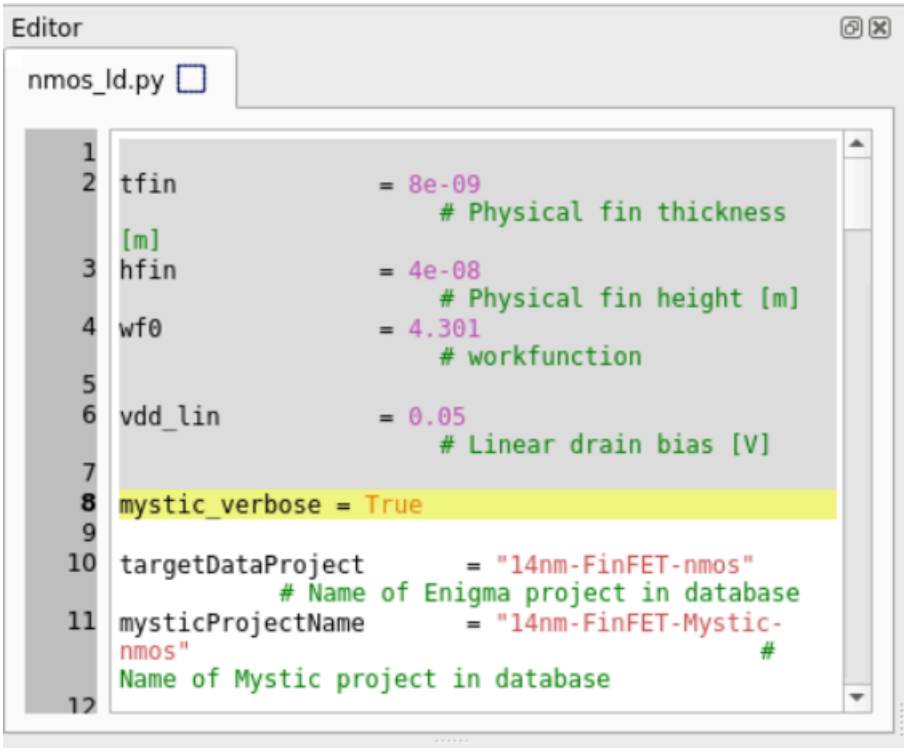
The table below describes the available menus:

Menu	Description
File	Buttons for opening and saving files, and quitting the GUI. Shortcuts for these operations are also provided.
Edit	Utilities for copying commands, finding patterns in the open file and accessing the GUI preferences.
Run	Buttons for running the strategy file in the console. These are described further in the console section.
Reset	Reset the File Editor, Console, Plot Area or the entire session.
Help	Contains a table of keyboard shortcuts and links to the Mystic API documentation.

The Run toolbar above the main panels controls the execution of the strategy. It is described in detail in the console section.

Editor Panel

The strategy itself is opened in the File Editor widget in the top left panel. When first opened, the full strategy is editable. You can make alterations and save these back to disk from the File menu. When an extraction is running, the contents of the file will be progressively blocked and made read only as each line is executed. The line highlighted yellow is the next line of execution. For longer running commands such as extraction stages the line that is currently being executed will be highlighted green. Executed lines then turned grey and become read only as shown in the figure below.



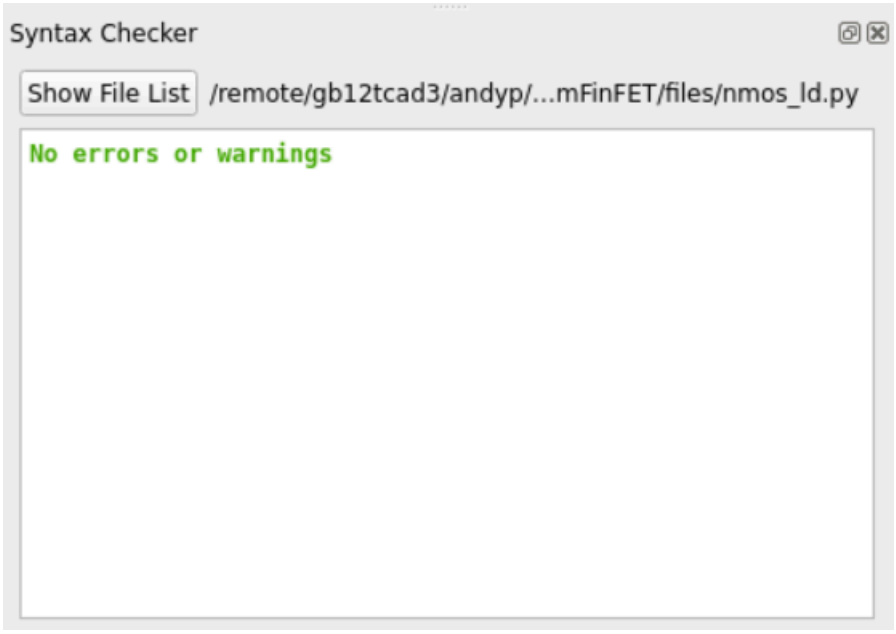
File Editor part way through an extraction. Executed line have turned grey and are now read only.

Breakpoints can be inserted by either clicking on the line number on the left hand side of the editor or moving the cursor to the desired line in the editor then clicking the beakpoint button on the Run Toolbar. Once a breakpoint has been set the script execution will stop at the breakpoint when the Run/Continue button in the Run Toolbar is clicked.

The Editor panel provides many useful features to help with strategy editing such as auto-indentation when there are line breaks, find and replace (press Ctrl+F), block commenting (press Ctrl+T), block tab indentation, and code completion.

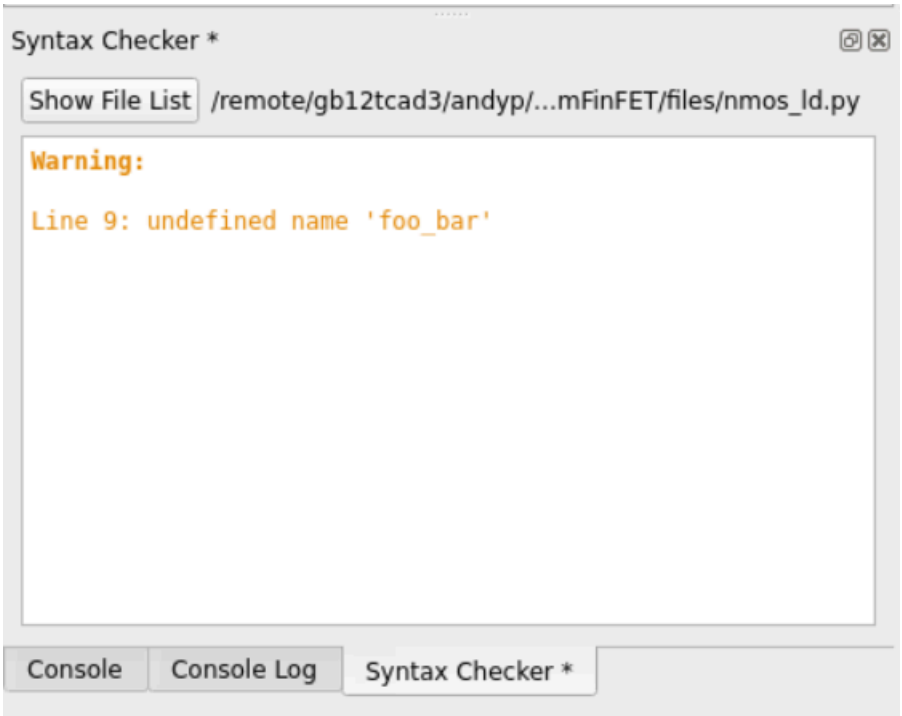
Syntax Checker

Prior to running a strategy it's important to check for potential errors in the syntax, The Syntax Checker widget in the bottom left panel checks for mistakes in general Python syntax and alerts you of potential problems with Python variable definitions. Identifying problems can help save time, particularly with long running strategies. If no problems are detected, you'll see the green message in the figure below.



Syntax Checker with no identified problems in the strategy.

If there are problems in the strategy file, an asterisk will appear on the Syntax Checker tab as shown in the figure below.



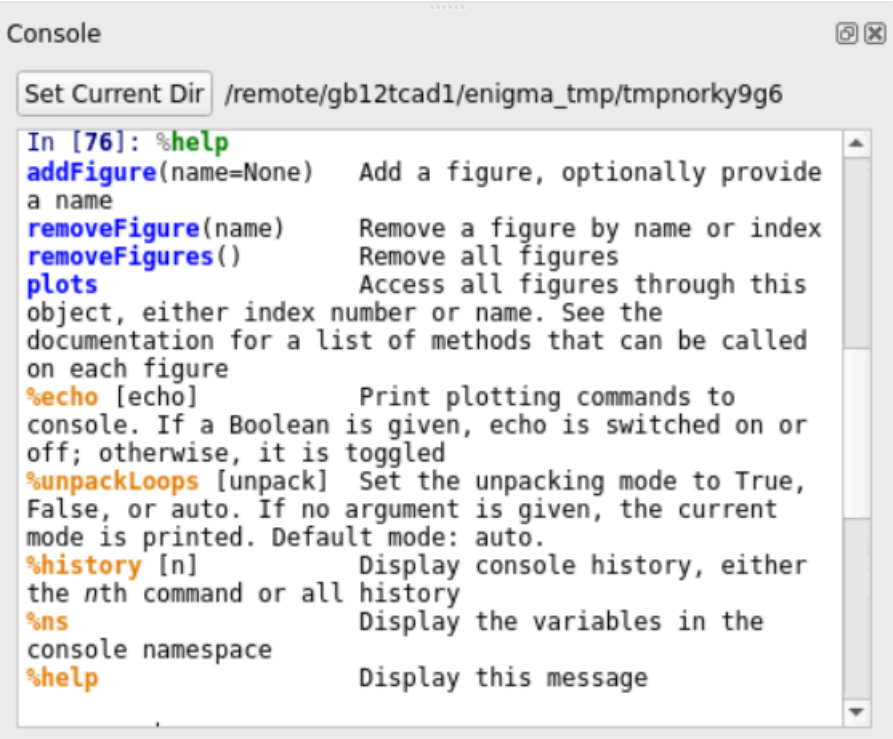
Syntax Checker with an identified warning.

Potential problems are split into two categories: errors and warnings. Errors come from problems that will definitely cause the extraction to fail, e.g. Python syntax errors. Warnings arise from problems that may cause failures but could be harmless. It's important to note that the Syntax Checker is designed to catch syntax related problems only and cannot predict failures in the extraction procedure.

Console Panel

The Mystic GUI runs commands via a built in python console contained in the console widget in the bottom left panel of the layout. The Mystic namespace described in the Extraction Environment section is automatically injected into the console when the GUI is initialized. The console receives commands from the File Editor and executes them in the context of the Mystic extraction environment. Commands can also be entered into the console executed directly. This can be useful for plotting results at the end of a strategy, making manual adjustments to the Model or debugging failures in the strategy. As well as the Mystic namespace the console also contains specific commands for creating, modifying and updating figures in the Plot Area. These commands are described further in the Creating Plots section. For a full list of plotting commands please consult the Mystic API documentation. The IPython console has a set of magic commands that provide extra functionality to the Console panel. Those commands specific to the Mystic GUI are described in the following table. IPython also provides a set of built-in magic commands. For more information, go to https://ipython.readthedocs.io/en/stable/interactive/magics.html.

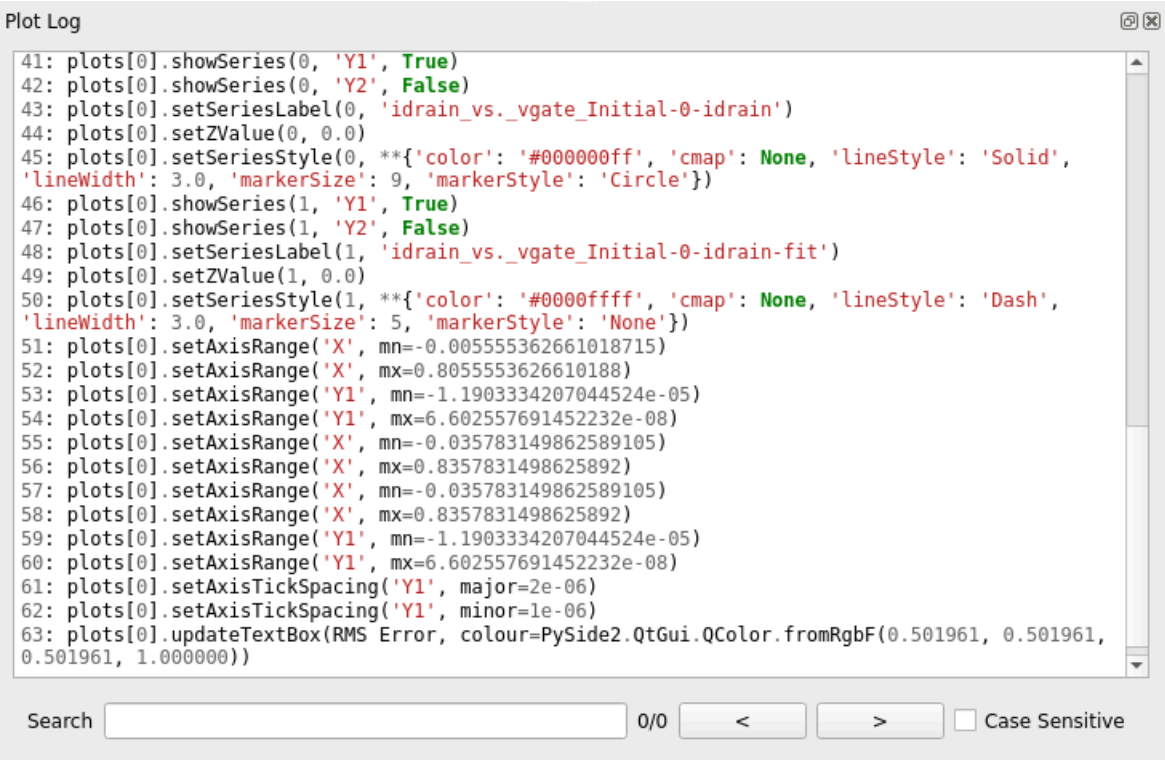
Command	Description
%echo	If this command is set, then making alterations to plots using the Mystic GUI will result in the equivalent command being printed to the Console panel.
%help	Prints a general help message, with an overview of some plotting commands and other commands (see figure below).
%history	Displays all the commands previously executed in the Console panel.
%ns	Lists the variables in the namespace.
%unpackLoops	When this command is set, for and while loops and if statements are unpacked when executing, so they can be stepped through line-by-line rather than being run as a single block.



Output of the %help command.

Plot Log Panel

The Plot Log Panel lists all the plotting commands that are being executed in the background when the plots are being created/updated b by the user through the GUI.



The Plot Log Panel.

This can be used to help understand which commands to add to the strategy in the Text Editor to automate the creating/updating of plots during the strategy execution. The panel is fully searchable using the bottom bar.

Controlling the Execution of the Strategy

The Run toolbar controls the execution of the strategy in the Console panel.



Run toolbar.

The Run/Continue button continuously executes commands until either a breakpoint or the end of the file is reached. This execution can be paused at any time by clicking the Pause button.

Note

The line that is currently being executed will run until completion first.

By default, indented code blocks such as if statements and for and while loops are treated as a single block of Python code in continuous run mode. This means that the entire block is parsed and executed as a single entity.

The Run Next Line button executes the next line of the strategy that has a yellow background in the Console panel. If the next line is the start of an indented block, then the block is unpacked, and the cursor and next line highlighting will move inside it. This means you can loop round a for or while loop multiple times, line by line, as you would in a debugger. You can adjust the rules for unpacking blocks by using the %unpackLoops command. Otherwise, to adjust the rules for unpacking block:

1. Choose Edit > Preferences. The Preferences dialog box opens.
2. In the left pane, click Run.
3. Under Loop Unpacking, select the required option.
4. Click OK or Apply

The loop unpacking rules will apply to the current session only. The Breakpoint button inserts a breakpoint at the current line where the cursor is positioned in the Editor panel. A red dot next to the line number indicates that the line has a breakpoint. When breakpoints are set, the Run/Continue button will execute only as far as the next breakpoint. You can also create breakpoints by clicking the line number in the Editor panel. The Reset button resets the entire console namespace, removing all plots and variables created or modified throughout the strategy, including the model. Click this button if the strategy is not converging as you expected, and you want to start again.

Note

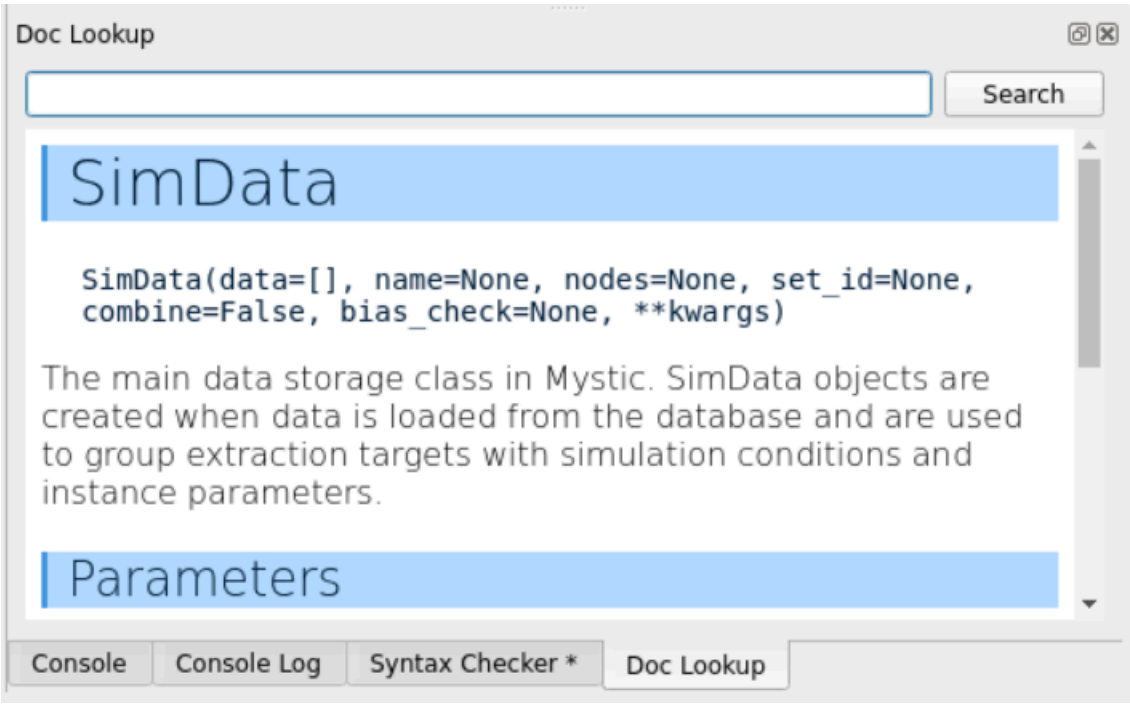
Caution: Any information saved to the database is not reset, and content written to disk is not removed.

Doc Lookup Panel

This panel shows the documentation for any Python variable. This includes all Python built-in functions and any functions, classes, modules, or variables available in the console namespace. For example, searching for SimData shows the documentation for the SimData class as shown in the figure below.

You can also search for methods of classes using dot-separated syntax, for example, `WritePLTFile()`.

You can press Ctrl+L to automatically search for the documentation related to the text directly under the cursor in either the Console or Editor panel.



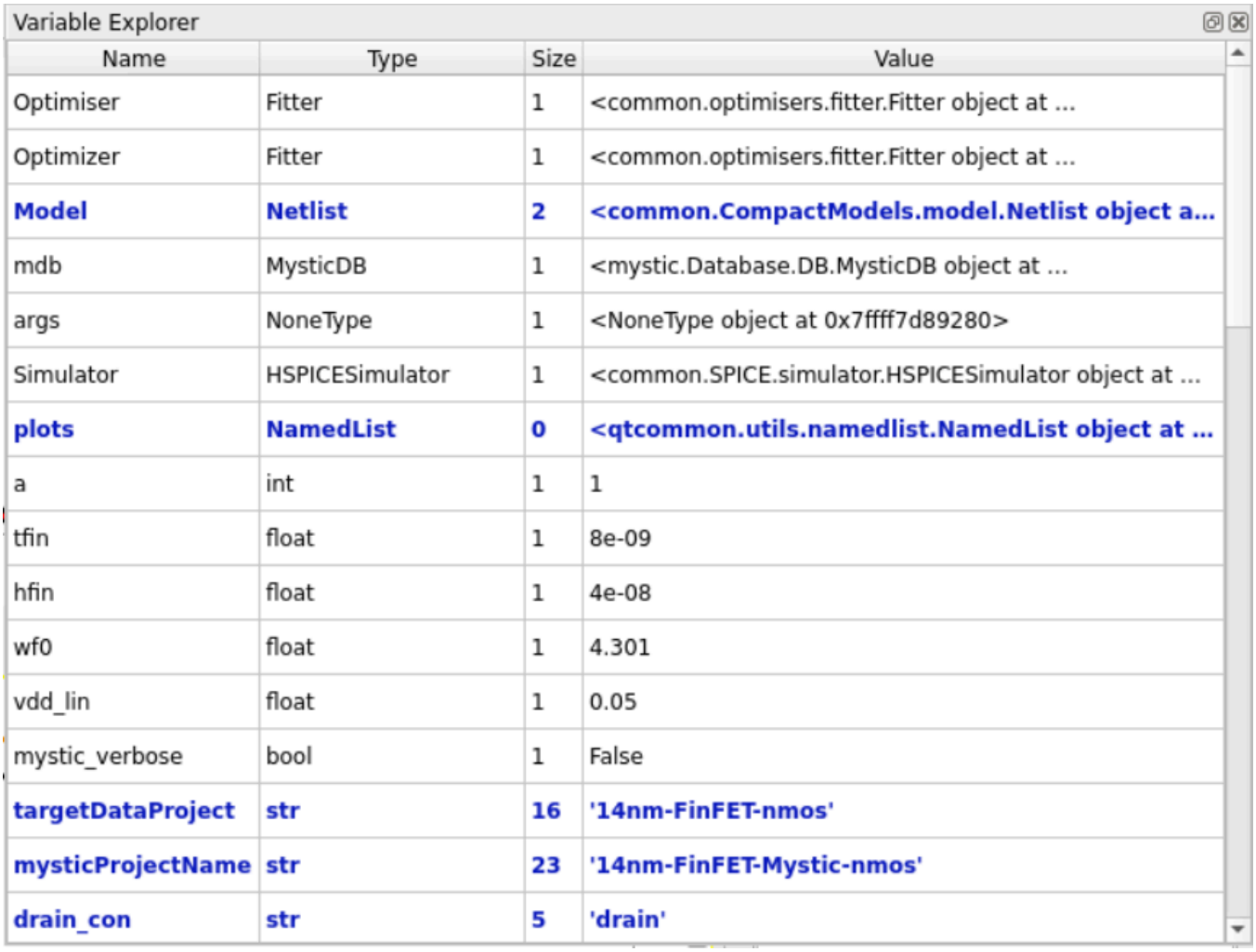
Documentation for the SimData class in the Doc Lookup panel.

Variable Explorer Panel

You can use the Variable Explorer panel to browse the Python variables currently defined in the Console panel. The variable name, type, size, and value are displayed in a table (see figure below). The list of variables grows as the strategy runs and new variables are defined.

Container-type variables highlighted in blue, such as list, dict, np.array, pd.DataFrame, `Data`, `SimData`, and `Netlist`, can be double-clicked to show a more detailed view of their contents.

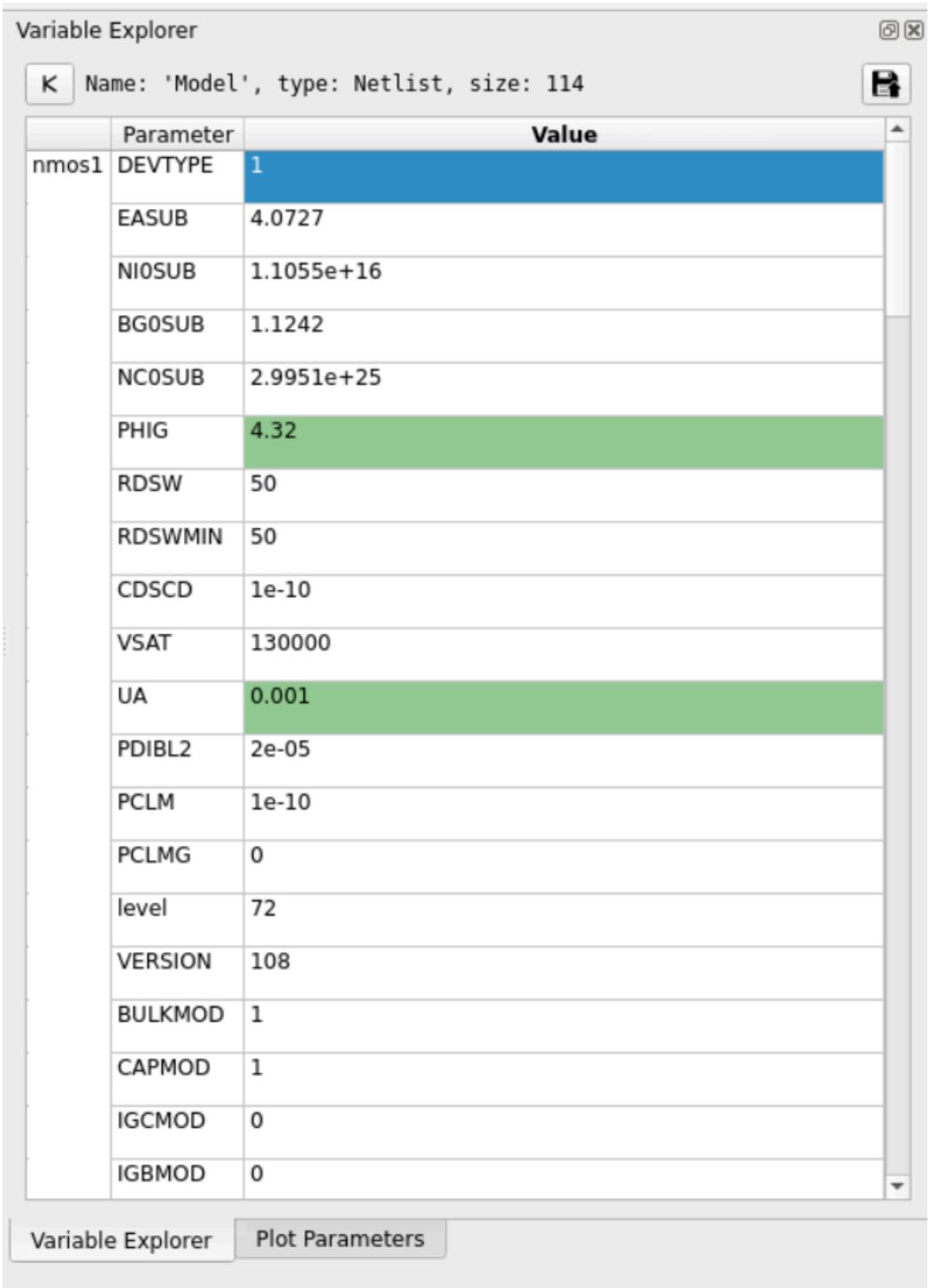
For example, double-clicking the `Model` object displays a table of all the SPICE model parameters and their current values.


Variable Explorer

Name	Type	Size	Value
Optimiser	Fitter	1	<common.optimisers.fitter.Fitter object at ...
Optimizer	Fitter	1	<common.optimisers.fitter.Fitter object at ...
Model	**Netlist**	**2**	**<common.CompactModels.model.Netlist object a...**
mdb	MysticDB	1	<mystic.Database.DB.MysticDB object at ...
args	NoneType	1	<NoneType object at 0x7ffff7d89280>
Simulator	HSPICESimulator	1	<common.SPICE.simulator.HSPICESimulator object at ...
plots	**NamedList**	**0**	**<qtcommon.utils.namedlist.NamedList object at ...**
a	int	1	1
tfin	float	1	8e-09
hfin	float	1	4e-08
wf0	float	1	4.301
vdd_lin	float	1	0.05
mystic_verbose	bool	1	False
targetDataProject	**str**	**16**	**'14nm-FinFET-nmos'**
mysticProjectName	**str**	**23**	**'14nm-FinFET-Mystic-nmos'**
drain_con	**str**	**5**	**'drain'**

Variable Explorer showing main table.

Parameters that have been modified during the extraction strategy are highlighted in green as shown in the figure below. The Model can be saved to a file at any time by clicking the Save button in the upper-right corner.



Netlist explorer view in Variable Explorer, where modified parameters are highlighted in green cells.

Double-clicking `SimData` objects shows the internal `Data` objects contained within the `SimData` object. The contents of the object can be plotted by clicking the button in the upper-right corner of the Variable Explorer panel. This is one of many ways of creating plots in the Mystic GUI. For more information, see Plot Area Panel.

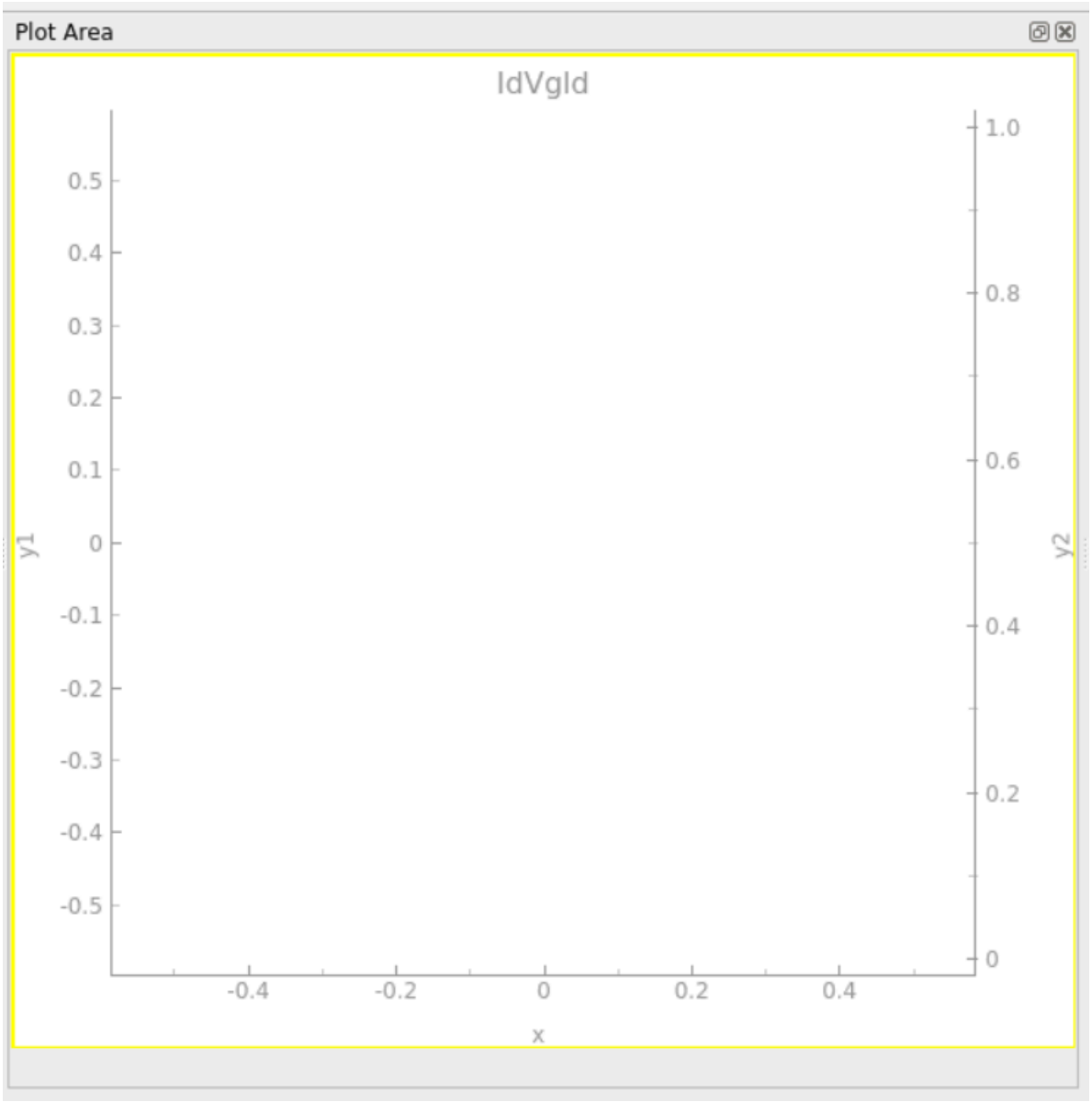
Plot Area Panel

The Plot Area panel is an interactive canvas that displays the plots created during the execution of the strategy. There are different ways to generate plots in the Mystic GUI (such as using the Variable Explorer panel) and different types of plot can be created.

This section describes how to create simple line plots in the Console panel using the built-in plotting commands and Mystic `SimData` objects. Plots can also be generated automatically and updated by the Mystic tool as it runs an extraction (see Running a Strategy Interactively). You can create a plot by entering the addFigure command in the Console panel as follows:

```
fig = addFigure('IdVgId')
```

This command creates an empty plot in the Plot Area panel (see figure below) and returns a FigureWidget object that can be used to populate the plot with data.



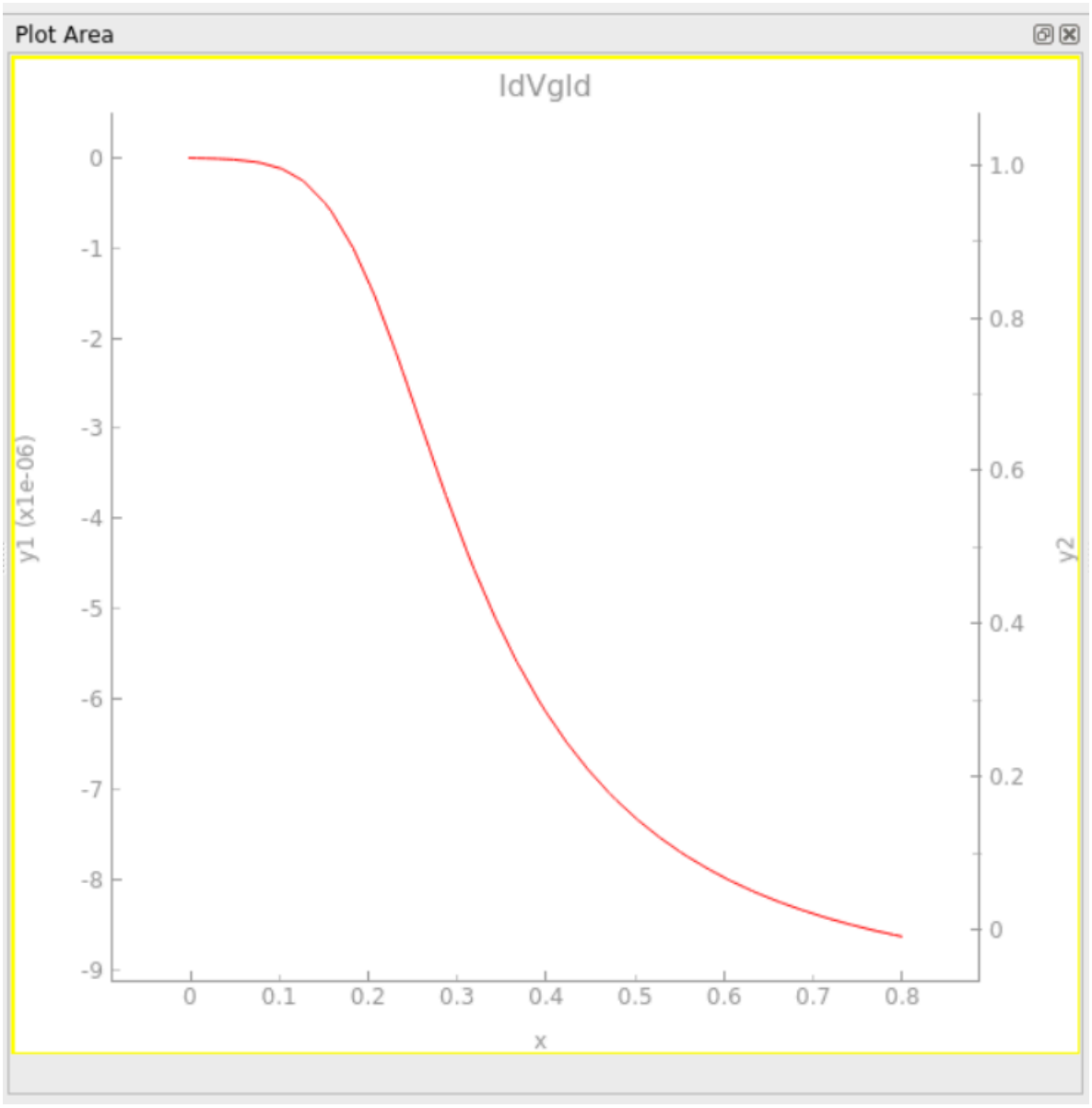
Plot Area panel showing empty plot.

The empty plot has a left and right y-axis allowing you to display two different data series on the same plot or to apply different scales to each axis, for example, a log10 scale. For more information about the axis properties, see Plot Parameters Panel.

Before adding data, first, you must decide which data you want to visualize. For this example, assume that a low-drain Id–Vg `SimData` object has been loaded into the environment already using the mdb.Load command. This data can be plotted directly using the FigureWidget.plot command as follows:

```
series = fig.plot(IdVgld)
```

The contents of the `SimData` object are plotted as shown in the figure below.

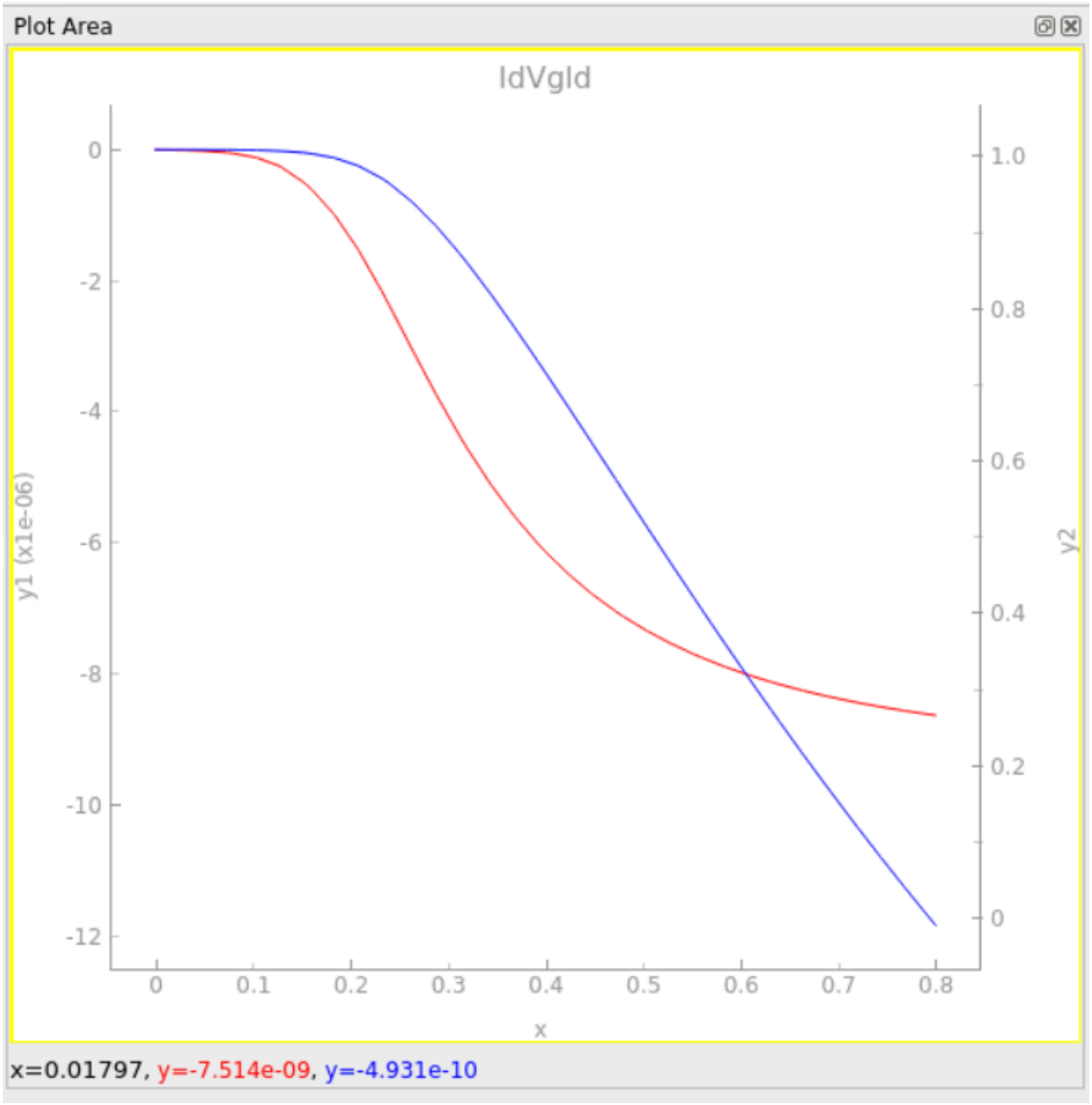


Simple low-drain Id–Vg line plot.

The FigureWidget.plot command returns a PlotSeries object, which can be used to update the curve properties. You can also do this by using the Plot Parameters panel (see Plot Parameters Panel). `SimData` objects should contain both target data and fit data, and so two series, IdVgld0 and IdVgld1, have been created in the Plot Parameters panel, but only one curve has been displayed on the plot. In this example, the IdVgld object has been freshly loaded from the database, so no fit data is present. You can calculate an initial error and update the data on the plot as follows:

```
ExtractionUtils.PrintErrors("IdVgld", IdVgld, Simulator,
                            Model, ErrorMethod)
series.updateData(IdVgld)
```

The fit data should now appear on the plot as shown in the figure below.



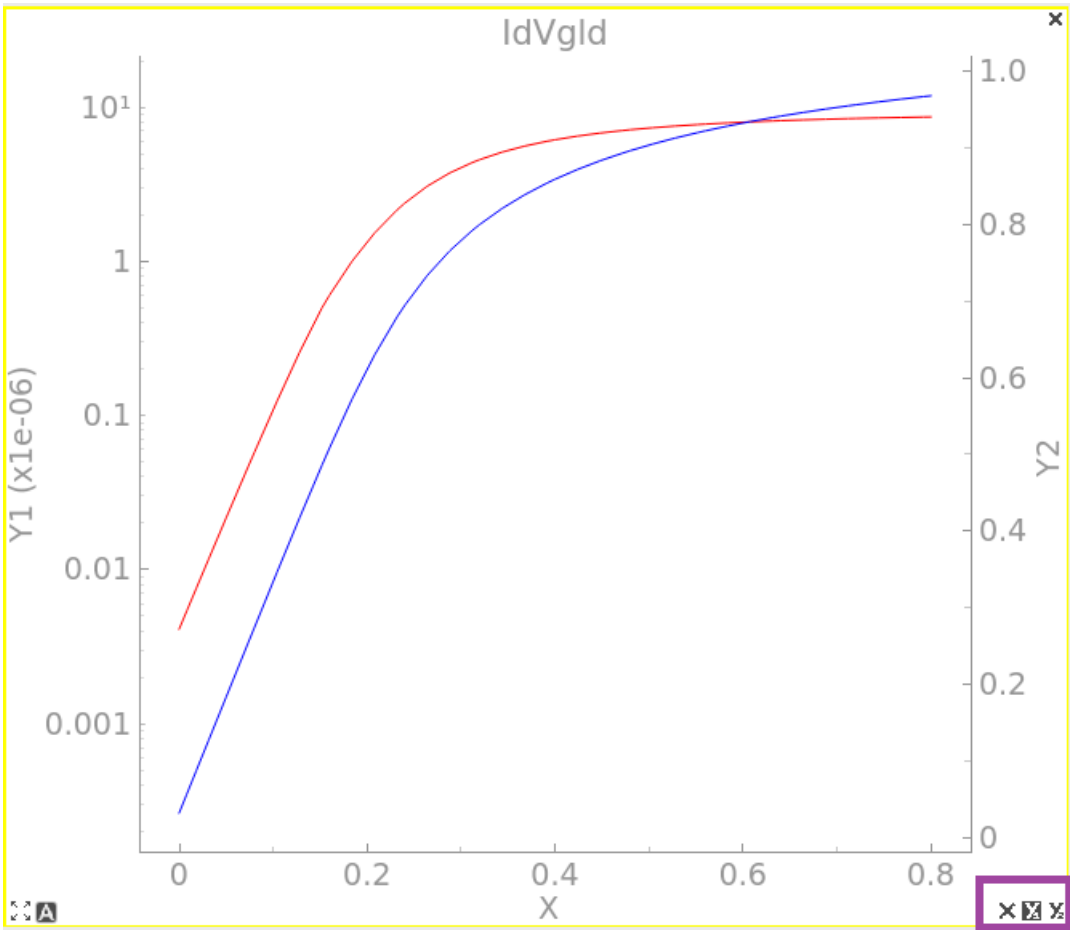
Line plot for low-drain Id–Vg data showing both target data and fit data.

You can select a series on the plot by clicking it, which highlights both the series and its associated menu in the Plot Parameters panel. There are several zoom and pan operations you can apply to a plot:

- To uniformly zoom in to or out of a plot, use the mouse wheel.
- For a free zoom that affects each axis independently, right-click and drag in any direction.
- Moving left and right zooms in to the x-axis, and moving up and down zooms in to the y-axes.
- To zoom only one axis, move the cursor on to that axis, and either use the mouse wheel or hold the right mouse button and drag along the direction of the axis.
- To pan around the data freely, hold the left mouse button and drag on the plot.
- To pan a single axis, hold the left mouse button on the axis and drag along the direction of the axis.

- At any point, you can auto-scale the plot and return to the initial zoom by clicking the A button in the lower-left corner of the plot.

Plot axes can be log transformed directly from the plot using the three buttons in the bottom right-hand corner of the plot as shown in the figure below.



Line plot IdVg data with a log 10 transform applied to the left-hand y-axis.

These transforms can also be applied from the Plot Parameters panel as shown in Plot Parameters Panel.

To remove a series from a plot, use the FigureWidget.removeSeries command. Plots can be removed from the Plot Area panel by using the removeFigure or removeFigures commands as follows:

```
removeFigure('IdVgId')
```

For a more comprehensive description of all the plotting commands, consult the Mystic GUI API Reference section. When multiple plots are added to the Plot Area panel, they are arranged in a grid fashion, which you can scroll through. By default, a 2×2 grid of plots is visible. If more plots are added, then a scroll bar appears, allowing you to move between plots. To adjust the number of plots that are shown without scrolling:

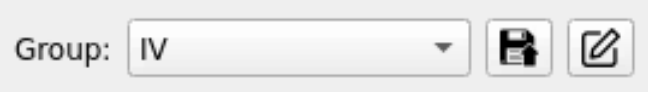
1. Choose Edit > Preferences. The Preferences dialog box opens.
2. In the left pane, click Plots.

3. Under Plot Grid, select Auto Adjust. With this option selected, the number of visible plots changes automatically as the size of the main window changes, in order to show the plots with the best aspect ratio.
4. Click OK or Apply.

You can make a plot full screened by clicking the four arrows in the lower-left corner of the Plot Area panel. When the Mystic GUI is full screened, you can cycle through the plots using the tabs on the Plot Parameters panel.

When adding a new figure, an optional plotGroup parameter can be passed. This creates a new tab in the Plot Area panel with the given name. All subsequent figures created with the same plotGroup are added to that tab.

You can view groups by switching between tabs in the Plot Area panel or by using the Group list in the Plot Parameters panel as shown in the figure below. You can edit groups by clicking the Edit button. Here, you can drag plots between groups. The Add and Remove buttons create a new group or remove the currently selected group, respectively. If a nonempty group is removed, then its plots are moved to the default group. When you are finished editing, click the Back button to return to the Plot Parameters panel.

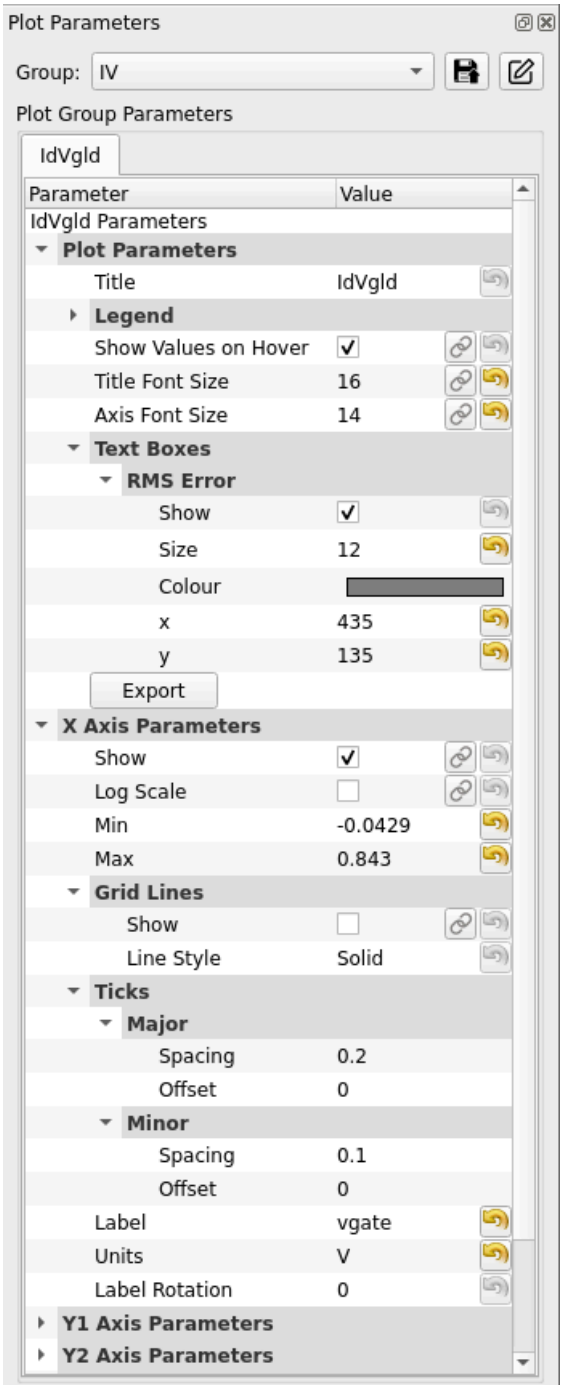


Viewing groups using the Group list in the Plot Parameters panel.

The Save button next to the Group list exports all plots in the current group. A dialog box opens and all the plots in the group are exported to the selected directory as PNG files.

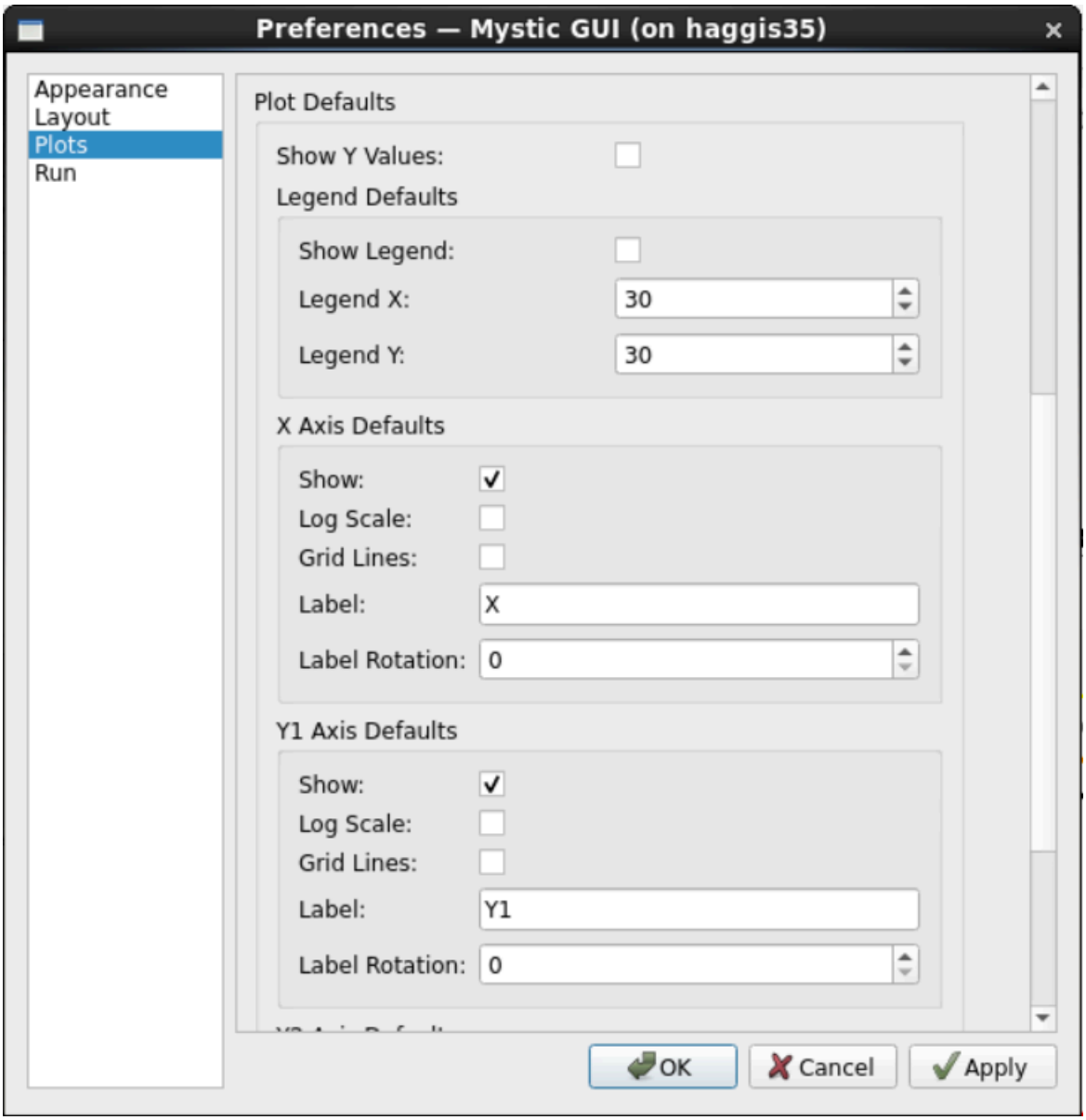
Plot Parameters Panel

The Plot Parameters panel allows you to update the properties of all plots in the Plot Area panel. One tab on the Plot Parameters panel is created per plot. When a plot is first created, a new tab is created in the Plot Parameters panel titled with the name of the plot. Before a curve is created, the tab contains menus for plot parameters and axis parameters as shown in the figure below.



Plot Parameters panel showing a tab for an empty plot.

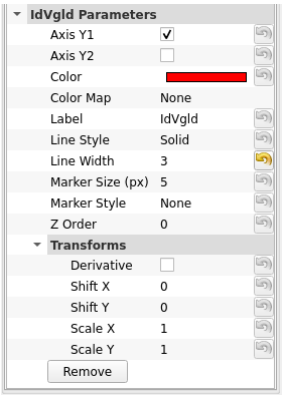
The default values of these parameters can be configured in preferences. To open the Preferences dialog box, choose Edit > Preferences. Then, click Plots in the left pane of the dialog box (see figure below).



Default values for plot and axis parameters.

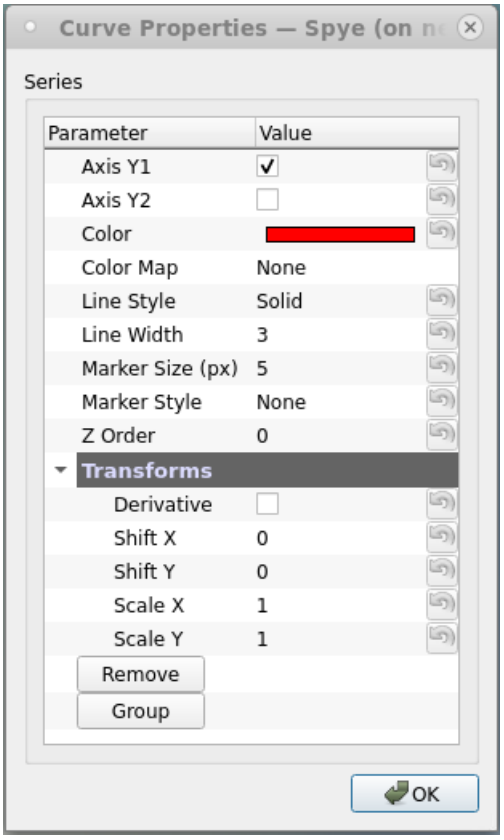
Some of the parameters feature a link icon on the right-hand side. This can be used to link the state of the parameter across all plots in a group. For example, if you have a group featuring multiple IV curves, you can set the Y1 Axis > Log Scale parameter in one plot then link it across all the plots. Once this is done, any change in this parameter in any of the plots will propagate all other IV plots in the group.

When the tab for a plot is selected, the corresponding plot in the Plot Area panel has a yellow border. When a series is added to the plot, a new menu is created for that series. The contents of the menu depend of the type of series that has been plotted. The figure below shows the parameter structure for a standard scatter/line series.



Plot Parameters panel showing menu for a scatter/line series.

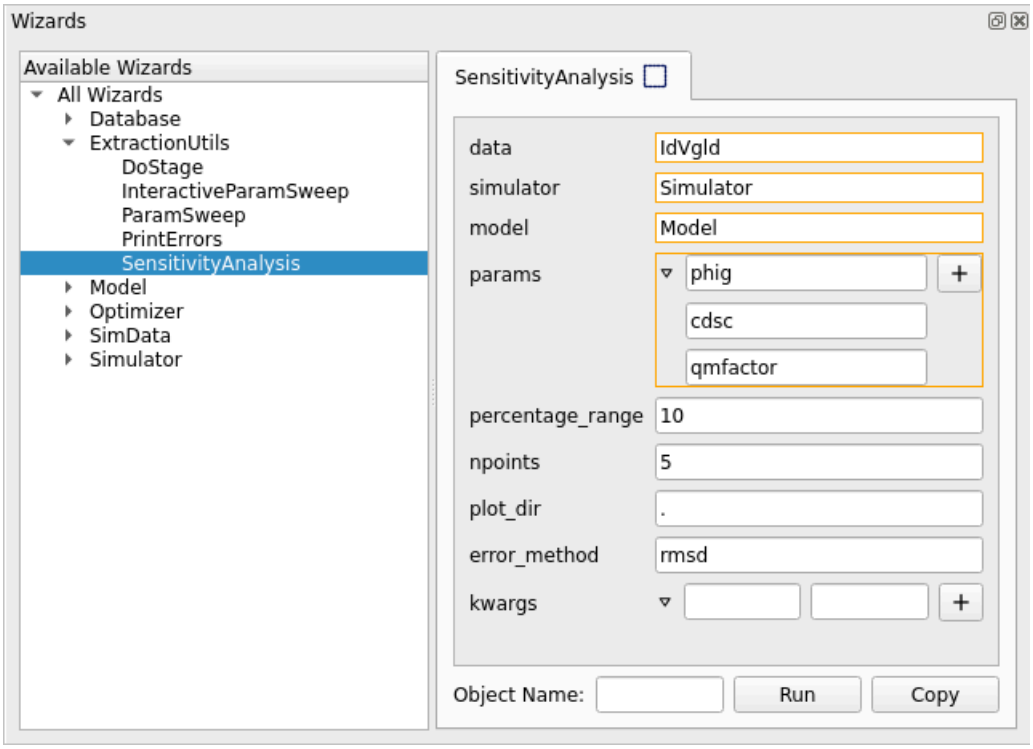
Changing any of these fields updates the series on the plot immediately. The menu for a series opens automatically when you click a series on the plot. You can edit the properties of multiple curves simultaneously by highlighting multiple menus, and then right-clicking and choosing Curve Properties. Any properties changed in the Curve Properties dialog box apply to all of the selected curves (see figure below).



Curve Properties dialog box for editing the properties of multiple curves simultaneously.

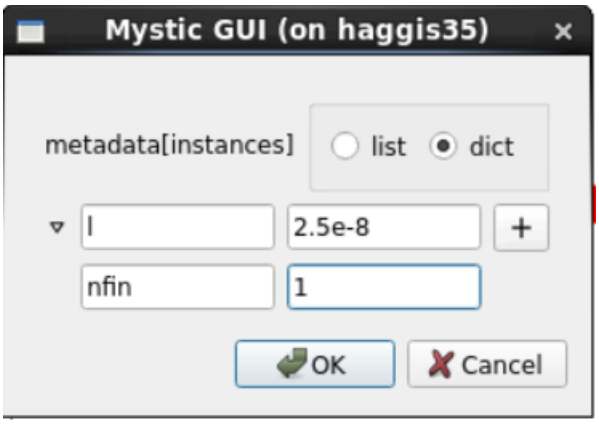
Wizards Panel

The Wizards panel facilitates many common Mystic operations by either adding commands to the Editor panel or executing commands directly in the Console panel. The left pane of the panel shows a categorized tree view of the available wizards. When a wizard is opened (by clicking its name in the left pane), it is added to a tab on the right side of the panel. The figure below shows the Wizards panel with the Sensitivity Analysis wizard opened.



Wizards panel displaying available Mystic wizards in left pane; Sensitivity Analysis wizard is opened and populated with arguments.

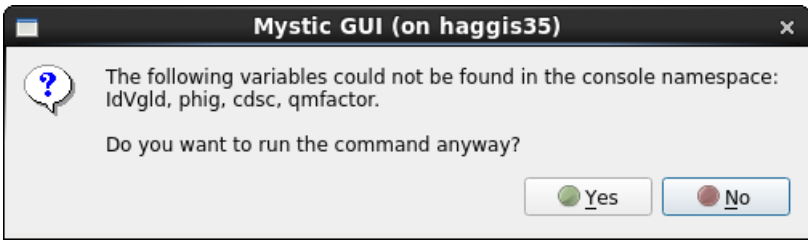
Each wizard has the same general structure: a table of argument names and input forms that you can fill out. Some of these are self-explanatory, for example, a text box or Boolean option. List and dictionary argument types have + buttons next to their input boxes to add a new entry to the list or dictionary. These can then be collapsed or expanded using the arrow on the right-hand side. Lists and dictionaries can also be nested by right-clicking the input box and choosing Create list or dict. This opens a dialog box where you can enter new data as shown in the figure below.



Nested dictionary dialog box.

Some arguments can also be references to other Python variables defined in the Console panel, for example, the DoStage utility function requires a Model and Simulator object. In this case, the text entered in the box should be the name of the variable as defined in the Console panel. When the arguments have been defined, you can copy the resultant command (click Copy) and then paste it into the Editor panel (press Ctrl+V), or you can executed the command directly in the Console panel by clicking the Run button. If the function to be executed has a return, then you can use the Object Name box to specify the name of the returned variable. When you click the Run button, the arguments are cross-referenced with the console to

ensure all Python variables defined in the argument table exist in the console namespace. If any variables are not defined, then a dialog box opens and you are prompted to confirm whether to proceed, as shown in Figure 39.



Dialog box listing missing variables: in the Sensitivity Analysis example, phig, cdsc, qmfactor, and IdVgld are missing.

Usage Examples

This section describes some common uses of the Mystic GUI in realistic Mystic examples, which focus specifically on plotting rather than the strategy itself. For more information and access to demonstration projects, contact your local TCAD support team.

Running a Strategy Interactively

When running a strategy in the Mystic GUI, you can monitor the quality of the fit of the model against the target data in real time as the strategy progresses. You can do this by making some minor updates to an existing strategy file. First, you must identify the target data you want to monitor throughout the strategy. This is usually the target against which the best overall fit is required by the end of the strategy. The identified target or targets can then be tagged as interactive as follows:

```
IdVg = mdb.Load(ivar="vgate", dvar="idrain",
                project=targetProjectName)
IdVgld = IdVg.FilterData('eq', 0.05, 'bias.drain')
IdVgld.interactive = True
IdVgld.name = 'IdVgld'
```

One plot is created in the Plot Area panel for each piece of interactive data, using the given name containing both the target data and fitted data. For example, if Id–Vg and Id–Vd data are both present and tagged as interactive, then two separate plots are created that are both updated in real time. In this case, you are working on a low-drain Id–Vg strategy, so you have only one target.

As mentioned in Plot Area Panel, figures can be divided into different plot groups. This feature can also be used when running in interactive mode by setting the plotGroup attribute of the SimData object. This can be used to split interactive data into different categories, for example, separating I–V and C–V data.

As the strategy runs, the interactive targets are updated at the end of every optimization stage (`ExtractionUtils.DoStage()`), even if the interactive object is not the target for that stage. A snapshot of an interactive target can be taken at any point throughout the extraction by calling `ExtractionUtils.PrintErrors()` with that target. The label passed to `ExtractionUtils.PrintErrors()` dictates how the snapshot is displayed. If the label passed matches the name of the object, that is:

```
ExtractionUtils.PrintErrors("IdVgld", IdVgld, Simulator,
                            Model, ErrorMethod)
```

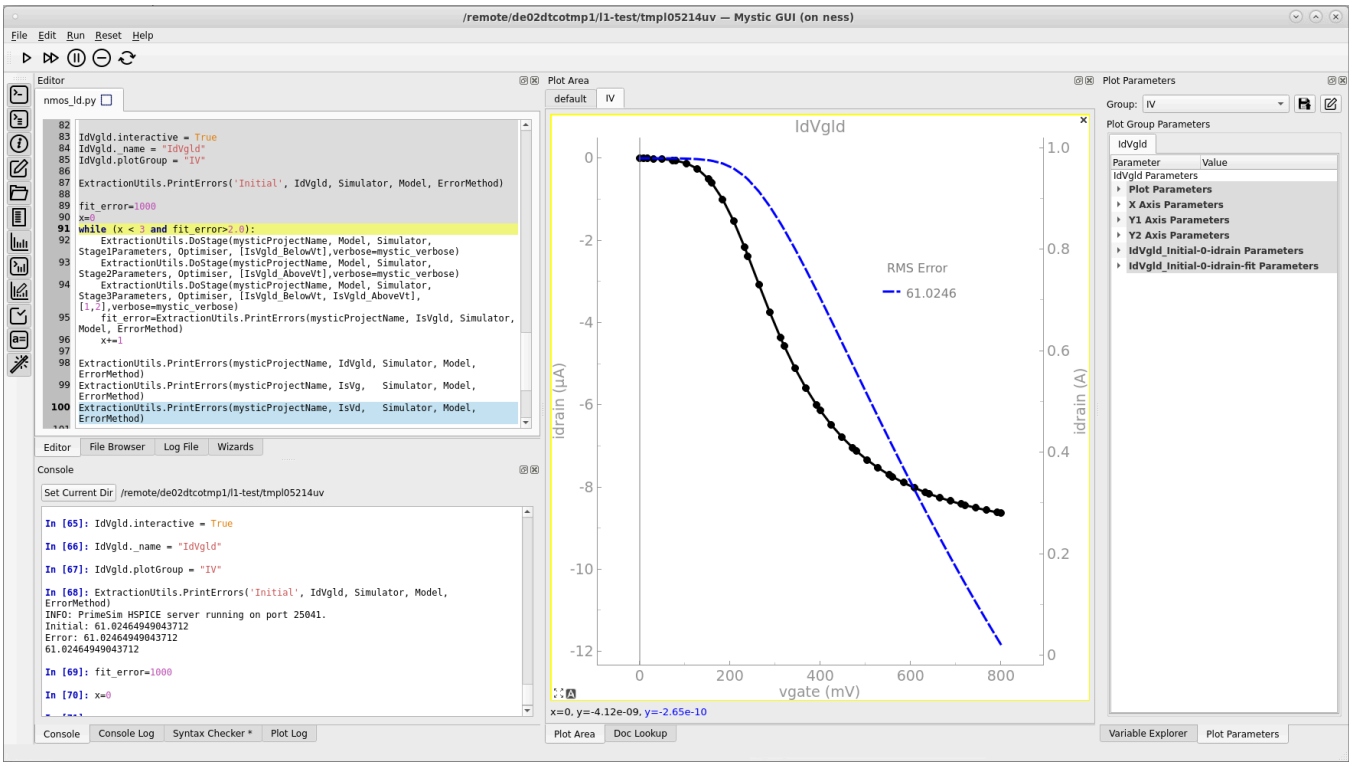
then the existing fit data is updated. The next time that `ExtractionUtils.DoStage()` is called, this snapshot is overwritten by new results. If the label does not match the name of the object, that is:

```
ExtractionUtils.PrintErrors("Initial", IdVgld, Simulator,
                            Model, ErrorMethod)
```

then a new series is created on the plot with the specified label. This series remains unchanged for the duration of the extraction unless it is updated again using `ExtractionUtils.PrintErrors()`. This can be used to create snapshots of certain conditions, such as the initial fit or the best overall fit that has been achieved so far. Providing a visual reference point for the remainder of the extraction, consider the following low-drain Id–Vg example strategy:

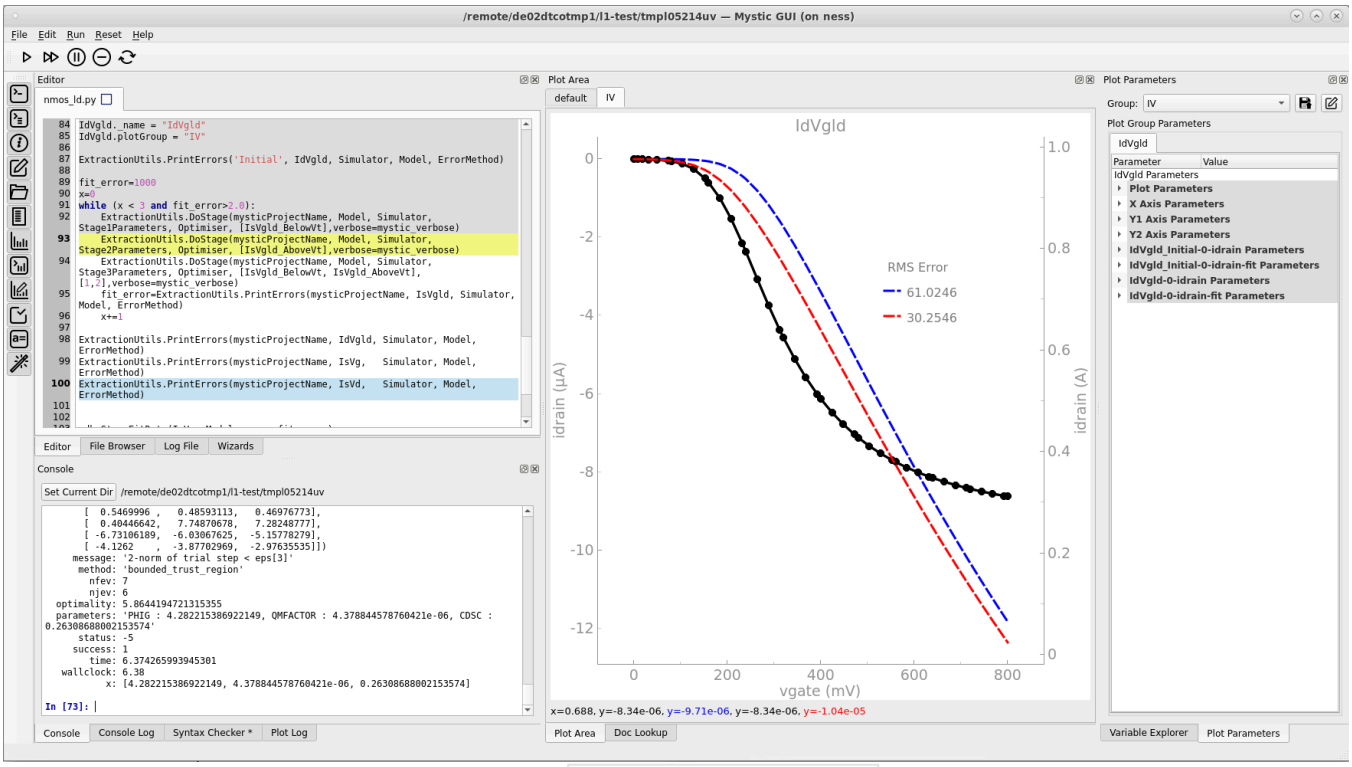
```
ExtractionUtils.PrintErrors("Initial", IdVgld, Simulator,
                            Model, ErrorMethod)
fit_error=1000
x=0
while (x < 3 and fit_error>2.0):
    ExtractionUtils.DoStage(mysticProjectName, Model, Simulator,
                            Stage1Parameters, Optimizer,
                            [IsVgld_BelowVt])
    ExtractionUtils.DoStage(mysticProjectName, Model, Simulator,
                            Stage2Parameters, Optimizer,
                            [IsVgld_AboveVt])
    ExtractionUtils.DoStage(mysticProjectName, Model, Simulator,
                            Stage3Parameters, Optimizer,
                            [IsVgld])
    fit_error=ExtractionUtils.PrintErrors("IsVgld", IsVgld, Simulator,
                                          Model, ErrorMethod)
    x+=1
```

In this example, an initial snapshot labeled “Initial” is created to retain a visual reference to the starting point of the extraction. Going through the example, line by line, you can see what is produced. The figure below shows the plot after the first call to `ExtractionUtils.PrintErrors()`.



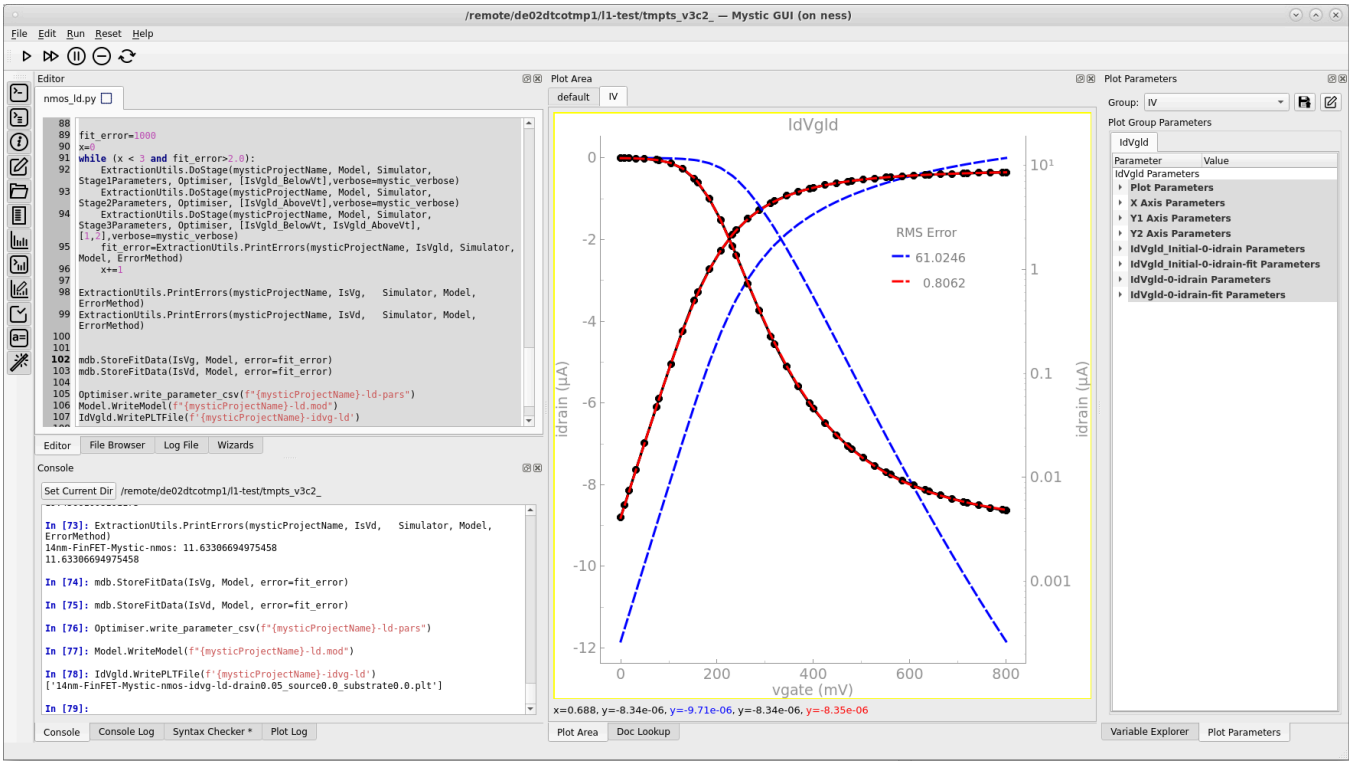
Snapshot of initial conditions of the SPICE model fitting.

The initial conditions are represented by the blue curve. There is also an RMS error value between the target a fitted curves. This can be enabled or disabled through the Plot Parameter panel. Moving passed the first call to `ExtractionUtils.DoStage()`, you see the plot shown in the figure below.



Real-time fit of the model after the first call to `ExtractionUtils.DoStage()`; initial conditions are shown by blue curve, and current fit is shown by red curve.

Now a red curve has been created showing the current status of the fit. The blue curve of the initial conditions remains on the plot. Now, you can run to the end and watch the extraction progress. The figure below shows the final result of the strategy. You can see that the red fit curve has been updated to reflect the current fit of the model. Now that the strategy has completed, the IPython console remains open for you to further analyze the results, rerun stages, or manually adjust SPICE model parameter values.



Final result of low-drain Id–Vg strategy.

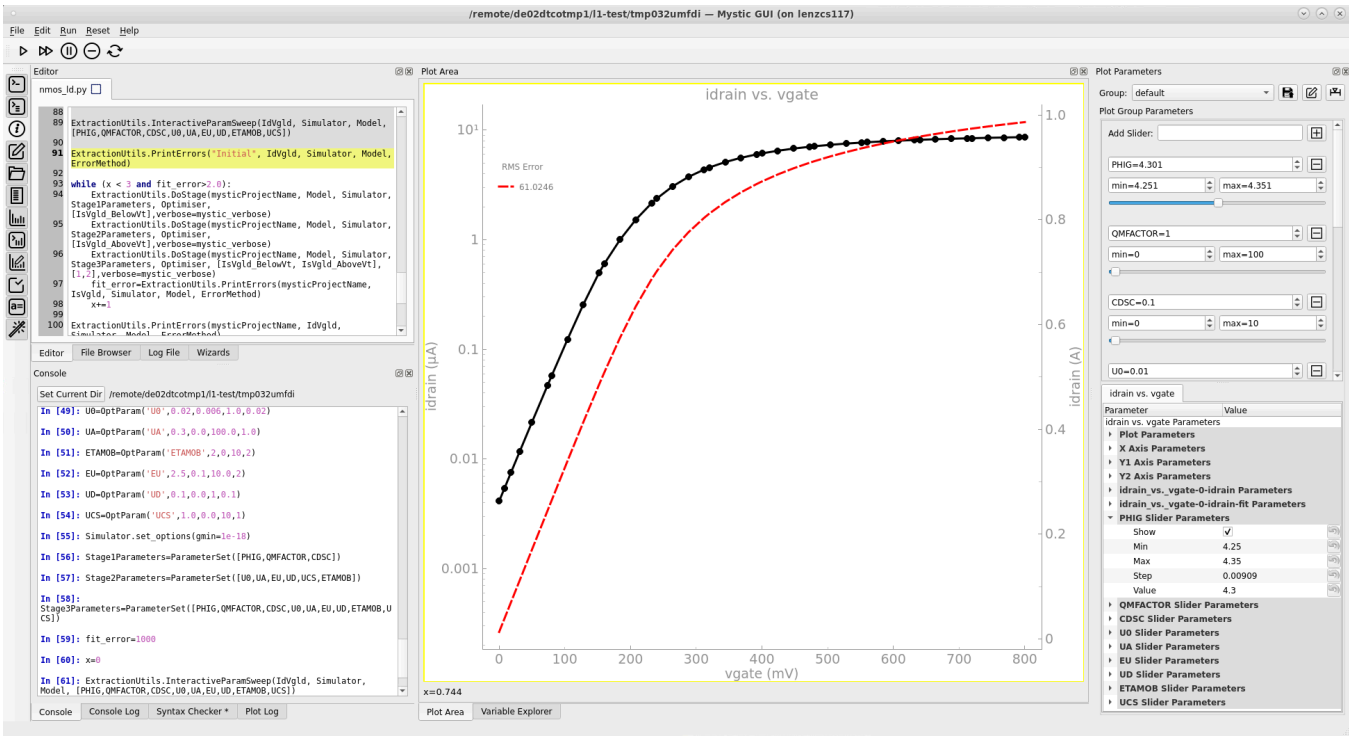
Exploring Parameters Interactively

When developing a strategy from the beginning, you can use the Mystic GUI to investigate parameter sensitivity interactively. The `ExtractionUtils.InteractiveParamSweep()` utility takes a single `SimData` target and a list of SPICE model parameters and creates an interactive plot where you can adjust parameter values using sliders and watch the response of the model in real time on the plot.

The following example shows an interactive parameter sweep for low-drain Id–Vg data:

```
pmax = {"QMFACTOR": 1.0,
        "CDSC": 0.2,
        "U0": 0.1,
        "UA": 10.0,
        "UD": 0.2}
ExtractionUtils.InteractiveParamSweep(IdVgId, Simulator, Model,
                                      [PHIG,QMFACTOR,CDSC,U0,UA,EU,UD,ETAMOB,UCS],
                                      pmax=pmax, num=101)
```

The optional pmax dictionary provides the upper limit of the slider for the parameters. If this is not provided, or certain parameter names do not exist in the dictionary, then the maximum bound supplied in the OptParam definition is used. Minimum bounds can also be supplied using an optional pmin dictionary. The number of levels between the minimum and maximum bounds of the parameters is determined by the num argument. The figure below shows the interactive plot produced by `ExtractionUtils.InteractiveParamSweep()`.



Interactive parameter plot for low-drain Id–Vg data.

When you move each slider, the Mystic tool runs a backend simulation to evaluate the new set of parameters for the given target conditions and updates the plot accordingly. The interactive parameter plot can be used to manually fit a set of parameters for the model or to provide guidance in assembling an extraction strategy for the model based on the sensitivity of parameters to certain sections of the target data. As can be seen in the right-hand side of the above plot, each slider has adjustable boxes for is current, minimum, and maximum value. The value box can be used to set values for the parameter that are not included in the step resolution of the slider, while the minimum and maximum boxes can be used to adjust the sliders range. When the range is changed the step of the slider will be automatically adjusted to one one-hundredth of the range. The step can also be adjusted manually in the dropdown menu for the slider in the Plot Parameters panel below. Sliders can be added by entering the name of an existing OptParam in to the Add Slider box and removed by clicking the minus symbol next to the slider to be removed. When you are satisfied with the quality of the fit, or you want to take a snapshot of the current model parameter values, you can use the model explorer in the Variable Explorer panel to save the model to disk.

Parallel Coordinates Plot

When using the Mystic tool to extract geometry- or temperature-dependent models, the number of data points you are fitting across increases significantly, making it difficult to assess the quality of the model fit across all conditions using standard line plots. You can use parallel coordinate plots to effectively visualize and compare higher-dimensional datasets.

This example demonstrates how to use the Mystic GUI to visualize a SPICE model fitted for a 14 nm FinFET device across multiple gate lengths and fin widths. It assumes the model has been prefitted to a good standard and the example focuses on plotting.

To learn more about fitting across multiple geometries or temperatures using the Mystic tool, contact your local TCAD support team.

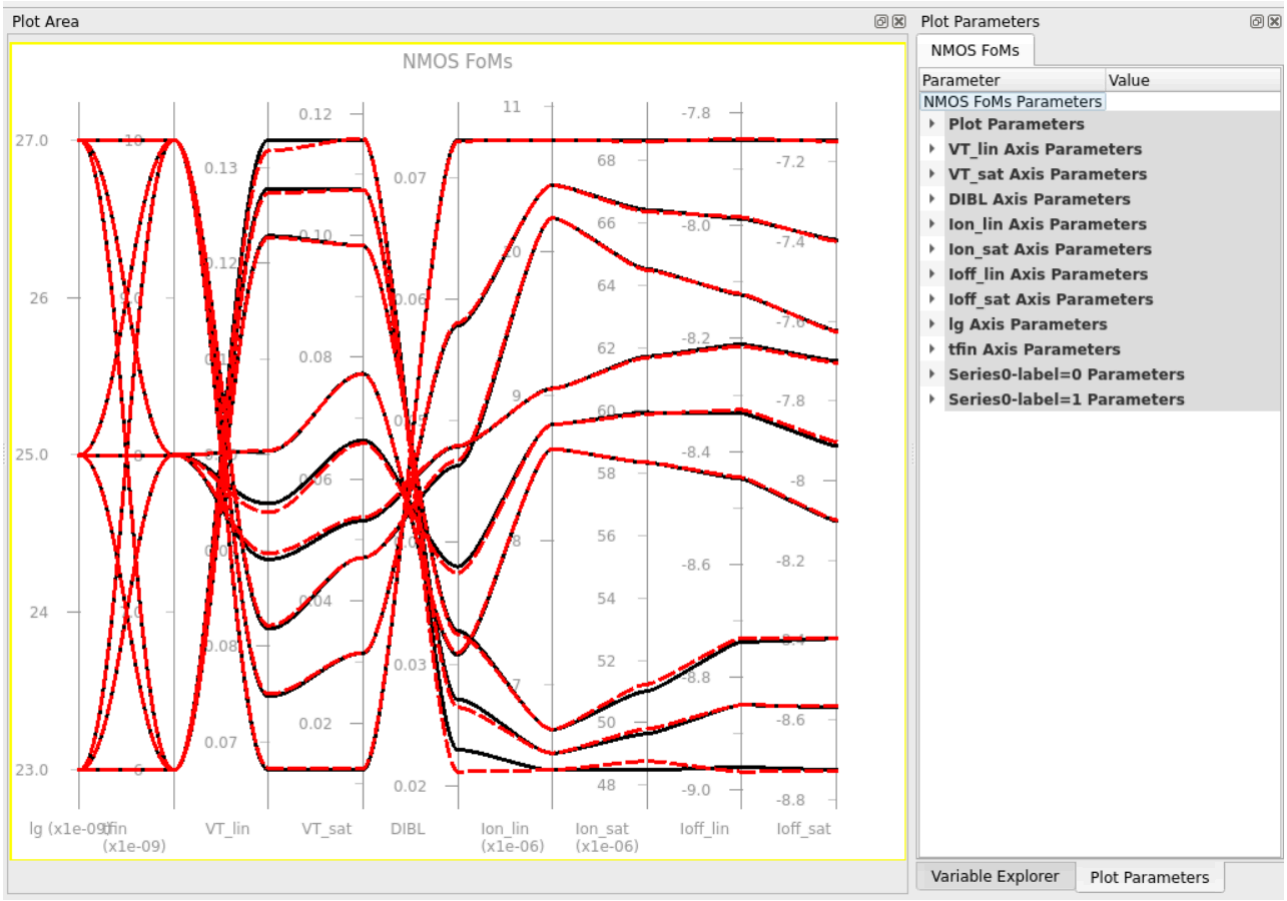
The starting point of this example is two `SimData` objects, IdVgld and IdVghd, representing low-drain and high-drain geometry-dependent data, respectively, for the 14 nm FinFET device. As the model is prefitted, both target data and fit data are already present. A parallel coordinate plot requires a set of features that can be represented as single points for each geometry. For this FinFET device, you want to evaluate the fit at these metrics: Ioff-lin, Vt-lin, Ion-lin, Ioff-sat, Vt-sat, and Ion-sat. To do this, you must first calculate them using a figure of merit (FoM) transform as follows:

```
ld_foms = IdVgld.MakeFoMTarget(['ioff','vt','ion'])
hd_foms = IdVghd.MakeFoMTarget(['ioff','vt','ion'])
```

Now, you can create the plot and pass the FoM data to the parallel coordinates plotting function:

```
fig = addFigure('NMOS FoMs')
series = fig.plotParallelCoordinates([ld_foms, hd_foms],
                                     suffix=['_lin','_sat'])
```

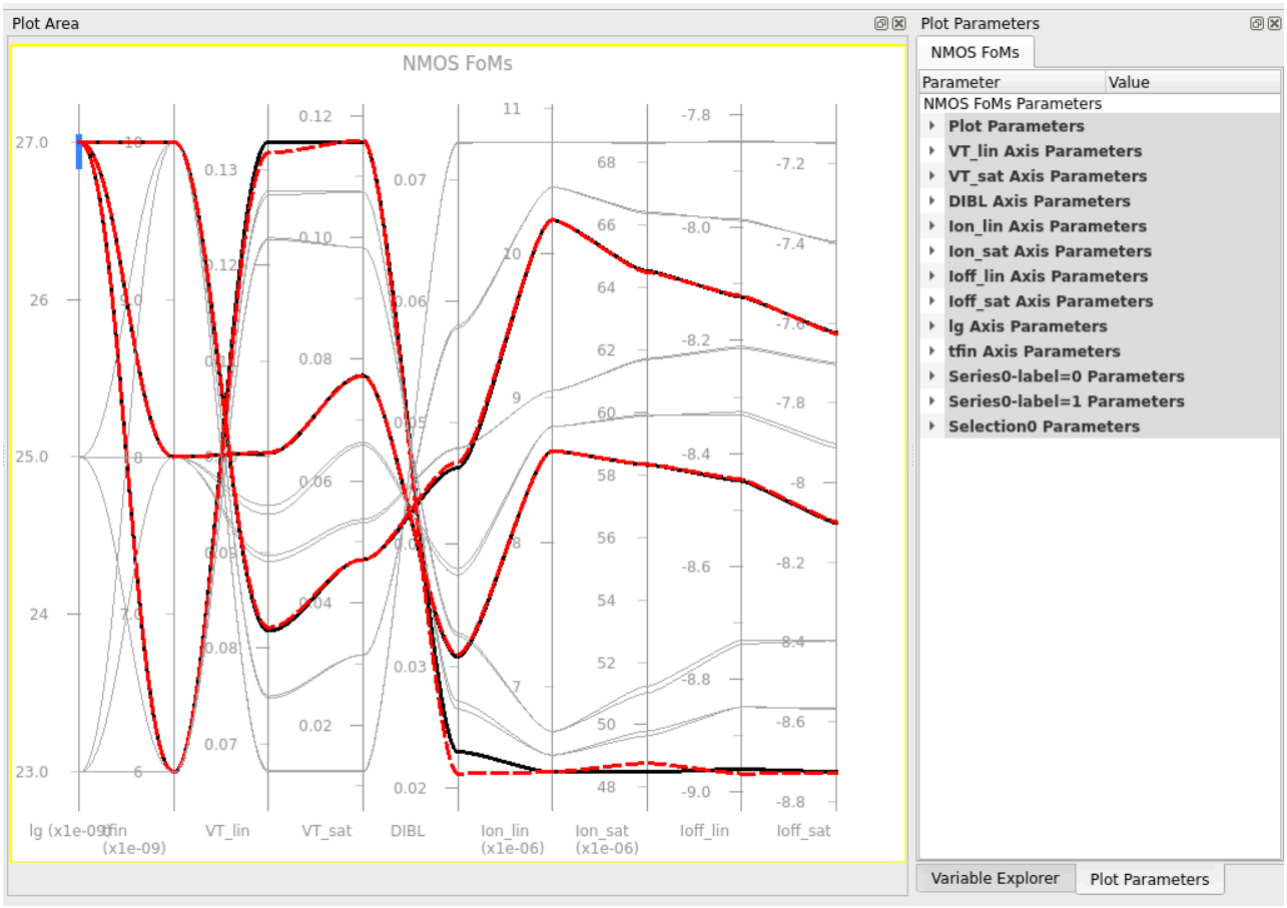
The figure below shows the resulting parallel coordinate plot.



Parallel coordinate plot of geometry-dependent fitting for a 14 nm FinFET device: black=target data and red=fit data.

One series is created for the target data (black) and one is created for the fit data (red). You can see from the figure above that the model fits well across most metrics. One of the main strengths of parallel coordinate plots is their ability to relate trends in the output metrics to changes in the inputs. When a large

amount of data is being analyzed on the plot, it can quickly become difficult to track these patterns in the data. In the Mystic GUI, the visible data on the plot can be filtered, based on selection criteria on each axis. To add a selection, hold the Shift key and drag one point of an axis to a new point. This filters out the data outside of the selection region. The figure below shows a selection filter placed at 27 nm on the Lg axis. The data at all other gate lengths is gray. The selection can be extended or moved at any time by holding the Shift key while dragging the selection line or by using the selection menu that has been added to the Plot Parameters panel. Multiple selections can be applied to any of the axes to further filter the data based on as many selection criteria as you want.



Parallel coordinates plot with a selection added at Lg=27 nm.

RF Smith and Polar Charts

As described in RF Extraction, RF devices require the scattering parameters (S-parameters) to be captured accurately. S-parameters come in the form of a 2×2 matrix with both real and imaginary parts and are most commonly plotted on a Smith and Polar charts. You can create Smith and Polar charts to compare target and fit S-parameter data for an RF device extraction. The reflection coefficients (S11 and S22) are normally viewed on a Smith chart and the forward and reverse gains (S12 and S21) are normally viewed on a Polar chart. This example focuses on how to plot S-parameter data that has been prefitted. For a full demonstration flow of an RF SPICE model extraction, contact your local TCAD support team. The starting point for this example is a `SimData` object named *S_data* that contains a set of S-parameters at frequencies ranging from 27.7 GHz to 28.2 GHz as shown in the figure below.

frequency	s(11)re	s(11)i	...	s(22)i-fit	errors
2.772000e+10	4.686165e-01	-7.190487e-01	...	-3.525271e-01	2.539432e-02
2.772600e+10	4.672343e-01	-7.194720e-01	...	-3.527785e-01	2.540475e-02
2.783200e+10	4.658533e-01	-7.198920e-01	...	-3.530283e-01	2.541532e-02
2.788800e+10	4.644735e-01	-7.203088e-01	...	-3.532768e-01	2.542601e-02
2.794400e+10	4.630949e-01	-7.207224e-01	...	-3.535237e-01	2.543684e-02
2.800000e+10	4.617175e-01	-7.211327e-01	...	-3.537692e-01	2.544779e-02
2.805600e+10	4.603414e-01	-7.215398e-01	...	-3.540132e-01	2.545888e-02
2.811200e+10	4.589664e-01	-7.219437e-01	...	-3.542558e-01	2.547011e-02
2.816800e+10	4.575927e-01	-7.223444e-01	...	-3.544970e-01	2.548147e-02
2.822400e+10	4.532203e-01	-7.227419e-01	...	-3.547367e-01	2.549297e-02
2.828000e+10	4.548490e-01	-7.231362e-01	...	-3.549750e-01	2.550460e-02

S-parameter data for the example RF device.

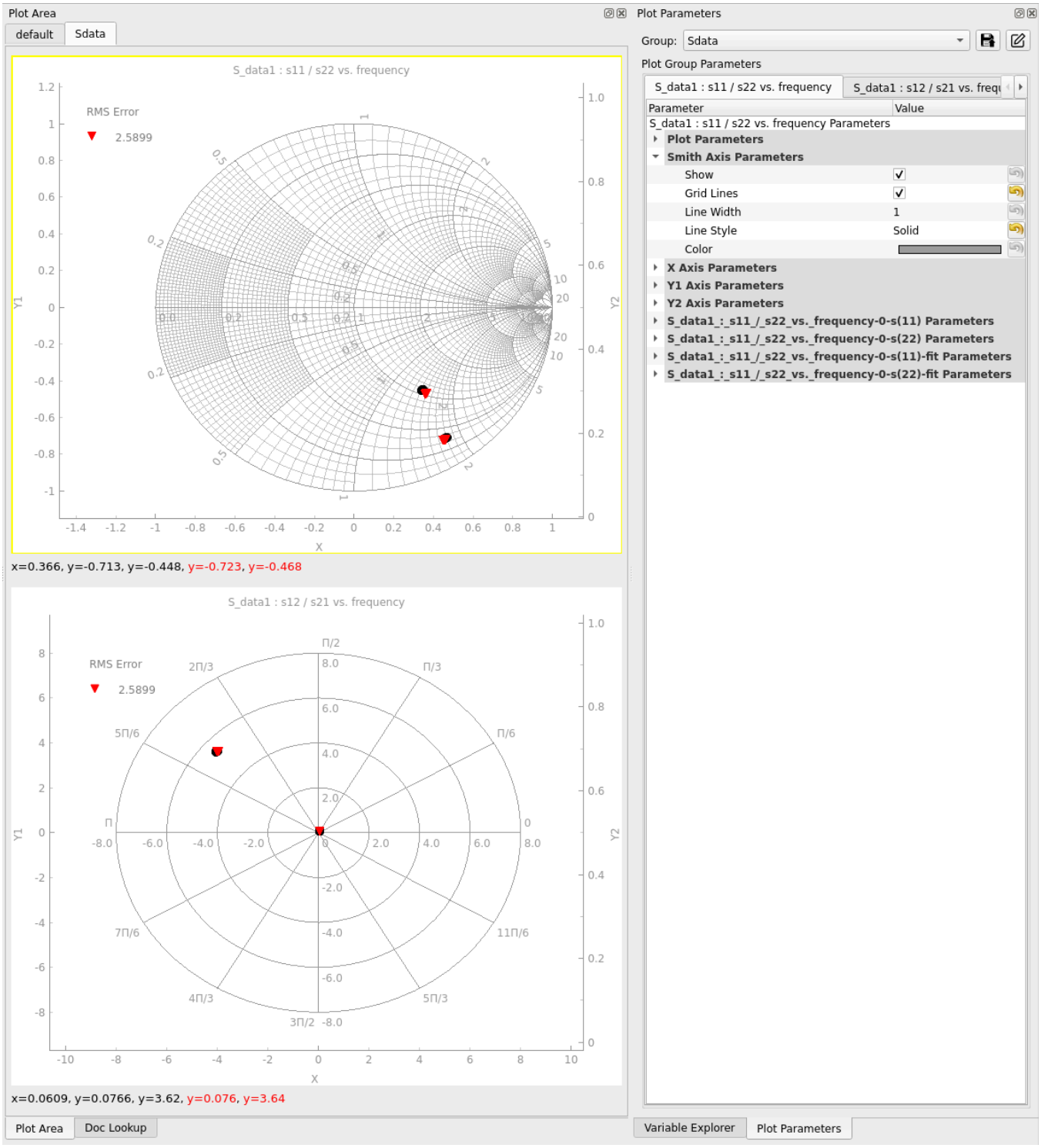
You can create a Smith chart from this data using the following commands:

```
fig = addFigure('Smith Chart')
target = fig.plot(
    S_data, col=['s(11)re', 's(11)i', 's(22)re', 's(22)i'],
    mode="smith", colour="#000000", markerStyle='o'
)
fit = fig.plot(
    S_data, col==['s(11)re-fit', 's(11)i-fit', 's(22)re-fit', 's(22)i-fit'],
    mode="smith", colour="#FF0000", markerStyle='Triangle'
)
```

Polar chats can be created from the same data using the following commands:

```
fig = addFigure('Polar Chart')
target = fig.plot(
    S_data, column=['s(12)re', 's(12)i', 's(21)re', 's(21)i'],
    mode="polar", colour="#000000", markerStyle='o'
)
fit = fig.plot(
    S_data, column=['s(12)re-fit', 's(12)i-fit', 's(21)re-fit', 's(21)i-fit'],
    mode="polar", colour="#FF0000", markerStyle='Triangle'
)
```

The figure below shows generated plot. For Smith charts, an extra Smith axis parameter menu is added to the Plot Parameters panel. This menu controls the properties of the circular Smith axis. For Polar Charts a Polar axis parameter is added for the same purpose.



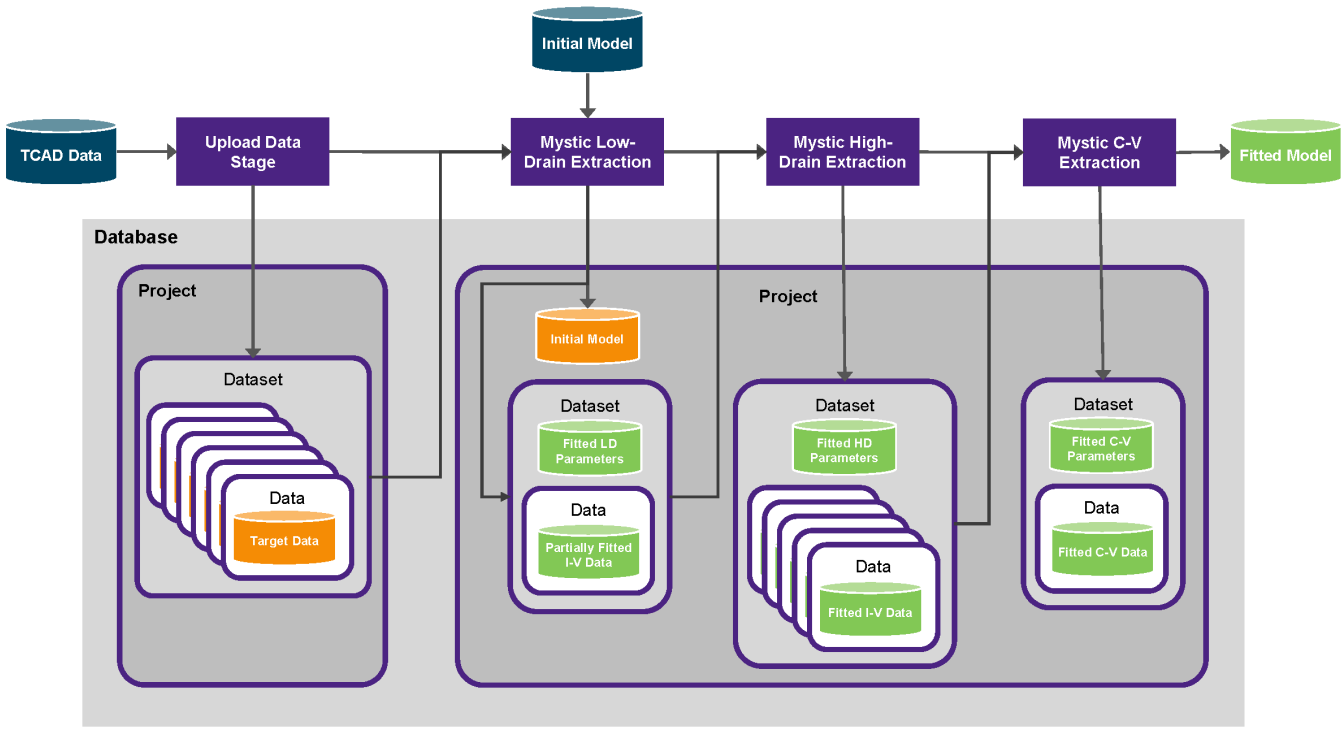
Smith chart for example RF device showing target data (black) and fit data (red).

SPICE Model Extraction

This chapter describes aspects of the Mystic environment and demonstrates how it contributes to the construction of a SPICE model extraction strategy as a whole.

Database

Mystic provides a link to the TCAD to SPICE database for consolidating target data and tracking the values of SPICE model parameters between sequential extraction stages. When running Mystic through Sentaurus Workbench, the database connection is managed automatically with the path to the database configuration file being passed to Mystic on the command line using the *-d* option. If you run Mystic standalone, then you must configure and manage the database connection manually. For information on how to do this, see SWB API Documentation. In the extraction environment, the `mdb` object is provided to store and retrieve information from the database. A standard Mystic extraction tool flow can contain up to five separate extraction stages, where each performs a subset of the fitting, allowing you to check successful parts of their extraction and to avoid continually rerunning the full strategy during development. The database ensures consistency of data and SPICE model parameter values between these stages as shown in the figure below.



Interaction with the database in standard Mystic model extraction.

This represents a three-stage SPICE model extraction for a MOSFET device with I–V and C–V target data. The upload data stage is a prerequisite for pushing target data into the database before the Mystic extraction. For details on this process, refer to the Uploading Target Data to The Database section. The database has a hierarchical structure with three levels: projects, datasets, and data objects. A project represents the full Mystic extraction strategy or one single node in Sentaurus Workbench. The main thing stored at this level is the initial model card, which can be added to the project at the start of the first extraction strategy as follows:

```
mdb.StoreInitialModel(Model)
```

where Model is the `Model` object passed in to the environment by the -m option. This model is then used as a point of reference in all downstream stages when setting up the `Model` object.

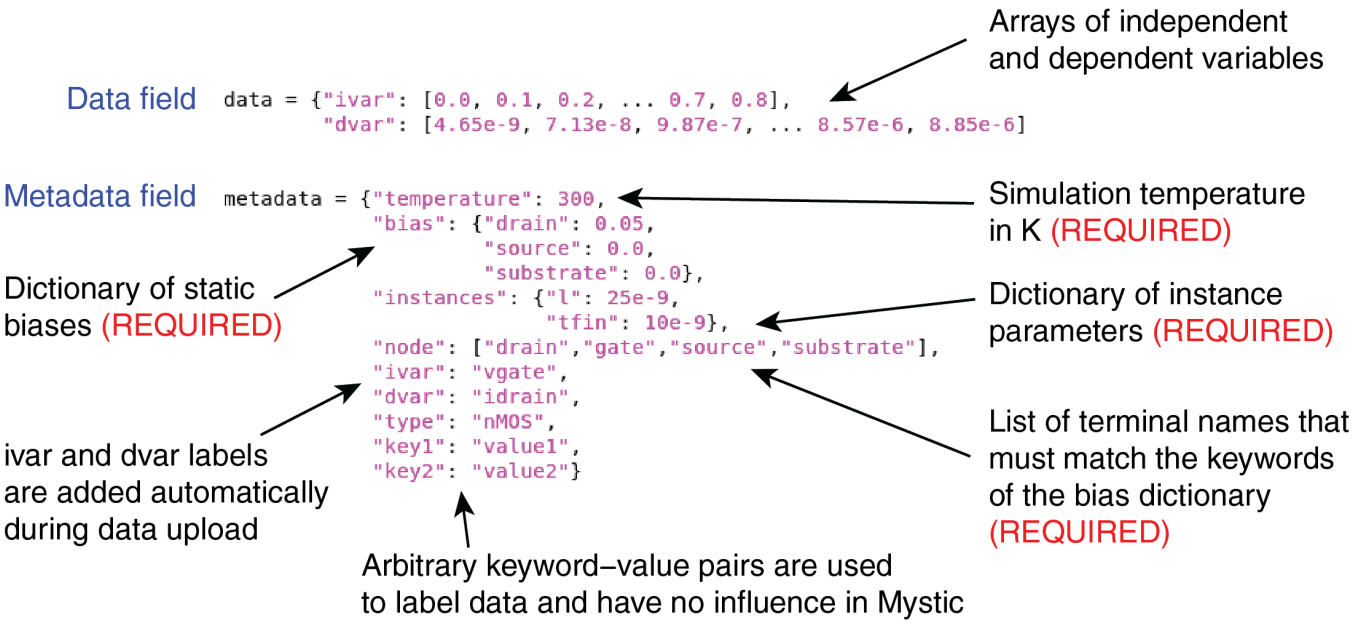
Underneath the project is a list of datasets representing each individual extraction stage. Again, these are created by Sentaurus Workbench and passed to Mystic using the *-ds* option. The datasets provide a container for extracted SPICE model parameters from each stage. Each dataset is linked to the previous one. In downstream stages, this makes it easy to retrieve the model parameters from the previous extraction stage using the `mdb.GetFitModel` method as follows:

```
Model = mdb.GetFitModel()
```

Datasets also contain a list of data objects, where each object represents one piece of fitted target data. Data objects are split into two main fields:

- The data field is a dictionary of arrays containing independent and dependent variable data.
- The metadata field is a hierarchical dictionary of extra information, which can be either simulation conditions that will be used by Mystic or simply unique tags that can be used to help query the data.

The standard Mystic metadata structure is shown in the figure below.



Structure of the metadata in the database.

It is important to understand the metadata requirements of Mystic, so you can define it correctly when uploading data and know which fields are available when loading data back from the database. In the Mystic strategy, data is retrieved from the database using the `mdb.Load` and `mdb.GetFitData` methods:

- `mdb.Load` is used to load target data from the upload data project in the first stage of an extraction strategy.
- `mdb.GetFitData` is usually used to load partially fitted data in downstream stages of the Mystic extraction.

Both methods take arbitrary filter arguments that select data from the database based on its project, data, and metadata fields. For example, the low-drain Id–Vg data shown the figure above could be loaded by using:

```
IdVgld = mdb.Load(temperature=300, bias__drain=0.05, instances__l=25e-9, ivar="vgate", dvar="idrain")
```

Note

To match hierarchical fields such as biases and instance parameters, a double underscore is used to separate the levels of hierarchy.

All data entries matching the filter criteria are included in the return. The `mdb.Load` and `mdb.GetFitData` methods return return `SimData` objects. For more information about these, see Target Data. Fitted data can be stored back in the database at the end of an extraction strategy using the `mdb.StoreFitData` method.

Target Data

Mystic can accept target data from any PLT-like or CSV-like file format. As described in the Uploading Target Data to The Database section, this data is uploaded to the Mystic database in a separate upload data stage at the start of the TCAD to SPICE tool flow and is attached with metadata such as simulation temperature and instance parameters. This provides Mystic with enough information to set up backend PrimeSim HSPICE simulations without further input from users.

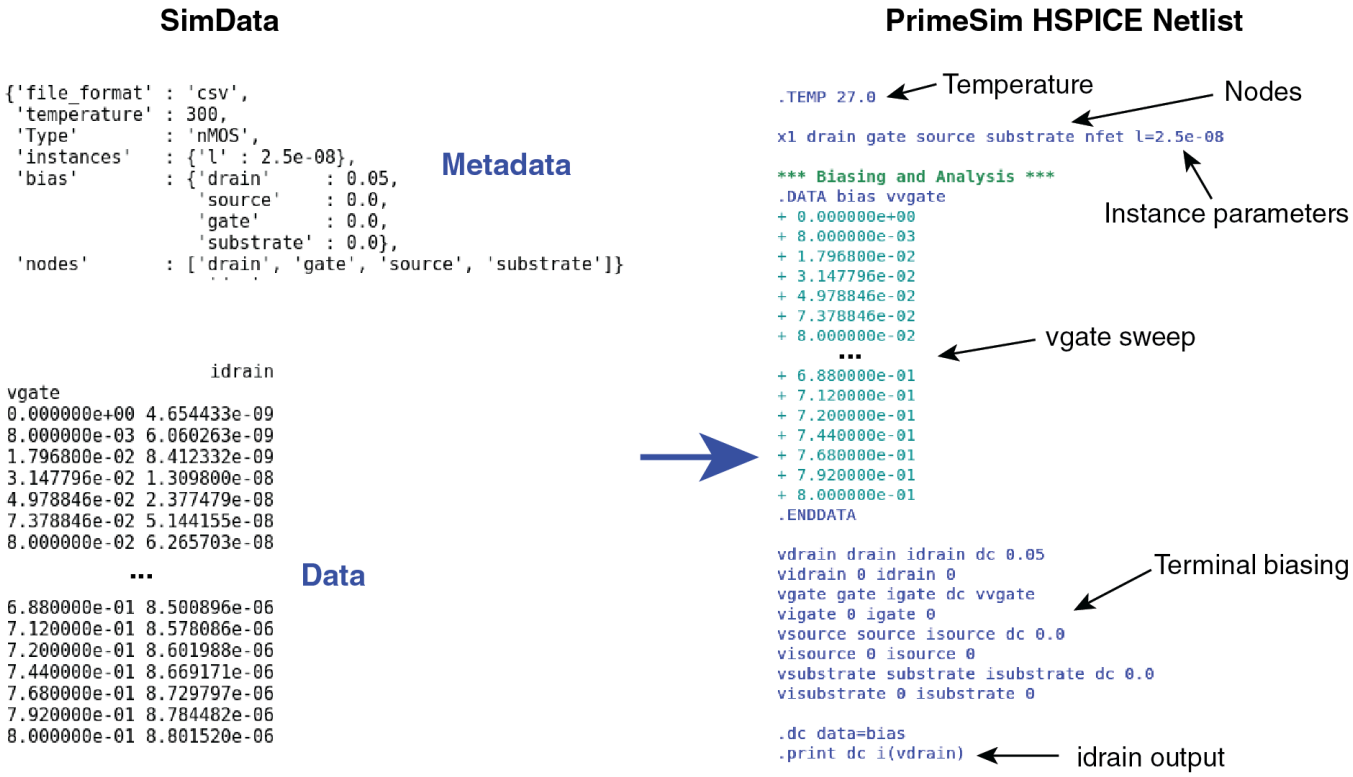
Note

Only I–V, C–V, and transient and RF scattering parameter data types are supported for extraction.

This section describes the target data structures that can be created and interacted with in a Mystic strategy, along with some of the additional functionality that they provide for specific use cases. As previously mentioned, data can be retrieved from the database in the extraction strategy using the `mdb` object. The `mdb.Load` and `mdb.GetFitData` methods both return `SimData` objects, which are the data containers used in Mystic. These consist of one or more Data objects described in Uploading Target Data to The Database and usually have four columns:

- An independent variable
- A target dependent variable (TCAD or measurement)
- A fitted dependent variable (HSPICE simulation data)
- A pointwise error

When Mystic creates backend PrimeSim HSPICE simulations, `SimData` objects are used to automatically create biasing, stimulus, and output commands in the PrimeSim HSPICE netlists that match the simulation conditions of the targets as shown in the figure below.



Example of how Mystic transforms SimData objects into PrimeSim HSPICE netlist commands.

This ensures consistency between target data and fitted data when calculating errors and removes the burden from users of having to define and manage this information themselves. From an optimization perspective, the most important column of the `SimData` objects is the pointwise error column, which is passed back to the Optimizer object after every evaluation and dictates how it adjusts the parameters. The table below lists the options of the pointwise error method.

Method	Description	Equation (f=fit, t=target)
rel	Relative	abs((f-t)/t)
abs	Absolute	abs(f-t)
log_rel	Relative of the log10 data	abs(log10(f/t)/log10(t))
log_abs	Absolute of the log10 data	abs(log10(f/t))
square	Square	(f-t)**2

The default option is *rel*, but you can change it at either a global level or on an object-by-object basis as follows:

```
SimData.SetOptions(error_method="abs")
```

When loading data from the database, multiple datasets of the same simulation type but with varying metadata fields can be combined to help significantly reduce the runtime of a Mystic extraction. This can be done by adding combine=True to any `mdb.Load` command. Consider the following example where there are two target Id–Vg simulations with drain biases of 0.05 V and 0.8 V, respectively:

```
IdVg = mdb.Load(ivar="vgate", dvar="idrain",
                project=targetProjectName)

IdVg = mdb.Load(ivar="vgate", dvar="idrain",
                project=targetProjectName,
                combine=True)
```

The first line returns one `SimData` object with two underlying datasets separated by drain bias. The second line returns one `SimData` object containing one underlying dataset with a two-level index of drain bias and gate bias. At runtime, the combined dataset will produce only one backend PrimeSim HSPICE netlist per evaluation instead of two, which results in a large runtime reduction. When the loaded data has more than one varying bias field, such as bulk device simulation data where there are multiple substrate biases and multiple drain biases, that data is combined based on the first varying field that is discovered. Alternatively, a combination field can be specified explicitly using the *extra_index* argument as follows:

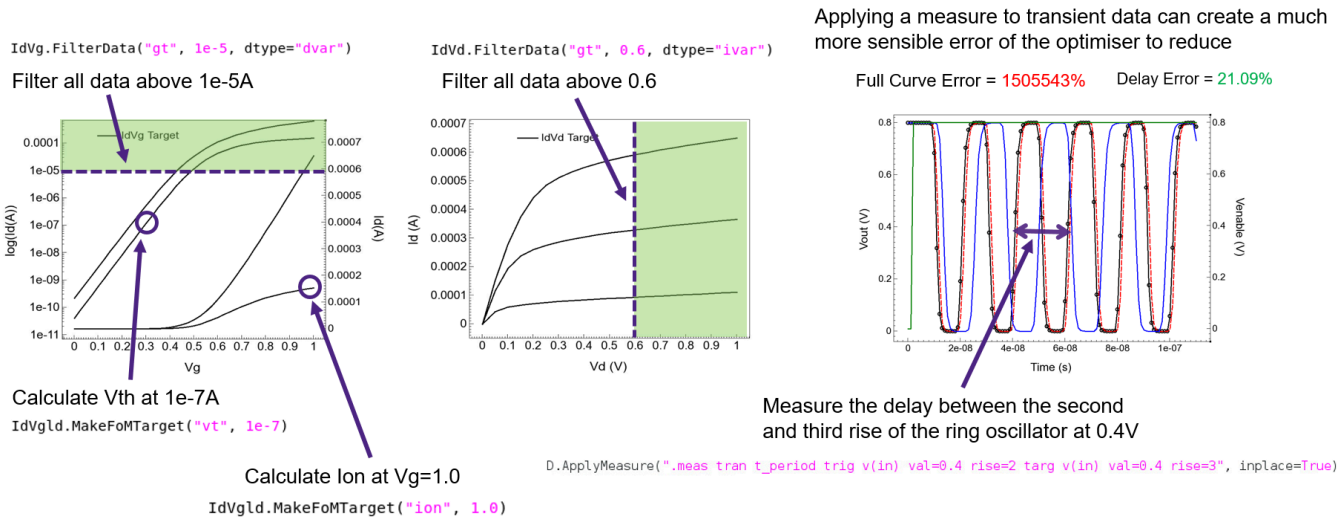
```
IdVg = mdb.Load(ivar="vgate", dvar="idrain",
                project=targetProjectName,
                combine=True, extra_index="vdrain")

IdVg = mdb.Load(ivar="vgate", dvar="idrain",
                project=targetProjectName,
                combine=True, extra_index="vsubstrate")
```

For the best performance increase, data should be combined based on the field that has the most varying values, that is, if there are two drain biases and three substrate biases, then specifying *extra_index="vsubstrate"* would be most reasonable.

Filtering Data

In SPICE model extraction, it is often necessary to target the fitting of a parameter or set of parameters to a specific region of the device-operating characteristics. For example, when fitting the workfunction, only the low-drain subthreshold characteristics are relevant. Fitting parameters against too broad a range of target data can overcomplicate the optimization problem and result in poor optimization performance. To help with this, `SimData` provides a range of Python methods for filtering, resampling, measuring, and calculating device figures of merit as shown in the figure below.



Common types of target data supported by Mystic.

Filters can be applied to any column of the data structure or any bias metadata value using the `FilterData()` method. Any of the standard mathematical operators shown in the table below can be applied.

Operator	Description
lt	Less than.
le	Less than or equal to.
eq	Equal to.
neq	Not equal to.
ge	Greater than or equal to.
gt	Greater than.

The following example illustrates how to filter data in a set of two Id–Vg curves into manageable segments:

```
# Load data from database.
IdVg = mdb.Load(ivar="vgate", dvar="idrain", project="upload-data")

# Split the data by drain bias.
IdVgld = IdVg.FilterData("eq", 0.05, dtype="bias.drain")
IdVghd = IdVg.FilterData("eq", 0.05, dtype="bias.drain")

# Split the low-drain data into below and above Vt.
IdVgld_BelowVt = IdVgld.FilterData("le", 0.3, dtype="ivar")
IdVgld_AboveVt = IdVgld.FilterData("ge", 0.4, dtype="ivar")
```

Data Masks

When filtering a `SimData` object in Mystic using the `FilterData()` method, or creating a FoM target with `MakeFoMTarget()`, a copy of the object is created and returned with the applied operation. In both these simulations the copy can be replaced with a mask that retains a reference to the root object. When a filtered object is used in a `ExtractionUtils.PrintErrors()` or `ExtractionUtils.DoStage()` call, the root object is simulated

instead of the filtered object. This means that when two or more `SimData` objects with the same root object are passed in to a `ExtractionUtils.DoStage()` optimization, only one PrimeSim HSPICE simulation is required per evaluation. This significantly reduces the runtime of a Mystic extraction. Data masks can be activated globally through the Data class as follows:

```
from libfromage.data import Data

Data.set_options(data_mask=True)
```

They can also be activated on a case-by-case basis using the *mask* argument of `FilterData()`:

```
IdVg = mdb.Load(ivar="vgate", dvar="idrain")

IdVgld = IdVg.FilterData("eq", 0.05, "vdrain", mask=True)
IdVgld_BelowVt = IdVgld.FilterData("lt", 0.3, "ivar", mask=True)
IdVgld_AboveVt = IdVgld.FilterData("gt", 0.4, "ivar", mask=True)

IdVghd = IdVg.FilterData("eq", 0.05, "vdrain", mask=True)
IdVghd_BelowVt = IdVghd.FilterData("lt", 0.3, "ivar", mask=True)
IdVghd_AboveVt = IdVghd.FilterData("gt", 0.4, "ivar", mask=True)
```

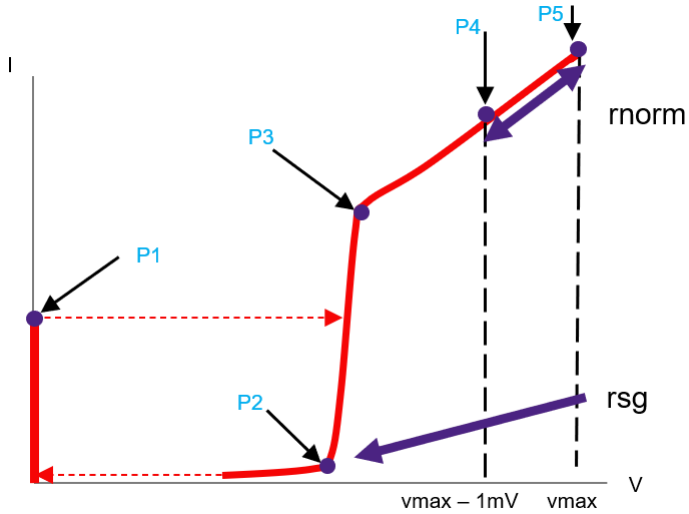
Figure of Merit Transforms

The `SimData` object also provides a set of figures of merit calculation methods specific to the behavior of MOSFET devices as shown in the table below.

fom	description
Vt	Threshold voltage of the device. Should be calculated on low drain Id-Vg data. The default current criteria is 1e-7A
Ioff	Off current of the device. Calculated at Vg=0.0 by default
Ion	On current of the device. Calculated at the max Vg point of the data by default.
Gm	Transconductance of the device at all bias points.
Gmmax	Maximum transconductance of the device.
VtGmmax	Threshold voltage based on the maximum transconductance of the device. Calculated value is the x intercept of the gradient of the maximum transconductance
Derivative	Derivative of the data.
Rout	Output resistance of the data.
ft	Calculate unity gain frequency of the device's short circuit current gain (ft). This figure of merit can only be applied to H-parameter data.
FMax	Calculate FMax of the device. This figure of merit can only be applied to Y-parameter data or Maximum unilateral gain data.

fom	description
DIBL	Drain-induced barrier lowering of the device calculated between the low and high drain IdVg data. The data must contain only two target data curves with different drain biases for the calculation to work.
Sum	Sum the target values across multiple targets. Targets should have the same independent variables, e.g. Vg, but don't necessarily need to have matching dependent variables. For example, Cgd + Cgs is possible.

Josephson junction I–V targets have a set of custom figures of merit specific to those devices described in the figure and table below.



Typical Josephson junction I–V curve shape (red) with key figures of merit: P1–P5, rnorm, and rsg.

fom	description
P1	Critical current of the device.
P2	Lower voltage in the gap voltage slope. The gap voltage can be calculated by adding P2 and P3 and dividing by 2.
P3	Upper voltage in the gap voltage slope.
P4	Junction voltage at vmax – 1 mV. It can be used to manually calculate the normal resistance.
P5	Highest junction voltage present in the data, vmax.
rnorm	Normal resistance of the device.
rsg	Subgap resistance of the device, calculated at the bottom of the gap voltage

For details about Josephson junction SPICE model extraction, see Josephson Junction Extraction.

Note

This functionality is not included in the main Mystic license and requires an add-on license to use. For more information, contact your local TCAD support team.

Each figure or merit has it's own method, for example:

```
Vt = IdVgld.VtGmmax()
```

In this case, Vt is a single float that can be used as a threshold for filtering. Figures of merit that require an argument, for example, Vt current criteria, are initialized with sensible defaults. These can be changed using the `SetFoMCriteria()` method. It is also possible to create an extraction target from a figure of merit using the `MakeFoMTarget()` method as follows:

```
Vt = IdVgld.MakeFoMTarget("vt", 1e-7)
```

In this case, Vt is a new `SimData` object that can be used as usual in extraction steps. The data of the root object (IdVgld) is used to create and run the PrimeSim HSPICE simulation, then the FoM transform is applied to both the target and fit columns as a postprocessing task. When used in an extraction step, the single error between the target and fitted FoM values will be used instead of the pointwise error between the curves. You can also define your own FoM transforms and apply them to any `SimData` object. The transform can either be applied to all data points, such as a log transform or multiplier, or it can aggregate the data to a single value, such as threshold voltage. The following example shows how to apply a log10 transform to all data:

```
import numpy as np

IdVg = mdb.Load(ivar="vgate", dvar="idrain")

def log10(data);
    return np.log10(data)

IdVg = IdVg.MakeFoMTarget(log10)
IdVgld = IdVg.FilterData("eq", 0.05, "vdrain")
IdVgld_BelowVt = IdVgld("lt", 0.3, "ivar")
IdVgld_AboveVt = IdVgld("gt", 0.4, "ivar")

IdVghd = IdVg.FilterData("eq", 0.05, "vdrain")
IdVghd_BelowVt = IdVghd("lt", 0.3, "ivar")
IdVghd_AboveVt = IdVghd("gt", 0.4, "ivar")
```

Your function should accept a data argument that is the internal Data object to which the transform is applied. As the transform is applied to the root Id–Vg object, all objects derived from that using `FilterData()` will also have the log transform applied to them. The following example shows how to apply a mean aggregation transform:

```
IdVg = mdb.Load(ivar="vgate", dvar="idrain")

def mean(data, col="target"):
    if col == "target":
        return data._values.mean()
    elif col == "fit":
        return data._fit_values.mean()

IdVg_mean = IdVg.MakeFoMTarget(mean, add_as_fom=True)
```

The *add_as_fom* argument signals that the function is an aggregation that should be added to the Data object API. In this scenario, the function must accept a column argument that selects whether the aggregation is applied to the target data or fit data. FoM transforms can also be chained together where appropriate. Consider this example that extracts against the log10 transform of the device transconductance:

```
IdVg = mdb.Load(ivar="vgate", dvar="idrain")

IdVgld = IdVg.FilterData("eq", 0.05, "vdrain")
Gmld = IdVgld.MakeFoMTarget("gm")

def log10(data):
    return np.log10(data)

Gmld_log = Gmld.MakeFoMTarget(log)
```

Generic PrimeSim HSPICE Measure Statements

More generic transforms can also be applied to `SimData` objects using the PrimeSim HSPICE .MEASURE syntax. PrimeSim HSPICE measures can be used in the same way as figures of merit to calculate electrical characteristics of a simulation but give you more freedom to define and apply your own transforms. In Mystic, these can be applied using the `ApplyMeasure()` method. The following is an example of calculating Vt and Ion of an Id–Vg target using .MEASURE commands:

```
# Create a Vt target.
Vt = IdVg.ApplyMeasure('.MEASURE DC vt FIND v(g) when i(vd)=-1e-7')
# Create an Ion target.
Ion = IdVg.ApplyMeasure('.MEASURE DC ion MAX i(vd)')
```

As with `MakeFoMTarget()`, `SimData` objects are returned from `ApplyMeasure()` and can be used as usual throughout the extraction. At any point in the extraction, the contents of a SimData object can be written to disk as either a PLT or CSV file, using the `WritePLTFile()` and `WriteCSVFile()` methods. The created files can then be plotted using Sentaurus Visual.

All bias terminal names referenced in the .MEASURE command must match terminal names attached to the data. For example, if a data object has the terminal names drain, gate, source, and substrate, then the following .MEASURE command will fail at runtime because the terminal name d does not exist:

```
Ioff = IdVgld.ApplyMeasure(".MEASURE DC ioff MIN i(vd)")
```

The following would correctly measure Ioff for the low-drain Id–Vg curve:

```
Ioff = IdVgld.ApplyMeasure(".MEASURE DC ioff MIN i(vdrain)")
```

You can test .MEASURE commands before using them in an optimization by utilizing the `TestMeasure()` method as follows:

```
Ioff = IdVgld.ApplyMeasure(".MEASURE DC ioff MIN i(vdrain)")
Ioff.TestMeasure(Simulator)
```

Multiple .MEASURE commands can also be applied to the same `SimData` object using the *inplace* argument. For example, Ioff and Vt of a low-drain Id–Vg simulation can be optimized simultaneously by specifying:

```
IoffVt = IdVgld.ApplyMeasure(".MEAS DC ioff MIN i(vdrain)")
IoffVt.ApplyMeasure(".MEAS DC vt FIND v(gate) WHEN i(vdrain)=1e-7",
                    inplace=True)
```

The error in both measured quantities will be passed to the optimizer during optimization. You can apply .MEASURE commands to DC, AC, or transient data, provided they are valid based on the simulation conditions of the data. For more information about how to construct a .MEASURE command, refer to the PrimeSim HSPICE® Reference Manual: Commands and Control Options.

Sometimes, you might want to calculate only the measure for the fitted data and to use a fixed measure value for the target data instead of using `TestMeasure()`. You can achieve this by using the fixed_target argument as follows:

```
Ioff = IdVgld.ApplyMeasure(".MEAS DC ioff MIN i(vdrain)", fixed_target=1e-12)
```

Bias Extension

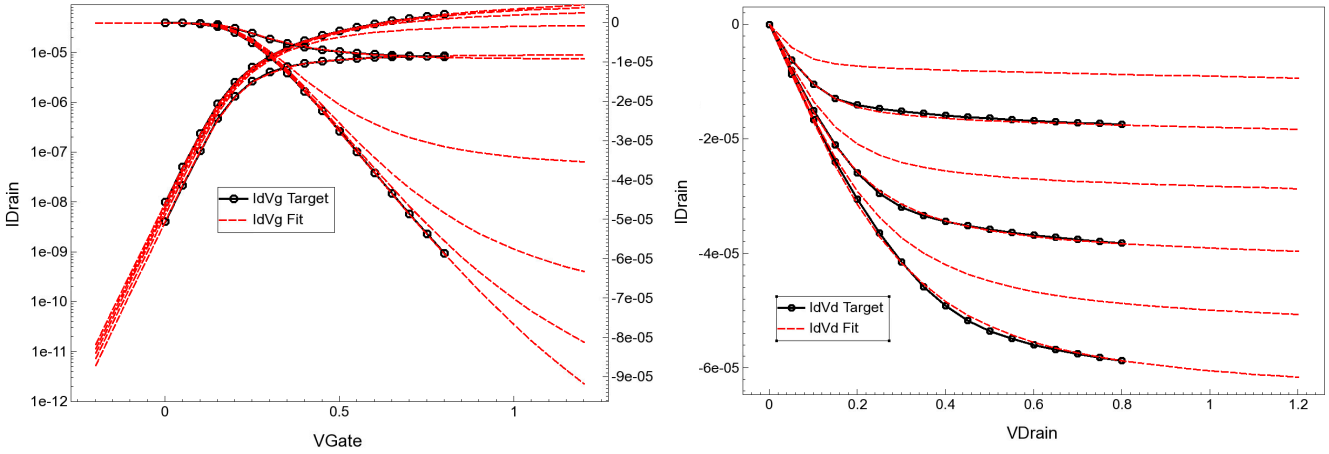
After model extraction, you might want to simulate your model at bias conditions outside of the extraction range. This can be done using `ExtendBias()`. Consider an extraction where the target data consists of two Id–Vg curves at Vd=0.05 V and 0.8 V, respectively, with a range of Vg=0.0 V – 0.8 V, and three Id–Vd curves at Vg=0.4 V, 0.6 V, and 0.8 V, with a range of Vd=0.0 V – 0.8 V:

```
# Extend the bias ranges of the data
IdVg_ext = IdVg.ExtendBias(ivar_min=-0.2, ivar_max=1.2, spacing=0.05,
                           vdrain=[0.2,0.4,0.6])
IdVd_ext = IdVd.ExtendBias(ivar_max=1.2, spacing=0.05,
                           vgate=[0.3,0.5,0.7])

# Simulate the model at the extended ranges

ExtractionUtils.PrintErrors("idvg-ext", IdVg_ext, Simulator, Model, "rmsd")
ExtractionUtils.PrintErrors("idvd-ext", IdVd_ext, Simulator, Model, "rmsd")

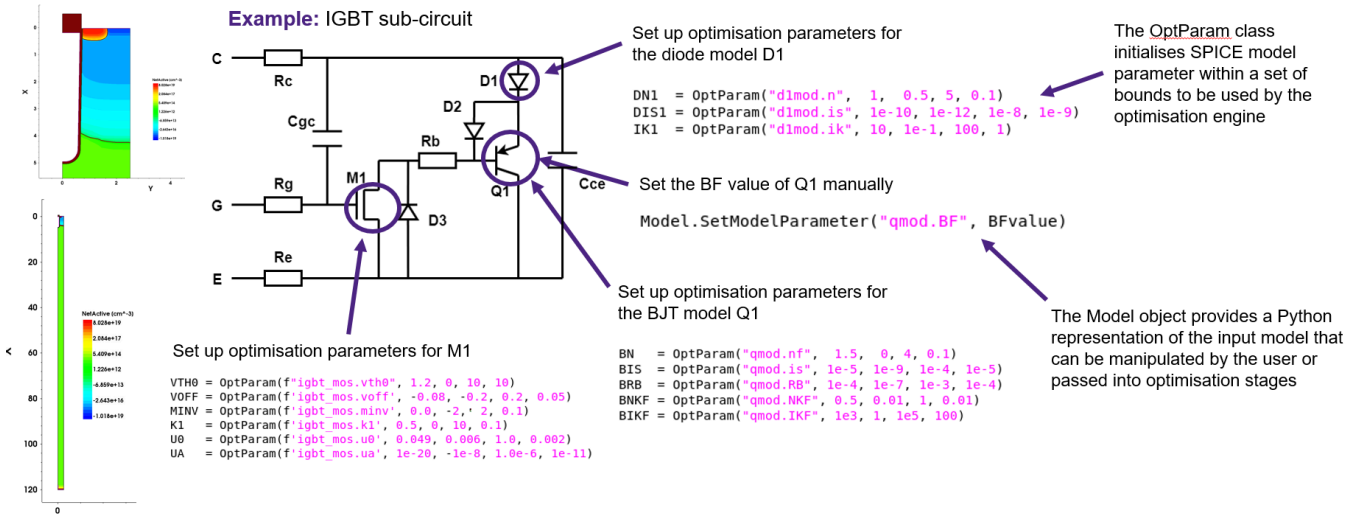
# Write the extended data to a PLT file
IdVg_ext.WritePLTFile('idvg-ext')
IdVd_ext.WritePLTFile('idvg-ext')
```



As can be seen in the images above, the data range has been extended to the bounds specified in `ExtendBias()` and new data curves have been simulated for the intermediate drain and gate biases.

SPICE Models

Mystic supports any PrimeSim HSPICE compatible netlist for extraction. This can be anything from a single modelcard to a complex configuration of circuit components, including sub-circuits and .INCLUDE commands. The figure below shows an example sub-circuit model used for the extraction of an IGBT power device.



An example IGBT sub-circuit model.

When a model card is supplied on the command line using the -m option, it is parsed by Mystic and injected into the extraction environment as a `Model` object. The `Model` object contains a hierarchical representation of the model definition in Python and has an API for manipulating the SPICE model parameter values at any level of the netlist structure using a dot-separated syntax. For example, in the netlist shown in the figure above, if you want to set the Beta factor (BF) for the BJT model (QMOD) to 100, then you can add the following line to your Mystic strategy:

```
Model.SetModelParameter("QMOD.BF", 100)
```

Conversely, the current value of any model parameter can be retrieved from the netlist using:

```
BF = Model.GetModelParameter("QMOD.BF")
```

The primary objective of a Mystic extraction strategy is to find the set of SPICE model parameter values that ensure the best fit between the provided target data and the output of an HSPICE simulation under matching simulation conditions. To achieve this, Mystic can control the model parameter values during optimization, which is done through the `OptParam` class. For more information about `OptParam`, see the Optimization Parameters section. At any point in the strategy, the current state of Model can be written to disk using Model.write or pushed to the database using `mdb.StoreFitParameters` or `mdb.StoreFitData`.


```
Model.SetModelParameter("QMOD.BF", 100)
```

Conversely, the current value of any model parameter can be retrieved from the netlist using:

```
BF = Model.GetModelParameter("QMOD.BF")
```

The model can also be checkpointed at any point in the strategy using the set_checkpoint method as follows:

```
Model.set_checkpoint("Stage1")
```

Where the supplied argument is a label that can be used to retrieve the state of the model at a later point. This functionality is most frequently used in interactive mode when developing the extraction methodology. Previously checkpointed states of the model can be recovered using the get_checkpoint method as follows:

```
Model.get_checkpoint("Stage1", inplace=True)
```

If inplace is True, the model state will be applied to the existing model object. If False, a copy of the model state will be returned from the method. When running an extraction strategy, automated checkpoints will also be created upon any call to `ExtractionUtils.DoStage()`. The label used for these checkpoints will be the label passed as the first argument to do stage. This effectively allows you to undo the action of one or more optimization stages. For example:

> ExtractionUtils.DoStage(“Stage1”, Model, Simulator, [PHIG,CDSC,QMFACTOR], Optimizer, [IdVgld_BelowVt]) Model.get_checkpoint(“Stage1”, inplace=True)

Will return the model to the state it was in prior to executing Stage1.

Simulator

Mystic uses the PrimeSim HSPICE tool as a backend simulation engine. Before running Mystic, you must ensure you have a valid PrimeSim HSPICE binary in your environment path or supply the full path to one using the -l option. The Mystic `Simulator` object provides functionality for writing netlists based on the supplied `Model` and `SimData` objects, running PrimeSim HSPICE simulations, parsing the outputs of those simulations, and comparing the results to the target data. In a standard extraction strategy, these operations are all handled automatically by Mystic inside the utility functions provided by `ExtractionUtils` so your only interaction with the simulator will be to set command-line options and to configure the global simulation options. The `Simulator` object is configured with the default PrimeSim HSPICE options listed in the table below:

Simulation Option	Default Value	Description
NUMDGT	8	Controls the listing printout accuracy.
MEASDGT	8	Controls the value precision in measure files.
INGOLD	2	Controls whether PrimeSim HSPICE prints output in exponential form. Default is exponential
NOMOD	1	Suppress the printout of model parameters in the output.
EXT_OP	1	Enable additional information output in PrimeSim HSPICE

Simulation Option	Default Value	Description
MEASFORM	1	Enables writing of measurement output files to Excel or HSIM formats as well as the traditional PrimeSim HSPICE formats.
GMINDC	1e-18	Specified minimum transconductance in parallel for PN junctions and MOSFET nodes in DC analysis.
ABSTOL	1e-15	Absolute error tolerance for branch currents in DC and transient analysis.

These options are added to all back-end netlists as .OPTION statements and can be altered using the `Simulator.set_options` method.

From a PrimeSim HSPICE licensing perspective, Mystic can run simulations in different modes:

- In normal mode, a PrimeSim HSPICE license is checked out, utilized, and then checked back in for every simulation. Depending on the license checkout time and the number of simulations required in an extraction, this might be very inefficient.
- In server mode, Mystic checks out and holds a PrimeSim HSPICE server license at the beginning of the extraction and then continually uses it throughout. To activate the PrimeSim HSPICE server mode, add a -C option to Simulator.spiceargs as follows:

```
Simulator.spiceargs.append('-C')
```

A single PrimeSim HSPICE simulation can be run from any `SimData` object using the `GenerateData()` method as follows:

```
IdVg.GenerateData(Simulator, Model)
```

The `Simulator` and `Model` are passed in as arguments to construct the netlist and run the backend PrimeSim HSPICE simulation. On completion of the simulation, the fit column of Id–Vg is updated with the result of the simulation, and the pointwise error column is updated with the pointwise error between the target and fit curves. When running inside an optimization, this error vector is passed back to the optimization algorithm. In a standard extraction strategy, the `Simulator` object is most commonly used as an argument to the utility functions `ExtractionUtils.PrintErrors()` and `ExtractionUtils.DoStage()`, and is rarely used standalone. For more information about these functions, see the Full Strategy section.

Parallel PrimeSim HSPICE Simulations

Mystic extracts the values of SPICE model parameters by continually running PrimeSim HSPICE simulations and comparing their results with target data coming from TCAD or measurement. For targets that have more than one dataset, for example, low-drain and high-drain Id–Vg data, multiple PrimeSim HSPICE simulations

are required to complete one evaluation. The Mystic PrimeSim HSPICE library allows these simulations to be executed in parallel. This can significantly reduce the runtime of complex extraction strategies. The maximum number of parallel PrimeSim HSPICE simulations is controlled by the property

```
Simulator.max_sims as follows:
Simulator.max_sims = 10
```

The default value for this property is 1, that is, serial simulation. The objective function of the `Optimizer` object has also been parallelized where possible, particularly in the Jacobian calculation of the commonly used bounded trust region algorithm. The Jacobian evaluation makes small perturbations in each of the parameters being optimized and uses the results to determine a gradient of best descent for the problem. Consider an optimization problem with five parameters and 2 bits of target data. The number of PrimeSim HSPICE simulations required for a Jacobian evaluation is 5 parameters × 2 samples per parameter × 2 targets = 20 simulations. As these simulations have no dependence on each other, they can all be performed in parallel. To activate this feature, pass parallel=True to any call to ExtractionUtils.DoStage as follows:

```
ExtractionUtils.DoStage("BelowVt", Model, Simulator, [PHIG,CDSC,QMFACTOR],
                        Optimizer, [IdVgld_BelowVt], parallel=True)
```

The number of parallel simulations in the `Optimizer` is still controlled by the *Simulator.max_sims* property.

Note

Running PrimeSim HSPICE simulations in parallel requires multiple PrimeSim HSPICE licenses. If you are limited by PrimeSim HSPICE licenses, then you should run PrimeSim HSPICE in server mode to avoid license failures in Mystic. For example:

```
Simulator.spiceargs = ["-C"]
```

With the PrimeSim HSPICE server mode and parallel simulations activated, a queue of PrimeSim HSPICE server licenses is created dynamically when needed, then retained for the duration of the extraction. This is a more reliable way to limit the number of licenses required when running in parallel to the exact number specified by *Simulator.max_sims*.

Parameter Space Exploration

Before extracting a SPICE model parameter set in Mystic, it is important to understand how each parameter in your chosen model affects the target data curves against which you are extracting. To help with this, Mystic provides the following utility functions to explore the parameter space.

Sensitivity Analysis: ExtractionUtils.SensitivityAnalysis

One-factor-at-a-time sensitivity analysis aims at understanding the response of the underlying model as a function of small changes to a single parameter, with all other parameters fixed. It reveals only the local gradient of the response surface of the model with respect to a given parameter. This local sensitivity analysis can be performed in Mystic by using the `ExtractionUtils.SensitivityAnalysis()` utility function as shown here:

```
IdVg_belowvt = IdVg.FilterData("le", 0.4)
IdVg_belowvt = IdVg.FilterData("gt", 0.4)
PHIG = OptParam("PHIG", 4.5, 4.2, 4.7, 1.0)
CDSC = OptParam("CDSC", 0.1, 0.0, 10.0, 1.0)
U0 = OptParam("U0",0.02,0.006,1.0,0.02)
UA = OptParam("UA",0.3,0.0,100.0,1.0)
sense_df = SensitivityAnalysis([IdVg_belowvt, IdVg_abovevt],
                               Simulator, Model,
                               [PHIG,CDSC,U0,UA])
```

The method can take any number of targets and parameters, and displays the sensitivity for each target individually and the combined root mean square deviation (RMSD) of the targets.

Parameter Sweep: ExtractionUtils.ParamSweep

You can perform a parameter sweep of a single parameter against a single target by using the `ExtractionUtils.ParamSweep()` utility function, which is a simpler version of the utility function `ExtractionUtils.SensitivityAnalysis()` and produces one PLT file for each evaluation of the target at a particular parameter value. The following example shows a workfunction parameter sweep using the BSIM-CMG parameter PHIG:

```
ExtractionUtils.ParamSweep("idvg-ld", IdVgld, Simulator, Model, PHIG, "rmsd")
```

The default number of parameters in the sweep is 11, and the default upper and lower bounds are taken from the PHIG `OptParam` definition. These can be overwritten using the num, pmin, and pmax function arguments, respectively.

Optimization Parameters

The core of a successful SPICE model extraction strategy is a set of parameters that can be varied to fit the data. Mystic provides the OptParam class for defining and constraining these parameter values. `OptParam` allows you to set an initial value (if the parameter does not already exist in the model), upper and lower bounds, scaling factors and transforms for the set of parameters you want to optimize. The naming conventions here are the same as for `GetModelParameter()` referenced previously, with a dot separating the levels of hierarchy in the netlist. For example, to initialize the Beta factor (BF) of the BJT (QMOD) for optimization in the IGBT device, use:

```
# name, default, lower bound, upper bound, scale factor
BF = OptParam("QMOD.BF", 100.0, 1.0, 1000.0, 1.0)
```

The scaling factor can be used to help control the influence the parameter has in the optimization when using gradient descent-based algorithms. Before calculating the parameter step for the jacobian calculation, the current value of the parameter is divided by the scaling factor. This can help normalize the parameters and balance the optimization. It if particularly useful for parameters with very small values, e.g. capacitance parameters in the order of 1e-16, or parameters which are initialized far below their expected final value. Below is an example of a capacitance parameter using an appropriate scaling factor:

```
cfs = OptParam("mosmod.cfs", 1e-11, 1e-15, 1e-8, 1e-11)
```

Each SPICE model parameter has a different impact on the data against which it is being optimized. Some parameters have a linear response, whereas others can be highly non-linear. The performance of gradient descent-based optimization routines is best when parameter vary close to linearly against the fit error. To help with this, transforms can be applied to the parameters to each of the OptParams in place of the scaling factor. For example:

```
cfs = OptParam("mosmod.cfs", 1e-11, 1e-15, 1e-8, "decimal-log")
```

Instead of scaling the parameter will now be transformed before the steps are calculated in the jacobian calculation, then inverse transformed to run the simulations. The available transforms are listed in the table below.

Transform	Description
‘scale’	Apply a scalar division to the parameter value. Equivalent to using a scaling factor directly.
‘shift’	Add of subtract from the parameter value. Useful to move parameters away from a zero starting value, which is often difficult for an optimization routine to converge from.
log	Apply a natural logarithm transform to the parameter value.
decimal-log	Apply a log 10 transform to the parameter value.
***log-exp* | Apply a logarithmic-exponential transform to the** parameter from below. It maps [lower, inf) to (-inf, inf).	
***exp* | Apply an exponential transform to the parameter** value.	
decimal-exp	Apply a decimal exponential transform to the parameter value, i.e. 10 ^ value.

Transform	Description
dual-log1p **| Apply a log transform to parameters with positive** and negative values. Parameters around zero are approximately linearly transformed.	
square | Apply a square transform to the parameter value.	
square-root | Apply a square root transform to the parameter value.	
power **| Apply a power transform to the parameter value.** Square and square root are examples of specific power transforms, e.g. 2 and 0.5.	

Optimal Parameter Transforms: ExtractionUtils.GetOptimalTransforms

In the ExtractionUtils model there is also a helper function to calculate the best transforms for a given set of SPICE model parameters. The best transform is defined as the transform which provides the most linear response against the data. The function can be used as follows:

```
PHIG = OptParam('PHIG', 4.3, 4.25, 4.35, 1.0)
CDSC = OptParam('CDSC', 7e-3, 1e-5, 1.0, 1.0)
QMFACTOR = OptParam('QMFACTOR', 1.0, 1e-5, 100, 1.0)

tphig,tcdsc,tqmfactor = ExtractionUtils.GetOptimalTransforms(
    [PHIG,CDSC,QMFACTOR], IsVgld_BelowVt,
    Simulator, Model, step=0.01
)

PHIG = OptParam('PHIG', 4.3, 4.25, 4.35, tphig)
CDSC = OptParam('CDSC', 7e-3, 1e-5, 1.0, tcdsc)
QMFACTOR = OptParam('QMFACTOR', 1.0, 1e-5, 100, tqmfactor)
```

The optimal transforms returned by the function can be passed in to redefinitions of the OptParams. These new OptParams can then just be used as normal throughout the rest of the extraction strategy. For more information on this and other helper function please see `ExtractionUtils.GetOptimalParameterTransform()`.

Optimization Algorithms

The core of Mystic is a powerful optimization library with several algorithms to help optimize SPICE model parameter values to fit the target data provided. The `Optimizer` object is injected into the environment at runtime and provides an interface to the underlying library. The available algorithms are described in the table below.

Interface	Description
TRUST_REGION	Use Intel MKL Trust Region least squares solver.
BOUNDED_TRUST_REGION	Use Intel MKL Bounded Trust Region least squares solver.
least_squares	Use least squares solvers. Available internal methods are trust region, dogleg and Levenberg-Marquardt.
minimize	Use minimizers. A cost function to aggregate the residuals must be given in this case (RMS error by default). Available minimizers include trust region, BFGS (default), L-BFGS-B, COBYLA, etc.
differential_evolution	Use a differential evolution algorithm to find the global minimum of the multivariate function.
doe	Provides acces to the Bayesian search-based optimization method.

The initial algorithm of the Optimizer is dictated by the *-o* option. If not supplied, then it defaults to *BOUNDED_TRUST_REGION*, which is used most frequently throughout Mystic extractions. Inside the strategy, the algorithm can be changed using `SetMethod()` as follows:

```
Optimizer.SetMethod("least_squares")
```

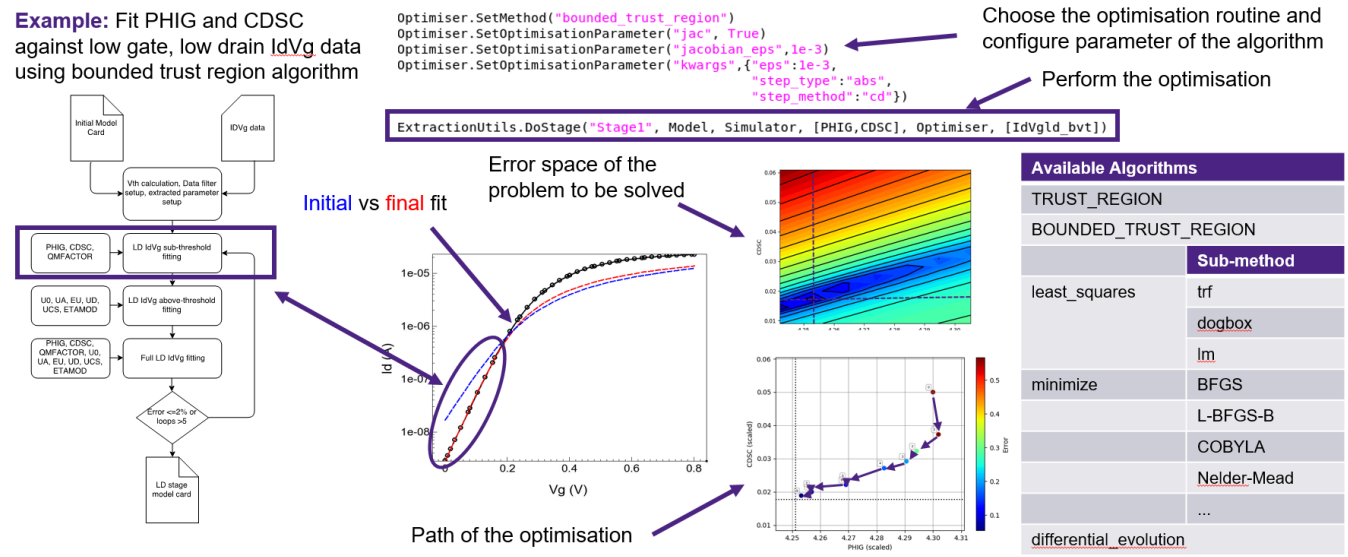
Each algorithm has a set of parameters that can be used to adjust tolerances, the method, the initial conditions, and so on. More detailed information about these algorithms can be found in the Optimizer section of the API documentation. These parameters can be set using the `SetOptimizationParameter()` method. The following code snippet illustrates some recommended settings for the *BOUNDED_TRUST_REGION* algorithm:

```
Optimizer.SetMethod("BOUNDED_TRUST_REGION")
Optimizer.SetOptimizationParameter("eps1", 1e-3)
Optimizer.SetOptimizationParameter("eps2", 1e-3)
Optimizer.SetOptimizationParameter("eps3", 1e-3)
Optimizer.SetOptimizationParameter("eps4", 1e-3)
Optimizer.SetOptimizationParameter("eps5", 1e-3)
Optimizer.SetOptimizationParameter("eps6", 1e-3)
Optimizer.SetOptimizationParameter("jac", True)
Optimizer.SetOptimizationParameter("kwargs", {"eps": 1e-3,
                                              "step_type": "abs",
                                              "step_method": "cd",
                                              "eps_fac": 1e-6})
```

The recommended settings for the least_squares algorithm are shown below:

```
Optimizer.SetMethod("least_squares")
Optimizer.SetOptimizationParameter("method", "trf")
Optimizer.SetOptimizationParameter("ftol", 1e-4)
Optimizer.SetOptimizationParameter("xtol", 1e-6)
Optimizer.SetOptimizationParameter("gtol", 1e-4)
Optimizer.SetOptimizationParameter("diff_step", 1e-3)
```

As mentioned in SPICE Models, `OptParam` objects are created for all parameters that will be optimized. How these are used varies depending on the optimization algorithm. For example, if *BOUNDED_TRUST_REGION* is used, then the parameter value will remain strictly within the bounds defined by the OptParam object during optimization. However, if *TRUST_REGION* is used, then these bounds will be ignored. The last argument in the `OptParam` definition is a scale factor that can play a very important role in the convergence of an optimization, particularly with gradient descent–based methods. The idea of the scale factor is to balance the sensitivities of a set of parameters being used in the optimization to prevent one parameter dominating another parameter. A sensible default value for the scale factor is the initial parameter value. It can then be increased to add more sensitivity or decreased to reduce the sensitivity. To better understand the sensitivity of a set of parameters against an extraction target, you can use the utility function `ExtractionUtils.SensitivityAnalysis()`. As with the `Simulator`, the `Optimizer` has limited use in the extraction strategy as a standalone object beyond algorithm selection and parameter setup. It is most commonly used in combination with other objects in the `ExtractionUtils.DoStage()` utility function, as shown in the figure below.



Overview of the Mystic optimization engine.

The `ExtractionUtils.DoStage()` utility function takes the `Model`, the `Simulator`, a list of `OptParam`, the `Optimizer`, and a list of `SimData` objects, and combines them to perform a single optimization step within a strategy. After a stage completes, the optimization result is written to screen or to the Mystic log file, the optimal parameter values are retained in the `Model` object, and the optimal simulation results are stored in the class:*~DataStructures.SimData* objects.

Bayesian Optimization Algorithm

The Mystic optimization library provides a Bayesian optimization algorithm. This algorithm performs an adaptive search of the parameter space guided by current available knowledge from all the previously performed simulations. It starts with Latin hypercube sampling to construct an initial Gaussian process

response surface model. In each iteration, an acquisition function (expected improvement) is evaluated, and new points are selected that could potentially improve on the current best solution. To use the Bayesian optimization algorithm in Mystic, add the following to your extraction strategy:

```
Optimizer.set_method("doe")
Optimizer.set_optimization_parameters(method="bayesian_optimizer")
```

A list of other options for the Bayesian optimizer can be found in the Optimizer section.

The Engine Module

The `Engine` module is designed to be an object-oriented replacement for the ExtractionUtils module. It groups together the `Model`, `Simulator`, and `Optimizer` objects and provides functionality for defining the flow of an extraction strategy. A full description of the `Engine` module can be found in the Mystic API documentation. A SPICE model extraction strategy is defined as a series of sequential optimization stages that fit sets of SPICE model parameters to a section of target data. The Engine module allows you to calculate parameter sensitivity, to define extraction steps, and to extract SPICE model parameters. Multiple stages can also be grouped together in a loop with a specified exit criterion.

The following code extract is from an example of a low-drain Id–Vg strategy:

```
Engine.add_extraction_step(IsVgld_BelowVt, [PHIG,QMFACTOR,CDSC])
Engine.add_extraction_step(IsVgld_AboveVt, [U0,UA,EU,UD,UCS,ETAMOB])
Engine.add_extraction_step(IsVgld_BelowVt + IsVgld_AboveVt,
                           [PHIG,QMFACTOR,CDSC,U0,UA,EU,UD,ETAMOB,UCS])

fit_error = Engine.extract(error_threshold=2.0, max_loops=3)
```

Individual extraction stages are defined using the add_extraction_step method based on a list of SPICE model parameters and a subset of the target data. The extract method is then used to perform the extraction. This method moves through the added stages sequentially, then calculates a single RMS error at the end across a combination of all the data in the stages. If the error_threshold is met, then the extraction concludes successfully, and it returns to the start of the loop and reruns the stages. If the exit criterion is not met after the maximum number of loops, then the extraction concludes in any case.

Full Strategy

This section describes how all of the individual components of a Mystic SPICE model extraction strategy work together to achieve an accurate SPICE model fitting of a template 14 nm FinFET device at low-drain bias conditions. To run this example yourself please consult the Mystic Examples section.

In general, a standard extraction strategy can usually be split into the following sections:

- Configuring Settings
- Setting Up the Model
- Loading and Filtering Data
- Defining the Optimization Parameters
- Executing the Extraction Strategy

- Writing the Results

Configuring Settings

This section controls top-level options for the data, simulator, and optimization algorithm. The following code snippet shows the settings applied to the low-drain Id–Vg extraction of a 14 nm FinFET device. In general, this section remains consistent across most extraction strategies.

```
tfin           = 8e-09        # Physical fin thickness [m]
hfin           = 4e-08        # Physical fin height [m]
wf0            = 4.301        # workfunction

mystic_verbose = False
modstr         = "mosmod."
error_method   = "rmsd"

# Data settings
SimData.SetOptions(data_mask=True)

# Simulator settings
Simulator.spiceargs =  ['-C']
Simulator.set_options(gmin=1e-18)

# Optimizer settings
eps=1e-3
Optimizer.SetMethod("BOUNDED_TRUST_REGION")
Optimizer.SetOptimizationParameter("jac", True)
Optimizer.SetOptimizationParameter("eps1",eps)
Optimizer.SetOptimizationParameter("eps2",eps)
Optimizer.SetOptimizationParameter("eps3",eps)
Optimizer.SetOptimizationParameter("eps4",eps)
Optimizer.SetOptimizationParameter("eps5",eps)
Optimizer.SetOptimizationParameter("eps6",eps)
Optimizer.SetOptimizationParameter("jacobian_eps",eps)
Optimizer.SetOptimizationParameter("kwargs",{"eps":eps,
                                             "step_type":"abs",
                                             "step_method":"cd"})
```

As the core of Mystic is a Python interpreter, any number of custom variables can be defined through the strategy. The example defines some physical characteristics of the device:

- mystic_verbose sets the verbosity of extraction stages.
- modstr is the name of the model in the model card.
- error_method is the single error method to use when reporting the evaluation error back in verbose mode.

Target data options are configured through the `SimData.SetOptions` method. Here, *data_mask=True*. This argument handles data more efficiently when filtering and reduces extraction runtime. For details, see Data Masks. Simulator settings can be grouped into two categories:

- Command-line arguments to pass to PrimeSim HSPICE, controlled with spiceargs
- Options to add to the backend netlists as .OPTION commands, controlled by *Simulator.set_options*

Here, you use the PrimeSim HSPICE server mode (see Simulator) and set the minimum conductance in PrimeSim HSPICE. The `Optimizer` object is configured with the *BOUNDED_TRUST_REGION* algorithm, which is the default. This can be changed using `SetMethod()`. The eps1 to eps6 arguments control the exit criteria of the algorithm. The value 1e-3 is usually a sensible value for these, although it might need to be reduced or increased for better accuracy or turnaround time. *kwargs* is a dictionary of options to pass to the jacobian calculation of the *BOUNDED_TRUST_REGION* algorithm. These are explained in more detail in the Optimizer section.

Setting Up the Model

Next, the initial `Model` object must be prepared for extraction by setting some default values. Here, you set the fin thickness, fin height, and workfunction based on the quantities defined earlier in the strategy. You also set the source and drain resistance of the model to zero to match the conditions of the target simulations. The specific parameter set used for a BSIM-CMG model extraction is beyond the scope of this user guide. For a more in-depth look at the model, refer to the TCAD to SPICE module of the TCAD Sentaurus Tutorial, accessible through Sentaurus Workbench.

```
Model.SetModelParameter(f'{modstr}TFIN',tfin)
Model.SetModelParameter(f'{modstr}HFIN',hfin)
Model.SetModelParameter(f'{modstr}PHIG',wf0)
Model.SetModelParameter('r_sc',0)
Model.SetModelParameter('r_dc',0)

Model.SetModelParameter(f'{modstr}ETA0', 0.0)
Model.SetModelParameter(f'{modstr}DVTP1',1.0)
Model.SetModelParameter(f'{modstr}CGSL', 1e-20)
```

Loading and Filtering Data

Now, you load in the target data. When running Mystic through Sentaurus Workbench, data is usually loaded in from the database using `mdb.Load` and `mdb.GetFitData`. This example is standalone, meaning you do not have a database connection and all data is file based. Here, you load the data in from PLT files using the `SimData.FromPLT` method. The target PLT files come from a Sentaurus Device simulation of a 14 nm FinFET template device. As can be seen in the following code snippet, the target data is attached with metadata containing the simulation temperature and a nodes list. The nodes list instructs Mystic about the terminal order when it constructs netlists and must match the terminal naming in the PLT file.

```
metadata = {"temperature": 300,
            "nodes": ["drain","gate","source","substrate"]}

IdVgld = SimData.FromPLT("targets/IdVg_ld.plt", ivar="vgate", dvar="idrain", metadata=metadata)

vt_gm_val = float(IdVgld.VtGmMaxFinal())

IdVgld         = IdVgld.FilterData('lt', -1e-13, dtype="dvar", mask=False)
IdVgld_BelowVt = IdVgld.FilterData('le', vt_gm_val-0.05, dtype="ivar")
IdVgld_AboveVt = IdVgld.FilterData('ge', vt_gm_val+0.05, dtype="ivar")

IdVgld._name = "IdVgld"
IdVgld.interactive = True
```

The *IdVgId* `SimData` object can then be filtered around *VtGmMax* in preparation for the extraction. In the Mystic Examples section, Mystic is run in interactive mode using the Mystic GUI. To ensure data is plotted in real time as the extraction runs it must be tagged as *interactive* as shown above.

Defining the Optimization Parameters

Now, you must initialize an `OptParam` of each SPICE model parameter to be extracted. For some parameters, such as the workfunction, it might be sensible to restrict the bounds based on its current value. In this case, the workfunction is obtained from the TCAD simulation, so you do not want it to vary significantly from that value.

```
phigval = Model.GetModelParameter(f'{modstr}PHIG')
PHIG    = OptParam(f'{modstr}PHIG',   phigval, phigval-0.5, phigval+0.5, 1.0)
CDSC    = OptParam(f'{modstr}CDSC',   0.001,   0.0,   10.0,  1.0)
U0      = OptParam(f'{modstr}U0',     0.02,    0.006, 1.0,   0.02)
UA      = OptParam(f'{modstr}UA',     0.3,     0.0,   100.0, 1.0)
ETAMOB  = OptParam(f'{modstr}ETAMOB', 2.0,     0.0,   10.0,  2.0)
EU      = OptParam(f'{modstr}EU',     2.5,     0.1,   10.0,  2.0)
UD      = OptParam(f'{modstr}UD',     0.1,     0.0,   1.0,   0.1)
UCS     = OptParam(f'{modstr}UCS',    1.0,     0.0,   10.0,  1.0)

Stage1Parameters = [PHIG,CDSC]
Stage2Parameters = [U0,UA,EU,UD,UCS,ETAMOB]
Stage3Parameters = [PHIG,CDSC,U0,UA,EU,UD,ETAMOB,UCS]
```

The parameters are assembled into lists that will control which segments of data they are optimized against:

- Stage1Parameters will be fitted against below Vt data.
- Stage2Parameters will be fitted against above Vt data.
- Stage3Parameters will be fitted against all the low-drain Id–Vg data.

Executing the Extraction Strategy

Now everything is set up to begin the extraction process. As the environment is Python, you can use loops and value comparisons to control the flow of the extraction. For low-drain Id–Vg data, the extraction strategy is very simple: calculate the single-value fit error of the model against the IdVgId target, and continue executing the extraction loop until the error has dropped below 2% or you have reached the maximum of five loops. In the core of the loop, first you fit the below-Vt data, then the above-Vt data, and then all the data together.

```
fit_error=ExtractionUtils.PrintErrors("Initial", IdVgld, Simulator, Model, error_method, plot_style={"fit":
{"colour":"#0000ff"}})
x=0

while (x < 5 and fit_error>2.0):
    ExtractionUtils.DoStage("Step1", Model, Simulator, Stage1Parameters,
                            Optimizer, [IdVgld_BelowVt],verbose=mystic_verbose)
    ExtractionUtils.DoStage("Step2", Model, Simulator, Stage2Parameters,
                            Optimizer, [IdVgld_AboveVt],verbose=mystic_verbose)
    ExtractionUtils.DoStage("Step3", Model, Simulator, Stage3Parameters,
                            Optimizer, [IdVgld_BelowVt,IdVgld_AboveVt], [1,2], verbose=mystic_verbose)
    fit_error=ExtractionUtils.PrintErrors("IdVgld", IdVgld, Simulator, Model, error_method)
    x+=1

ExtractionUtils.PrintErrors("IdVgld", IdVgld, Simulator, Model, error_method)
```

When using gradient descent–based algorithms such as bounded trust region, it is unlikely that fitting all the parameters against all the data, at the same time, will find a sensible minimum. The problem space is too big and the relationship between the SPICE model parameters is too nonlinear. It is possible to achieve a good result in a single extraction step by using a global optimization routine such as differential evolution, but this can be extremely time-consuming, taking thousands of iterations. Usually, an iterative approach as shown here can find a balance between the final error and the extraction time. After the extraction has finished, the final error is printed to screen.

Writing the Results

All that remains is to write out the results of the extraction. If you run Mystic through Sentaurus Workbench, results will be stored in the database there. Data is written to a PLT file for visualization in Sentaurus Visual, and the `Model` is written as a .mod file. This will be used as the initial model for the next extraction strategy. The evaluation history of the `Optimizer` can be written to a CSV file. This can be useful as it shows the single error and parameter value progression across the entire extraction strategy, which can help to identify problems or inefficiencies.

```
IdVgld.WritePLTFile("./results/IdVg-ld")
Model.write("./models/nmos-ld.mod")
Optimizer.write_parameter_csv("./results/IdVg-ld-parameters")
```

Specific Use Cases

This section presents specific use cases in Mystic.

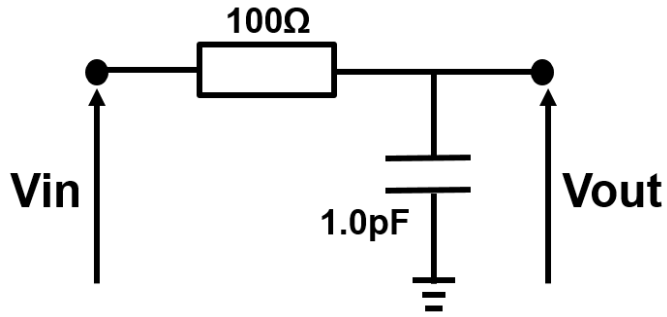
Transient Simulation Extraction

Mystic can handle and extract model parameters against transient simulation data coming from Sentaurus Device as any CSV-like data. This section describes how to upload transient simulation data and use it to extract SPICE parameters based on a very simple RC time constant example shown in the figure below.

```
.subckt rc in out

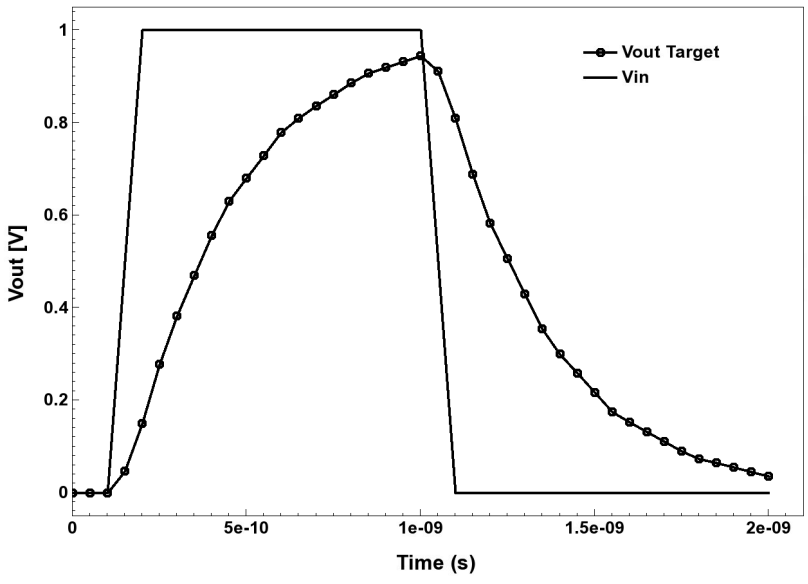
r in out 100.0
c out 0 1e-12

.ends
```



Simple RC netlist example schematic.

The subcircuit *rc* will become the initial model for the extraction. Synthetic target data has been generated by PrimeSim HSPICE at R=100 and C=3e-12, with a square wave input stimulus. This data has been written to a CSV file as shown in the figure below.



time	vout	vout-fit	errors	vin
0.000000e+00	0.000000e+00	nan	nan	0.000000e+00
5.000000e-11	0.000000e+00	nan	nan	0.000000e+00
1.000000e-10	0.000000e+00	nan	nan	0.000000e+00
1.500000e-10	4.620000e-02	nan	nan	5.000000e-01
2.000000e-10	1.500000e-01	nan	nan	1.000000e+00
2.500000e-10	2.780000e-01	nan	nan	1.000000e+00
3.000000e-10	3.830000e-01	nan	nan	1.000000e+00
3.500000e-10	4.700000e-01	nan	nan	1.000000e+00
4.000000e-10	5.570000e-01	nan	nan	1.000000e+00
4.500000e-10	6.300000e-01	nan	nan	1.000000e+00
5.000000e-10	6.800000e-01	nan	nan	1.000000e+00
5.500000e-10	7.290000e-01	nan	nan	1.000000e+00
6.000000e-10	7.790000e-01	nan	nan	1.000000e+00
6.500000e-10	8.100000e-01	nan	nan	1.000000e+00
7.000000e-10	8.360000e-01	nan	nan	1.000000e+00
7.500000e-10	8.620000e-01	nan	nan	1.000000e+00
8.000000e-10	8.870000e-01	nan	nan	1.000000e+00
8.500000e-10	9.070000e-01	nan	nan	1.000000e+00
9.000000e-10	9.200000e-01	nan	nan	1.000000e+00
9.500000e-10	9.330000e-01	nan	nan	1.000000e+00
1.000000e-09	9.450000e-01	nan	nan	1.000000e+00
1.050000e-09	9.110000e-01	nan	nan	5.000000e-01
1.100000e-09	8.100000e-01	nan	nan	2.040000e-15
1.150000e-09	6.890000e-01	nan	nan	0.000000e+00
1.200000e-09	5.830000e-01	nan	nan	0.000000e+00
1.250000e-09	5.060000e-01	nan	nan	0.000000e+00
1.300000e-09	4.300000e-01	nan	nan	0.000000e+00
1.350000e-09	3.540000e-01	nan	nan	0.000000e+00
1.400000e-09	3.000000e-01	nan	nan	0.000000e+00
1.450000e-09	2.590000e-01	nan	nan	0.000000e+00
1.500000e-09	2.170000e-01	nan	nan	0.000000e+00
1.550000e-09	1.750000e-01	nan	nan	0.000000e+00
1.600000e-09	1.530000e-01	nan	nan	0.000000e+00
1.650000e-09	1.320000e-01	nan	nan	0.000000e+00
1.700000e-09	1.110000e-01	nan	nan	0.000000e+00
1.750000e-09	9.050000e-02	nan	nan	0.000000e+00
1.800000e-09	7.390000e-02	nan	nan	0.000000e+00
1.850000e-09	6.440000e-02	nan	nan	0.000000e+00
1.900000e-09	5.500000e-02	nan	nan	0.000000e+00
1.950000e-09	4.550000e-02	nan	nan	0.000000e+00
2.000000e-09	3.610000e-02	nan	nan	0.000000e+00

Before performing a parameter extraction, the target data must be uploaded to the TCAD to SPICE database as follows:

```
Data.from_csv("rc.csv", ivar="time", stimulus="vin",
              metadata={"nodes":["in","out"], "t":300},
              upload_ds="rc-target-data")
```

For more details on uploading target data, see the Uploading Target Data to The Database section.

Note

The ivar label for transient data must be time because Mystic uses this in the backend to determine the simulation type and to set up the appropriate backend SPICE simulation.

The Mystic strategy for extracting against transient data is essentially the same as a standard DC or AC extraction:

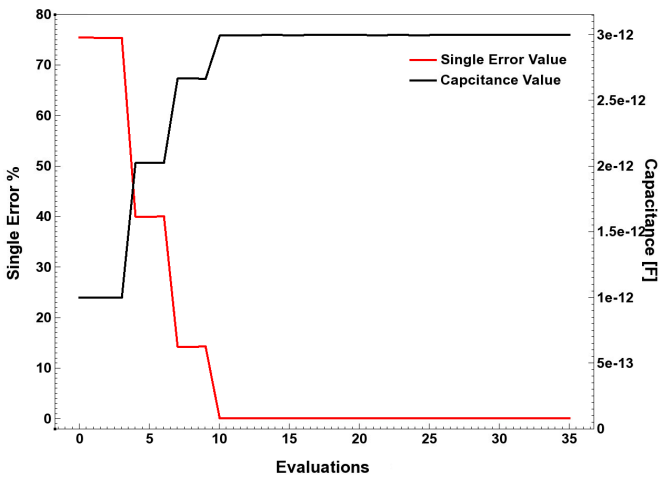
```
# Load the target data from the database
RC = mdb.Load(ivar="time", dvar="vout")

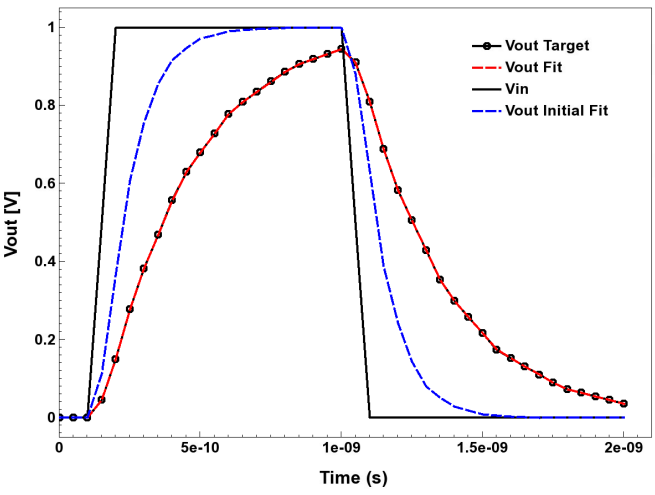
# Calculate an initial error and write a PLT file
ExtractionUtils.PrintErrors("RC-trans", D, Simulator, Model, "rmsd")
RC.WritePLTFile("RC-trans-init", bias_info=False)

# Extract the capacitance
C = OptParam("C", 1e-12, 1e-13, 1e-11, 1e-12)
ExtractionUtils.DoStage("RC-trans", Model, Simulator, [C], Optimizer, [D])

# Write the final PLT file and the parameter history CSV file
RC.WritePLTFile("RC-trans", bias_info=False)
```

The stimulus columns of the target data (vin in this case) are converted into piecewise linear (PWL) functions in the backend SPICE netlist to ensure the simulation receives the same triggers used to generate the target data. The figure below shows the parameter value history of the `Optimizer` and the initial and final fits of the capacitance.





MOSFET Reliability (MOSRA) Extraction

As CMOS technology scales down, reliability requirements become more challenging and more important. Hot-carrier injection and bias temperature instability can change the characteristics of a device over time. PrimeSim HSPICE MOSRA analysis can be used to model these effects, allowing circuit designers to predict the reliability of their design. Mystic provides the capability to extract PrimeSim HSPICE MOSRA parameters against ΔVth and ΔIds targets.

Target Data

The target data for a MOSRA extraction should be presented in a CSV file with the following layout:

```
temperature,stresstime,l,vdrain,vgate,delvth0,dids
85,3000,2.8e-08,1.1,1.1,0.0601,0.0206
85,3000,2.8e-08,1.3,1.1,0.0918,0.1460
85,3000,2.8e-08,1.5,1.1,0.1261,0.9991
85,3000,2.8e-08,1.1,1.3,0.1815,0.0165
85,3000,2.8e-08,1.3,1.3,0.2693,0.0742
85,3000,2.8e-08,1.5,1.3,0.4189,0.5421
85,3000,2.8e-08,1.1,1.5,0.5309,0.0071
85,3000,2.8e-08,1.3,1.5,0.7941,0.0466
85,3000,2.8e-08,1.5,1.5,1.1795,0.3020
```

The only required labels are ‘temperature’, ‘stresstime’ and at least one target, e.g. ‘delvth0’ or ‘dids’. The target labels must match the fields output to the radeg0 stressing file produced by PrimeSim HSPICE. Other, optional column labels are instance parameters, e.g. ‘l’, and terminal biases, e.g. ‘vdrain’, ‘vgate’.

Unlike conventional data that must be uploaded to the database, the MOSRA target is loaded directly into the strategy using the `CreateMOSRATarget()` function as shown below. In this case, we are only interested in the delvth0 degradation type.

```
dvth = SimData.CreateMOSRATarget("mosra_target.csv",
                                 target=["delvth0"],
                                 instances=["l"],
                                 metadata={"nodes":["drain","gate","source","substrate"],
                                           "mosra":{"bias":{"source":0.0,
                                                            "substrate":0.0},
                                                    "mosra_model":"nfet_ra",
                                                    "base_model":"nfet"}})
```

Lists of instance parameters and target labels must be passed to the function to allow Mystic to identify the appropriate columns in the CSV file. Extra metadata can also be passed in during target creation to pass additional information that is not included in the CSV file such as static biases. As with any standard Mystic target, the terminals present in the metadata *bias* dictionary must match those in the metadata nodes list. Note that the *mosra_model* and *base_model* keywords, which identify the MOSRA and base model definitions in the initial model card, are required entries in the *mosra* metadata dictionary if the built-in Mystic equations are used. These allow Mystic to identify the MOSRA and base model definitions in the initial model card. After the targets have been created, they can be treated as standard Mystic target data objects. When passed into an optimization stage, Mystic inherently knows how to set up the backend HSPICE MOSRA aging simulation and to parse the outputs accordingly.

Initial Modelcard

The initial model for a MOSRA model extraction should include three components: A base model definition, on top of which the MOSRA effects will be applied; A MOSRA model definition, with default values for the MOSRA parameters that are to be extracted; and a .APPENDMODEL command, to tie the two model definitions together. An example MOSRA initial modelcard is shown below.

```
.model nfet_ra mosra level=3
+HCIAP = 1.0e-6
+HCIm = 2.0
+HCID = 9.0
+HCIG = 0.0
+HCIEa = 0.05
+HCIn = 0.5
+VGSMIN = 0.2
+VGSMAX = 1.5
+BTIAP = 1
+BTId = 0.0
+BTIg = 5.0
+BTIm = 3.0
+BTIr = 0.6
+BTIEa = 0.143
+BTIn = 0.175
+HCIFLAG = 1
+BTIFLAG = 1
+HCILFLAG = 1
+BTILFLAG = 1

.model nfet nmos
+DEVTYPE = 1
+LEVEL = 72

.appendmodel nfet_ra mosra nfet nmos
```

Where nfet is a pre-extracted BSIM-CMG model and nfet_ra is the MOSRA model overlay. As mentioned in the previous section, Mystic will construct the appropriate back-end PrimeSim HSPICE simulations based on the target data so there is no need to include any model instances, bias statements, .MOSRA statements or output statements in the intitial modelcard.

Using Built-in Equations

When extracting against multiple MOSRA targets simulataneously, Mystic must run one PrimeSim HSPICE simulation per target, which can cause a significant increase in extraction time. To combat this, the PrimeSim HSPICE MOSRA equations have been built in to Mystic and can be evaluated directly without calling PrimeSim HSPICE. To select this mode of operation, set the `UseMOSRAEquation()` attribute at the top of the Mystic extraction strategy.

```
Simulator.UseMOSRAEquation = True
```

When running in this mode of operation it is necessary to run a validation PrimeSim HSPICE simulation after the model parameters have been extracted to ensure the RMS error is consistent.

Changing The Equations

To correctly model physical reliability effects for a particular device, it may be desirable to customise the model equations in PrimeSim HSPICE. Doing so will create a mis-match between the outputs of PrimeSim HSPICE and the outputs of the MOSRA equations built-in to Mystic. To prevent this, Mystic provides the capability to change the built in equations to match those that are being used in PrimeSim HSPICE using the `SwitchMOSRAEquation()` method as shown below.

```
Simulator.SwitchMOSRAEquation(new_equations, Model, delvth0, error_threshold=1.0)
```

Here, *new_equations* is a Python callable with the function signature shown below, i.e. it must accept model and data arguments. Inside the function, the model and data objects can be used to aid the construction of the equation.

```
def new_equations(model, data):
    # metadata and model parameters can be retrieved from the model and data objects and used within the custom
equations
    L = data.metadata["mosra"]["instances"]["l"]
    hciap = model.get_model_parameter("nfet_ra.hciap")
    btiap = model.get_model_parameter("nfet_ra.btiap")

    # define custom equations and perform calculations. Note that the equations shown below are for
illustration only
    dvth0 = 1e-3 * btiap * L
    dids = hciap * L

    # return a dictionary of values calculated by the custom equations
    return {"delvth0":  dvth0, "dids": dids}
```

The return of the function is a Python dictionary, the keys of which must match the degradation labels from the target CSV file.

When the equations are changed, Mystic will run a validation simulation in PrimeSim HSPICE to ensure that the outputs of the new equations match the outputs of PrimeSim HSPICE within a particular tolerance dictated by the error_threshold argument. If this is not the case, any parameters extracted against the Mystic equations will give incorrect or unexpected degradation effects when run in PrimeSim HSPICE.

Outputing The Data

After the extraction has concluded, the data can be output to CSV using the `WriteMOSRAFile()` method as shown below. The target and fit column headers will be appended with either '_target' or '_fit', e.g. 'delvth0_target, 'delvth0_fit'.

```
delvth0.WriteMOSRAFile("mosra_output.csv")
```

Extraction of Variability Using Principal Component Analysis

Adding variation to a SPICE compact model can be achieved in different ways. In TCAD to SPICE tool flows, variation models are created through a combination of variability TCAD simulations, Mystic SPICE model extractions, and RandomSPICE simulations. In these flows, Mystic produces individual model cards without variation information, which are amalgamated into a variation library by RandomSpice. This library accurately captures the variability trends shown by the input TCAD data and can be used to run simple circuit benchmarks for the target technology. Mystic can also add simple variations to SPICE models themselves, without the need for RandomSpice. This section discusses how to do this using the `CorrelatedParameters` class.

Correlated Parameters

When running PrimeSim HSPICE Monte Carlo simulations, you can add a Gaussian distribution to a single parameter using the agauss statement. For example, the following model will apply a Gaussian distribution to PHIG, causing a Vt shift:

```
.param phig_std = 0.001
.param pvar = agauss(0,1,1)

.model nmos1 nmos
+ level = 72
+ devtype = 1
+ phig = '4.3 + phig_std*pvar'
```

The arguments passed to the *agauss* statement are the mean, the standard deviation, and the number of standard deviations. It is common practice in SPICE modeling to create zerocentered distributions with a standard deviation of one, and then shift them afterwards. In this example, *PHIG* will be centered around 4.3 with a single standard deviation of 0.001. The simplest form of variation SPICE model is to apply these single distributions to multiple parameters. In reality, the variation of SPICE model parameters is likely to be highly correlated and this oversimplified approach produces unrealistic output distribution. For example, as Vt decreases, the subthreshold slope also starts to decrease, so variation parameters for the workfunction and subthreshold slope are likely to be highly correlated. Applying two uncorrelated Gaussian distributions to these parameters is likely to underestimate the overall variation and give combinations of a high Vt and steep subthreshold slope that would likely not occur. The `CorrelatedParameters` class can help to add correlations to variation parameters. It uses a principal component analysis (PCA) technique to automatically calculate variation terms for each parameter based on correlation and standard deviation information provided by users, and then it applies these terms to the model. Consider the following example:

```
PHIG = OptParam("PHIG", 4.5, 4.2, 4.7, 1.0)
CDSC = OptParam("CDSC", 0.1, 0.0, 10.0, 1.0)
U0 = OptParam("U0",0.02,0.006,1.0,1.0)
UA = OptParam("UA",0.3,0.0,100.0,1.0)

CP = CorrelatedParameters([PHIG,CDSC,U0,UA])

CP.setup_model(Model)
CP.update_model(Model)
print(Model)

# Set parameter standard deviations
CP.set_sigma("PHIG",0.01)
CP.set_sigma("CDSC",0.001)
CP.set_sigma("U0",0.005)
CP.set_sigma("UA",0.002)

# Set some correlation between parameters. These are for demonstration
# only and do not represent anything real
CP.set_correlation("PHIG","CDSC", 0.7)
CP.set_correlation("U0","UA", 0.8)

CP.update_model(Model, components=2)
print(Model)
```

Here, you start from a BSIM-CMG model that has been prefitted to some nominal performance target data. The example does not consider variability target data as there is no need for optimization (which is covered later in the section). Four parameters are selected to apply variation to, covering the workfunction, subthreshold slope, and mobility. These parameters are first initialized as OptParams as in any other standard Mystic extraction and are used to create the `CorrelatedParameters` object. Before adding variation, the model is set up with the correct parameter structure using `setup_model()` and `update_model()`. The first printing of the Model object should look something like this:

```
.model nmos1 nmos
+DEVTYPE=1
+LEVEL=72
+PHIG='4.5 + PHIG_var'
+UA='0.001 + UA_var'
+CDSC='0.1 + CDSC_var'
+U0='0.01 + U0_var'

.param PHIG_var = '0.0*pca0 + 0.0*pca1'
.param CDSC_var = '0.0*pca0 + 0.0*pca1'
.param U0_var = '0.0*pca0 + 0.0*pca1'
.param UA_var = '0.0*pca0 + 0.0*pca1'
.param pca0_tmp = agauss(0,1,1)
.param pca0 = pca0_tmp
.param pca1_tmp = agauss(0,1,1)
.param pca1 = pca1_tmp
.param pca2_tmp = agauss(0,1,1)
.param pca2 = pca2_tmp
.param pca3_tmp = agauss(0,1,1)
.param pca3 = pca3_tmp
```

The parameter definitions in the base model definition have been replaced with *‘<value> + <param>_var’*. The variation parameters are then defined below the model as a combination of multiple *agauss* statements.

Note

Before you apply any standard deviations, all of the coefficients are zero, that is, there is no variation.

There are as many *agauss* statements as there are variation parameters. Each *agauss* statement represents one eigenvector of the multivariate parameter distribution, and they are ordered from largest influence to smallest influence, that is, the coefficients of *pca0* represent the largest contribution to the overall variation and *pca3*, the lowest. You can also see that only *pca0* and *pca1* are applied to the parameters. This is controlled by the *components* argument that is passed to `update_model()` method. Depending on the complexity of the variation, the terms of least influence can be omitted without significantly changing the shape of the overall parameter distribution, that is, you can use the components argument to reduce the dimensionality of the variation. Each parameter then has a standard deviation applied to it by using the method `set_sigma()`, and some correlations are applied between parameters using `set_correlation()`. (The standard deviations are solely for demonstration purposes and do not represent any real technology.) After variation has been applied, you can call `update_model()` again to apply these changes to the model. The final model printout is shown here:

```
.model nmos1 nmos
+DEVTYPE=1
+LEVEL=72
+PHIG='4.5 + PHIG_var'
+UA='0.001 + UA_var'
+CDSC='0.1 + CDSC_var'
+U0='0.01 + U0_var'

.param PHIG_var = '-0.0050724244*pca0 + 0.0076987343*pca1'
.param CDSC_var = '-0.00050724244*pca0 + 0.00076987343*pca1'
.param U0_var = '-0.0041036776*pca0 + -0.0023790397*pca1'
.param UA_var = '-0.0016414710*pca0 + -0.00095161589*pca1'
.param pca0_tmp = agauss(0,1,1)
.param pca0 = pca0_tmp
.param pca1_tmp = agauss(0,1,1)
.param pca1 = pca1_tmp
.param pca2_tmp = agauss(0,1,1)
.param pca2 = pca2_tmp
.param pca3_tmp = agauss(0,1,1)
.param pca3 = pca3_tmp
```

The variation coefficients have now been updated to reflect the combined variation of all the parameters. The model can now written to a file and used in a PrimeSim HSPICE Monte Carlo simulation.

Variability Target Data

The variations produced by a Mystic PCA-based variability model can be compared against variability TCAD simulations or measurement target data. This section presents an example of this based on simulation data from the Synopsys statistical variability TCAD simulator Garand. As Garand is part of the TCAD to SPICE tool suite, all of its simulation results are uploaded to the TCAD to SPICE database, allowing Mystic to seamlessly load and use the results as target data. As with any other target in Mystic, all of the information required to set up and run PrimeSim HSPICE simulations under the same conditions as the target is stored in the structure and the metadata of the target itself. Garand data can be loaded in to the Mystic environment as usual by using the `mdb.Load` method as follows:

```
IdVg = mdb.Load(ivar='vgate', dvar='idrain', combine=True)
```

In this scenario, the *combine* argument is essential to ensure that the Garand data is kept together in a single *Data* object. The second-level independent variable of the data is an increasing integer field labeled *ensemble_id*. The label here informs Mystic that this is variation data and Mystic should set up a PrimeSim HSPICE Monte Carlo netlist when running against this simulation data. It is assumed, at this point, that the `Model` object in the environment has already had some sensible variations applied to it. The variation data can then be passed as usual into `ExtractionUtils.PrintErrors()`:

```
ExtractionUtils.PrintErrors('Variation', IdVg, Simulator, Model, 'rmsd')
```

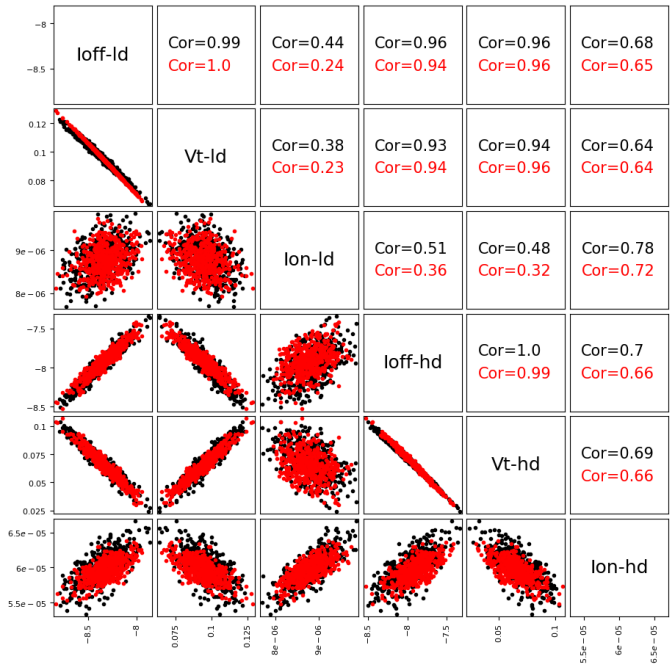
When the simulation is finished, the fit column of the Id–Vg *Data* object is populated with the result of the PrimeSim HSPICE Monte Carlo simulation. At this point, the error values between the curves are meaningless because each dataset in the target and fit simulations has different random seedings that do not align. To calculate a more sensible error, the results can be transformed, first into single figures of merit and then into the moment of the figure of merit distributions as follows:

```
VtIon = IdVg.MakeFoMTarget(['vt','ion'], [5e-8,None])
MeanStd = VtIon.MakeFoMTarget(['mean','std'])
```

A printout of MeanStd would look something like this:

```
               mean          std      mean-fit      std-fit       errors
index
Vt     2.143298e-01 1.986727e-02  2.143137e-01 2.483426e-02 1.767829e-01
Ion   -9.023801e-07 2.105117e-08 -9.017812e-07 2.633861e-08 1.776053e-01
```

The error column of the transformed data should now contain much more manageable values. Any of these data objects can be written to file using `WritePLTFile()` and plotted externally to Mystic using Sentaurus Visual or the *matplotlib* Python plotting library provided with TCAD to SPICE Python. The figure below shows an example of figure of merit distributions generated by Garand being matched by a PCA-based variation SPICE model generated by Mystic.



Statistical variability figure of merit distributions generated by Garand (black) and PCA variation model created by Mystic (red).

The transformed targets can also be used to optimize the parameters of the CorrelatedParameters class to give a variation model that better matches the Garand target distributions, but this is not covered in this document. For more information as well as access to demonstration PCA projects, contact your local TCAD support team.

RF Extraction

The general procedure of SPICE model parameter extraction for RF applications is slightly more complex than for standard CMOS applications. The main challenges in RF SPICE modeling are:

- There is a very high accuracy requirement on the SPICE models, because the admittance (Y) and scattering (S) parameters of an RF device are determined by the derivatives of I–V characteristics (transconductance and output resistance). To achieve good Y- and Sparameter accuracy, at the device active region, the RMS error of the SPICE model I–V must be under 1%.
- For very high frequency applications required by an RF SPICE model, distributed parasitic networks that might not be important for general CMOS logic application must be considered.

To help address these challenges, Mystic supports the fitting of SPICE model parameters against Y- and S-parameter targets and the extraction of RF-related parasitic networks. This section describes how to interact with RF-related target data within the Mystic framework but it does not describe in any detail the parameter extraction procedure. For a full demonstration flow of RF SPICE model extraction, contact your local TCAD support team.

Uploading Data

As with I–V, C–V, and transient targets, Mystic expects scattering parameter data to be formatted in a certain way and to provide enough information about the simulation to allow Mystic to reconstruct the simulation conditions in the backend PrimeSim HSPICE netlists it creates and runs. This section describes how to

upload PLT target data from a Sentaurus Device small-signal AC simulation to the TCAD to SPICE database for use in a Mystic extraction. The figure below shows the PLT file containing admittance data versus frequency for a two-port network representing the device being modeled.

```
DF-ISE text

Info {
  version   = 1.0
  type      = xyplot
  datasets  = [
    "frequency"
    "v(1)" "v(2)" "a(1,1)" "c(1,1)" "a(1,2)"
    "c(1,2)" "a(2,1)" "c(2,1)" "a(2,2)" "c(2,2)" ]
  functions = [
    Frequency
    Voltage Voltage Admittance Capacitance Admittance
    Capacitance Admittance Capacitance Admittance Capacitance ]
}

Data {
      7.00000000000000E+09
      4.00000000000000E-01   1.00000000000000E+00   2.05746555054810E-05   2.97051216276209E-14  -5.35215470262388E-06
     -8.78627105607206E-15   5.60590721073175E-02  -3.60398335248718E-14   2.66178221353005E-03   1.68118975683240E-14
      1.40000000000000E+10
      4.00000000000000E-01   1.00000000000000E+00   8.22337630452501E-05   2.96824687541098E-14  -2.13926407327530E-05
     -8.78061866298883E-15   5.59809725014359E-02  -3.60108012253218E-14   2.68220851905252E-03   1.68047545845361E-14
      2.10000000000000E+10
```

Sentaurus Device AC small-signal PLT output file

The admittance data can be extracted against directly in Mystic or converted to an S-parameter matrix internally before extracting.

As shown in the PLT File Upload section, *Data.from_plt* can be used to simultaneously read in and upload the PLT data to the TCAD to SPICE database. For admittance data, some extra options and metadata fields are required. The following example demonstrates an admittance file upload:

```
ds = dbi.create_dataset(node_prj, "rf-data", clean=True)

metadata = {"temperature": 298,
            "instances": {"l":3e-8, "w":5e-5},
            "ports": {"1": "gate", "2": "drain"},
            "nodes: ["drain","gate","source","substrate"],
            "bias": {"source": 0.0, "substrate": 0.0}}

Data.from_plt("RFQ_ac_des.plot", ivar="frequency",
              metadata=metadata, upload_ds=ds,
              rf_data=True, combine=True)
```

The metadata dictionary contains the same temperature, instances, nodes, and bias fields that are required for DC target data. There is an extra *ports* field that maps port numbers in the network to terminals of the SPICE model. In general, for standard MOSFETs represented as two-port networks, this is always the same: port 1 is mapped to the gate contact and port 2 is mapped to the drain contact.

For admittance data, two extra arguments must be passed to *Data.from_plt*. The rf_data argument instructs the *Data* class to produce an RF_Data object. This is a derivative of the Data class described in The Data Class with some extra functionality for matrix conversion and extracting the real and imaginary parts of the individual data components. The combine argument indicates that all of the individual admittance components should be combined horizontally into a single structure as shown here:

```
                 y(11)re       y(11)i       y(12)re       y(12)i ...
frequency                                                       ...
2.795000e+10 1.784044e-04 3.542094e-03 -3.196667e-05 -9.527018e-04 ...
2.797500e+10 1.787211e-04 3.545245e-03 -3.202406e-05 -9.535513e-04 ...
2.800000e+10 1.790381e-04 3.548396e-03 -3.208150e-05 -9.544008e-04 ...
2.802500e+10 1.793553e-04 3.551547e-03 -3.213899e-05 -9.552503e-04 ...
2.805000e+10 1.796729e-04 3.554698e-03 -3.219653e-05 -9.560998e-04 ...
```

As with I–V and C–V data, Mystic expects the column headers to be in a specific format that it uses to infer the simulation type when constructing backend PrimeSim HSPICE netlists. For Y- and S-parameter data, the table below shows the required header formatting.

Notation	As ivar	As dvar
y(<port1><port2>)<real>	n/a	The admittance between port1 and port2 of the network, where real denotes whether it is the real or imaginary part.
s(<port1><port2>)<real>	n/a	The scattering parameter between port1 and port2 of the network where real denotes whether it is the real or imaginary part.
y(<port1><port2>)<real>	AC frequency sweep. This is the required independent variable for admittance or S-parameter data.	n/a

As mentioned in Uploading PLT Files, there is a standard mapping to convert Sentaurus Device columns to the expected Mystic format. The default mapping for small-signal AC data is as follows:

- “a(<port>,<port>)” is converted to “y(<port><port>)re”
- “c(<port>,<port>)” is converted to “y(<port><port>)i”

If the data is not coming from the output of a Sentaurus Device simulation, then you can use *column_map* to convert and upload any CSV-like data format. For a description, Uploading CSV Files.

Loading and Manipulating Data

When the Y-parameter target data has been uploaded to the TCAD to SPICE database, it can be loaded into the Mystic extraction strategy as usual by using the `mdb.Load` method (see Database). The following example shows how to do this for RF-specific data:

```
Y_data = mdb.Load(ivar='frequency',
                  dvar=['y(11)re','y(11)i',
                        'y(12)re','y(12)i',
                        'y(21)re','y(21)i',
                        'y(22)re','y(22)i'])
```

Here, you load the reflection and gain from both ports, giving you the full matrix. This is required if the data should be converted to S-parameter data as the conversion function requires all components to be present. After the data is in the Mystic environment, it can be filtered and resampled like any other data object:

```
Y_f28 = Y_data.FilterData('eq', 28e9, dtype='ivar')
Y22 = Y_f28.FilterData("eq", ["y(22)re","y(22)i"],
                       dtype="column", mask=True)
Y12 = Y_f28.FilterData("eq", ["y(12)re","y(12)i"],
                       dtype="column", mask=True)
Y21 = Y_f28.FilterData("eq", ["y(21)re","y(21)i"],
                       dtype="column", mask=True)
```

The first filter selected the first harmonic of the frequency sweep, and the subsequent filters use *dtype='column'* to isolate individual Y-components, so they can be extracted separately.

Note

In these filters, mask=True. This is essential so a reference to all components is retained in order to construct the PrimeSim HSPICE simulation netlist. It also helps to speed up the extraction when multiple components are being extracted against simultaneously as only a single simulation of the root object is required. Only the RMS error will be calculated on the filtered components.

The RF_Data objects have a specific function for performing matrix conversions between Yparameters and S-parameters. The following example shows how to create an S-parameter target from Y-parameter data using `MakeFoMTarget()`, which is described in Figure of Merit Transforms:

```
S_data = Y_data.MakeFoMTarget("convert", "s")
```

SPICE model parameters can be extracted against either Y-parameters or S-parameters depending on the required effect.

Extracting Parameters

In general, RF data can be treated in the same as any other type of data when it is used to extract SPICE model parameters. Individual data objects can be passed to `ExtractionUtils.PrintErrors()` to calculate single RMSD errors and to `ExtractionUtils.DoStage()` to optimize parameter values. The following example shows a two-parameter fringing capacitance extraction against some Y-parameter target data:

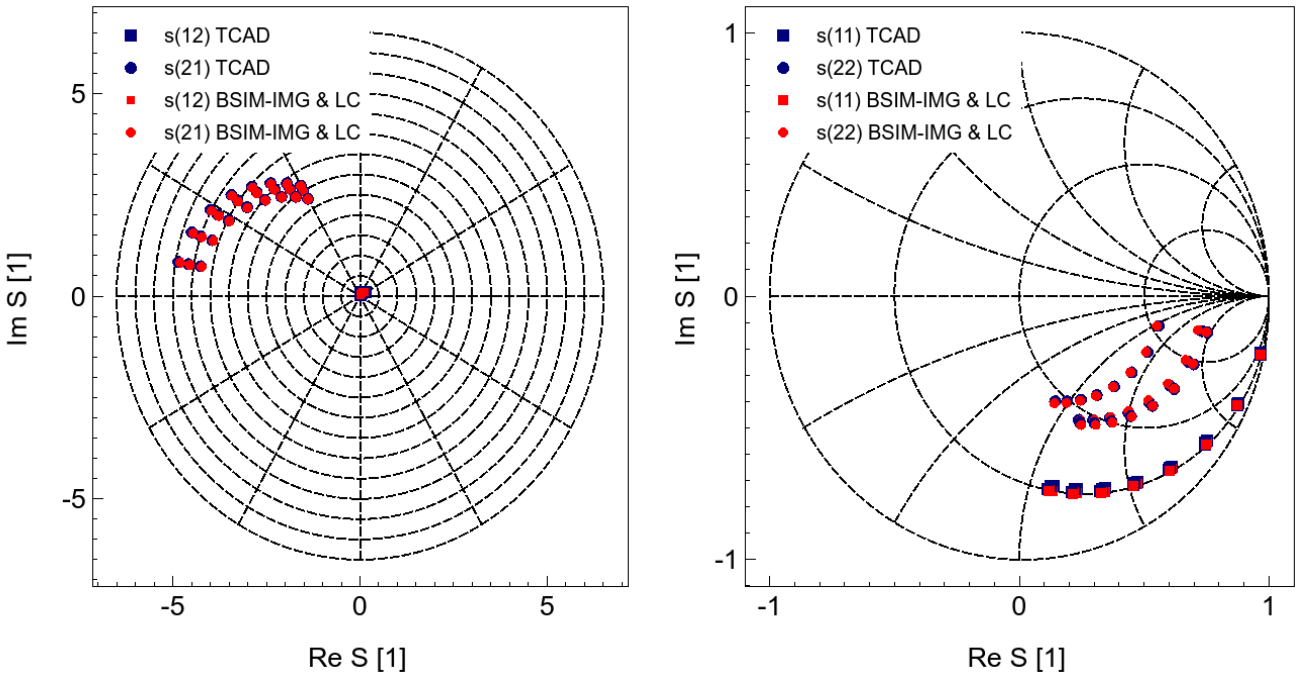
```
cfs = OptParam("submosn.cfringes", 1e-16, 1e-17, 1e-14, 1e-18)
cfd = OptParam("submosn.cfringed", 1e-16, 1e-17, 1e-14, 1e-17)
#Y12_data1 : vg=0.4 vd=1.0
#Y12_data2 : vg=0.4 vd=1.2
#Y12_data3 : vg=0.6 vd=1.2
ExtractionUtils.DoStage("Step1", Model, Simulator, [cfd,cds],
                        Optimizer, [Y12_data1,Y12_data2,Y12_data3],
                        [2,2,1])
```

The target here is the Y12 gain component of the matrix. One of the complexities of RF parameter extraction is that the device often must perform over a range of frequencies and biases. In this example, the full set of Y-parameter data includes three biases denoted in the commented lines.

While you want the fit to be as accurate as possible across all data, it sometimes makes sense to apply weighting factors to a section of the data to prioritize accuracy in that area. In this case, you are doubling the influence of the *vg=0.4* targets and deprioritizing the *vg=0.6* target. This gives extra flexibility where extreme accuracy cannot be achieved across the full performance range. Note that this is only a single extraction step in a complex multi-stage strategy.

Data Output

RF simulation targets can be written to disk using the `WritePLTFile()` or `WriteCSVFile()` method. For details about these methods, see the Mystic API documentation. After the data is written, it can be plotted externally using Sentaurus Visual as seen in the figure below.



The plotted data covers a range of bias conditions and frequencies, and shows an accurate fitting for all components between the BSIM-IMG SPICE model and TCAD data.

Lookup Table (LUT) Modelling

Lookup Table (LUT) models are analytical, behavioral models that can be used to perform fast analysis of a new technology before a reliable SPICE model or extraction strategy has been established. Mystic can generate LUT models based on grids of target IV and QV information using Synopsys PrimeSim.

Target data for a LUT extraction should be uploaded to the TCAD to SPICE database in the normal way as described in the Uploading Target Data to The Database section. The target data should include biases and current for all the terminals of the device, i.e. drain, gate, source and substrate voltages and currents. It should also have length (L), width (W) and temperature metadata fields. These are required by the PrimeSim LUT model.

Once the data has been loaded from the Database into the environment using `mdb.Load` it should be resampled using the `resampleNd()` method to give an evenly spaced, dense grid of points as shown below. Note that a denser target grid will give a smoother model response in PrimeSim but will result in slower simulation times. You should evaluate some different options and choose a reasonable grid density for your application.

```
IV = mdb.Load(project=targetDataProject, ivar='vgate',
              combine=True, extra_index='all')

IVr = IV.resampleNd(points={'vdrain':51,
                            'vgate':51,
                            'vsubstrate':11},
                    overwrite=False)
```

Note that to make the grid resampling possible the *combine=True* and *extra_index='all'* flags should be passed to the mdb.Load method to condense all the data into a single object. The `resampleNd()` method should be passed either a points or step dictionary that determines the density of the data for all terminal biases. Once the dense data has been created the LUT table can be written using the `write_LUT()` method as shown below.

```
column_map = {"vdrain":     "Vds",
              "vgate":      "Vgs",
              "vsubstrate": "Vbs",
              "idrain":     "Id",
              "igate":      "Ig",
              "isource":    "Is",
              "qdrain":     "Qd",
              "qgate":      "Qg",
              "qsource":    "Qs"}

Simulator.write_LUT(IVr[0], "nmosmod", path="LUT_MODELS",
                    column_map=column_map)
```

The PrimeSim LUT model requires specific formatting of voltages, current and charges. It is therefore necessary to supply a column_map dictionary to convert the naming conventions in the target data to the expected format of PrimeSim. The LUT model is written to the directory specified by the path argument using the 'nmosmod' model name. In the above example the output model will be written to 'LUT_MODELS/nmosmod_W1u_L32n_T27.dat' where the device width is 1um, the gate length is 32nm and the temperature is 27K. This model file is used by PrimeSim when running SPICE simulations. Once the LUT model has been written a validation simulation can be run using the `ExtractionUtils.PrintErrors()` function as shown below.

```
Simulator.executable = "primesim"

ExtractionUtils.PrintErrors("IV", IVr, Simulator, Model, "rmsd")
```

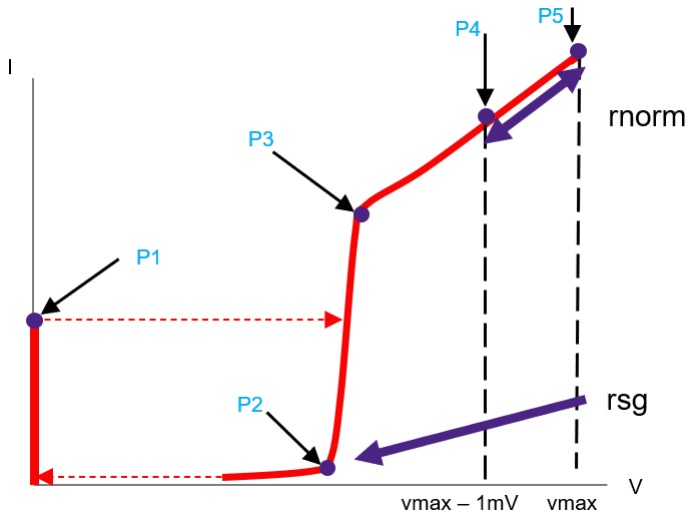
Prior to running the simulation, ensure a PrimeSim binary is available is your $PATH and switch to using PrimeSim in Mystic by setting the *Simulator.executable* attribute. If the LUT model is operating correctly the error printed to screen should be close to zero as the data generated by the PrimeSim simulation should match the target data perfectly.

Josephson Junction Extraction

The Josephson effect is a phenomenon that occurs when two superconducting electrodes are separated by an insulating (nonsuperconducting barrier). When cooled below the critical temperature of the material, Tc, a current, known as a supercurrent, can flow continuously across the barrier without any voltage applied. A device that takes advantage of this effect is known as a Josephson junction. Josephson junctions can be used in a range of quantum-mechanical superconducting circuits, such as rapid single flux quantum (RSFQ) digital logic, superconducting quantum interference devices (SQUIDs), and superconducting qubits. The PrimeSim HSPICE tool contains a SPICE model representing a Josephson junction. It can be used to simulate the behavior of these devices in a range of circuit applications. The Mystic tool can extract SPICE model parameters for these devices against quasistatic I–V target data coming from either TCAD or measurement.

Extracting Figures of Merit

The I–V target for a Josephson junction is a transient simulation where a stimulus current is applied across the device, eliciting a voltage response across the junction. As with most transient targets, the I–V curve for a Josephson junction can be noisy and is a difficult shape to fit holistically. The model response also oscillates, making a direct extraction against the full I–V near impossible. To overcome this, Mystic tool provides some specific transforms and figures or merit to split the full target into a smaller set of key characteristics. The figure below shows the parts of the curve that should be captured accurately.



Typical Josephson junction I–V curve shape (red) with key figures of merit: P1–P5, rnorm, and rsg.

First, you apply a time-averaged transform to dampen the oscillations of the SPICE model response:

```
iv = mdb.Load(ivar="time", temperature=4.2)
iv_smooth = iv.ApplyMeasure("jj-avg")
iv_smooth.TestMeasure(Simulator)
```

This ensures that both the target and fitted data is smooth and continuous. Then, you extract points P1 to P5 as well as the normal resistance (rnorm) and the subgap resistance (rsg) as follows (see Figure above):

```
p1 = iv_smooth.P1()[0]
p2 = iv_smooth.P2()[0]
p3 = iv_smooth.P3()[0]
p4 = iv_smooth.P4()[0]
p5 = iv_smooth.P5()[0]
rnorm = iv_smooth.rnorm()[0]
rsub = iv_smooth.rsg()
```

The Josephson junction SPICE model available in the PrimeSim HSPICE tool is behavioral, meaning that many of the parameter values can be set directly based on the extracted electrical FoMs without the need for optimization. For example, the critical current parameter ics can be set using p1:

```
Model.SetModelParameter('ics', p1)
Model.SetModelParameter('rnorm', rnorm)
Model.SetModelParameter('rsub', rsub)
Model.SetModelParameter('vgap', (p2+p3)/2)
Model.SetmodelParameter('delv', p3-p2)
```

Optimization

Although most parameters can be set behaviorally, some optimization might be required to obtain a more accurate fit or to extract parameters that cannot be set in this way, for example, the temperature dependence of the device. As the simulation temperature increases, the gap voltage of the junction deceases. To model this effect, you can create FoM targets based on p2 and p3 and extract the two available temperature dependence parameters tc and tcfct.

```
Optimizer.set_method("doe")
Optimizer.set_optimization_parameters(method="bayesian_optimizer", Ns=10, do_plots=True)

tcfct = OptParam("jj_u.tcfct", 1.5, 1.5, 2.5, 100.0)
tc = OptParam("jj_u.tc", 9.28, 7.1, 9.5, 10.0)

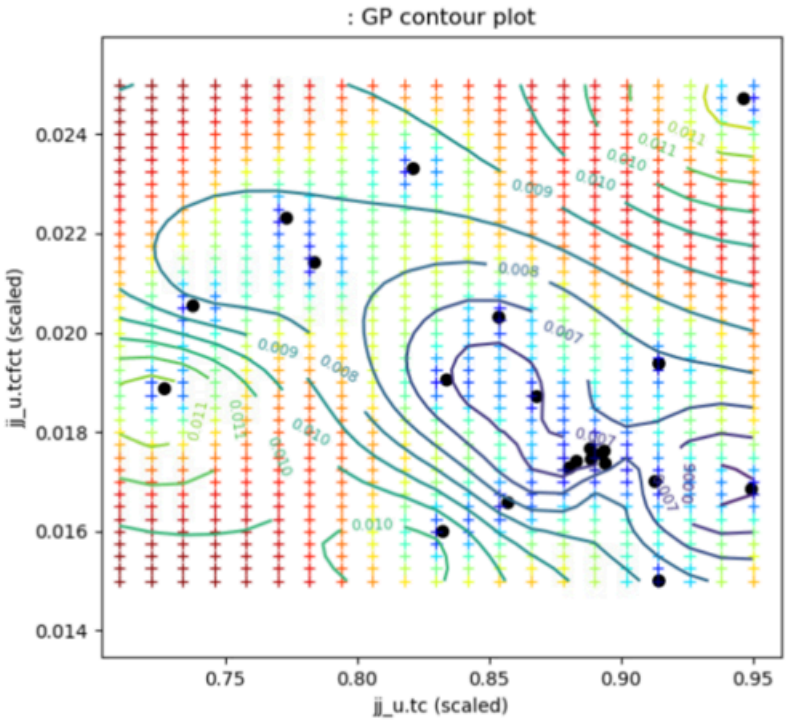
p2 = iv_smooth.MakeFoMTarget("p2", add_as_fom=True, apply_to_measure=True)
p3 = iv_smooth.MakeFoMTarget("p3", add_as_fom=True, apply_to_measure=True)

ExtractionUtils.DoStage("Stage1", Model, Simulator, [tc,tcfct], Optimizer,
                        [p2,p3], verbose=True, parallel=True)
```

As you are going through two layers of data transformation here (time-averaged smoothing and FoM extraction), there will be significant noise in the output response based on small changes in the input parameter values. This means a conventional gradient-descent-based optimization algorithm will struggle to converge to a good solution. To overcome this issue, you can use the derivative-free Bayesian optimization algorithm from the Mystic optimizer library.

This algorithm samples the parameter space between the upper and lower bounds, which you defined, and then builds a surrogate objective function based on these evaluated points. It quantifies the uncertainty of the surrogate using Gaussian process regression. The algorithm then resamples the parameter space based on an acquisition function defined by the surrogate to obtain more information about the objective in areas

where the surrogate predicts the error will be low. This process is repeated until the algorithm has established a minimum error within the parameter space. At each iteration of sampling, the algorithm produces a variation of the plot shown in the figure below.

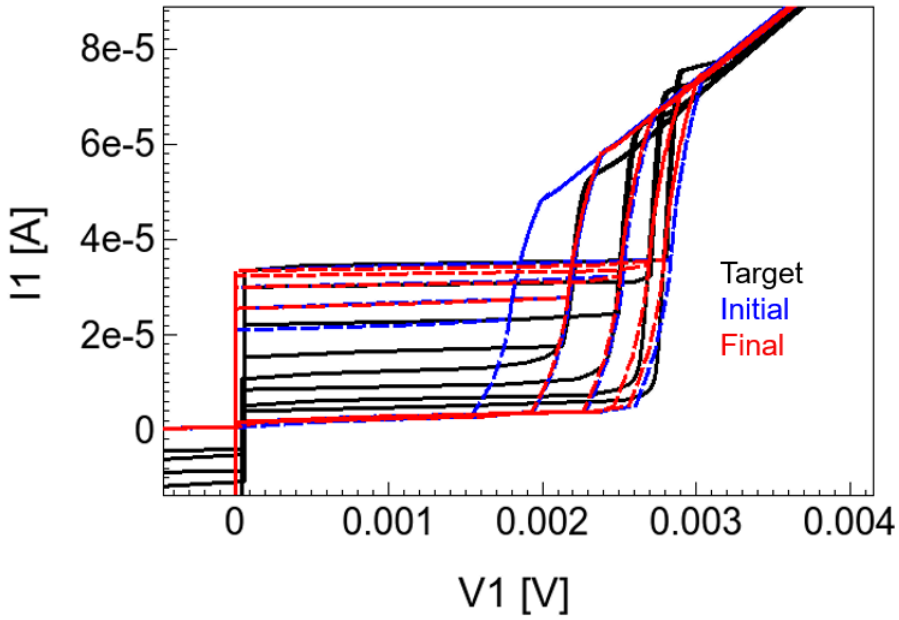


Two-dimensional space sampled during Bayesian optimization of TC and TCFCT.

The plot shows the points that have been sampled in all the previous iterations, along with the current 2D representation of the current surrogate model (lines) and the uncertainty of the model (crosses) across the parameter space. If you have more than two parameters, then one 2D plot will be produced for each combination of two parameters. In this case, the 2D model represents a cutplane of the actual N-dimensional model going through the current optimal point.

Data Output

The I–V data for Josephson junctions can be treated like any other target data object in the Mystic tool and can be written to PLT or CSV files using the `WritePLTFile()` and `WriteCSVFile()` methods, respectively. For details about these methods, see the Mystic API documentation. The figure below shows an example of temperature-dependent I–V curves plotted externally in Sentaurus Visual, having been output from the Mystic tool. This data shows that the Mystic tool has extracted values for tc and tcfct that accurately capture the temperature dependence shown by the target data.



Josephson junction I–V curves representing target data (black), initial fit data (blue), and final fit data (red) at various temperatures from 4 K to 7 K.

Machine Learning in Mystic

Data Storage & Retrieval

When Mystic executes, it is continually evaluating PrimeSim HSPICE simulations at a particular state of the user's target model. The results of these simulations contain valuable information about how the behaviour of the model responds to changes in parameter values. In the normal execution mode of Mystic, these simulations results are used to determine the convergence of the optimization algorithm, then immediately discarded. The base of any good machine learning (ML) model is a broad training set. Mystic provides the capability to store all of it's simulation results in it's database and organise them by SPICE base model, parameter values and data type. This data can form the training set for complex

Data storage can be activated when storing the initial model in the database for the first time through *StoreInitialModel* using the spice_ml_name flag:

```
mdb.StoreInitialModel(Model, spice_ml_name="<my-nmos>")
```

Where '<my-nmos>' is a label supplied by the user. Once the base model has been stored, all evaluations executed by Mystic will be stored in the database along with the parameter values that differ from the base model configuration.

As well as providing a data set for training ML models, the data storage framework can provide a couple of advantages to traditional extraction methodologies, described below.

Re-running a strategy

When developing a strategy, it is common to build the optimization steps iteratively making small adjustments to parameter sets and re-running the strategy regularly until it gives the desired results. When re-running, the strategy must move through all steps sequentially, even if these steps have been run many times before. This can be time consuming, especially for more complex strategies with many steps. When ML data storage is activated, simulation results that have been executed previously can be retrieved directly from the database instead of re-running the simualtion. This can result in steps that have been executed previously under identical conditions to run in a fraction of the time taken for the initial execution and can help to significantly speed up the strategy development process. Below is an example of a simple re-executed strategy:

```
ExtractionUtils.DoStage(
    'Step1', Model, Simulator, [PHIG,QMFACTOR,CDSC],
    Optimiser, [IsVgld_BelowVt]
)
ExtractionUtils.DoStage(
    'Step2', Model, Simulator, [U0,UA,EU,UD,UCS,ETAMOB],
    Optimiser, [IsVgld_AboveVt]
)
```

Results of the first execution of the strategy as follows:

```
Fitting Result Data
-------------------
name       = Step1
parameters = PHIG : 4.282215386922149, QMFACTOR : 4.378844578760421e-06, CDSC : 0.26308688002153574
iterations = 7
fit error  = 16.42402671579762
converged  = 1
method     = bounded_trust_region
wallclock  = 5.88s
message    = 2-norm of trial step < eps[3]


Fitting Result Data
-------------------
name       = Step2
parameters = U0 : 0.018977614727324156, UA : 0.43215905812510336, EU : 1.4205355569299252,
             UD : 0.015526925366273082, UCS : 1.0577150103348911, ETAMOB : 4.9028241881361065
iterations = 17
fit error  = 1.5499651660012832
converged  = 1
method     = bounded_trust_region
wallclock  = 24.11s
message    = ||F(x)||_2 - ||F(x) - J(x)s||_2 < eps[4]
```

The total run time of the extraction is ~30s. If we then re-run the strategy from scratch in a new Mystic process the results are as follows:

```
Fitting Result Data
-------------------
name       = Step1
parameters = PHIG : 4.282215386922149, QMFACTOR : 4.378844578760421e-06, CDSC : 0.26308688002153574
iterations = 7
fit error  = 16.42402671579762
converged  = 1
method     = bounded_trust_region
wallclock  = 0.26s
message    = 2-norm of trial step < eps[3]


Fitting Result Data
-------------------
name       = Step2
parameters = U0 : 0.018977614727324156, UA : 0.43215905812510336, EU : 1.4205355569299252,
             UD : 0.015526925366273082, UCS : 1.0577150103348911, ETAMOB : 4.9028241881361065
iterations = 17
fit error  = 1.5499651660012832
converged  = 1
method     = bounded_trust_region
wallclock  = 1.15s
message    = ||F(x)||_2 - ||F(x) - J(x)s||_2 < eps[4]
```

The previous simulation results are now being retrieved from the database and the total run time reduces to ~1.4s while the results remain identical. It’s important to understand that if changes are made to any extraction steps, e.g. adding a new parameter to the set, changing the target data; the convergence path of the optimization will change and all downstream steps will generate new results instead of retrieving results from the database. For example, let’s remove ETAMOB from step two in the above example:

```
Fitting Result Data
-------------------
name       = Step1
parameters = PHIG : 4.282215386922149, QMFACTOR : 4.378844578760421e-06, CDSC : 0.26308688002153574
iterations = 7
fit error  = 16.42402671579762
converged  = 1
method     = bounded_trust_region
wallclock  = 0.26s
message    = 2-norm of trial step < eps[3]


Fitting Result Data
-------------------
name       = Step2
parameters = U0 : 0.019329232614071462, UA : 1.4596532051181694, EU : 1.482413401780866,
             UD : 0.0006473925995545809, UCS : 2.4998441400909347e-07
iterations = 8
fit error  = 1.650754293204923
converged  = 1
method     = bounded_trust_region
wallclock  = 7.36s
message    = ||F(x)||_2 - ||F(x) - J(x)s||_2 < eps[4]
```

The run time for the first step remains low as the change has been made in the second step, which now takes ~7s to run.

Reverse Search Extractions

When running multiple Mystic extractions across multiple conditions in the same, e.g. different geometries, device flavours, or when re-running a project regularly with small changes in target data from new TCAD / measurement results, it may also be useful to retrieve the best existing parameter set in the database using a reverse search algorithm. Mystic provides the *ExtractionUtils.ReverseSearch* function to do this. The following example will demonstrate this capability by first executing a multi-step extraction, adjusting the target data values by 5%, finding the current best parameter set using a reverse search, then using the *optimize* flag to perform an optimization to find the true best parameter set for the new target data.

```
ExtractionUtils.DoStage(
    'Step1', Model, Simulator, [PHIG,QMFACTOR,CDSC],
    Optimiser, [IsVgld_BelowVt]
)
ExtractionUtils.DoStage(
    'Step2', Model, Simulator, [U0,UA,EU,UD,UCS,ETAMOB],
    Optimiser, [IsVgld_AboveVt]
)
ExtractionUtils.DoStage(
    'Step3', Model, Simulator, [PHIG,QMFACTOR,CDSC,U0,UA,EU,UD,UCS,ETAMOB],
    Optimiser, [IsVgld]
)

# Scale the target by 5%
IsVgld.scale(1.05)

# Perform a reverse search agains the new target and find the best existing parameter
# set in the database
ExtractionUtils.ReverseSearch(
    mdb, IsVgld, Model, optimize=True,
    parameters=[PHIG,QMFACTOR,CDSC,U0,UA,EU,UD,UCS,ETAMOB]
)
```

The output of the above *ExtractionUtils.ReverseSearch* command is shown below.

```
Best existing parameter set:
+----------------+----------------------+
|   Parameter    |        Value         |
+----------------+----------------------+
|   nmos1.cdsc   | 0.14238785666429357  |
|   nmos1.cgsl   |        1e-20         |
|  nmos1.dvtp1   |         1.0          |
|   nmos1.eta0   |         0.0          |
|  nmos1.etamob  |  7.135360337604873   |
|    nmos1.eu    |  1.200097982796041   |
|   nmos1.hfin   |        4e-08         |
|   nmos1.phig   |  4.2778232413144535  |
| nmos1.qmfactor | 0.04298636993612226  |
|   nmos1.tfin   |        8e-09         |
|    nmos1.u0    | 0.024312196905548547 |
|    nmos1.ua    |  0.5051960473845137  |
|   nmos1.ucs    |  1.0955676204214733  |
|    nmos1.ud    |  0.1005523392201767  |
+----------------+----------------------+
|     error      | 0.03559017295353863  |
+----------------+----------------------+

Optimizing best parameter set...

Fitting Result Data
-------------------
name       = ReverseSearch
parameters = PHIG : 4.277645478136995, QMFACTOR : 0.041770274825027626, CDSC : 0.13461388556090664,
             U0 : 0.0257424436598912, UA : 0.5132704063652896, EU : 1.21036528573285,
             UD : 0.09876425384099387, ETAMOB : 7.262428737240803, UCS : 1.0624421492241178
iterations = 14
fit error  = 0.7521253104902021
converged  = True
method     = least_squares
wallclock  = 9.13s
message    = `xtol` termination condition is satisfied.
```

From the output, you can see that the first 3 steps take a total of ~38s with a final error of 0.88%.

ML Classifier Accelerated Extraction

Population based optimization algorithms such as evolutionary algorithms and DoE algorithms explore a parameter space iteratively and converge to the global optimal solution of the space. For higher dimensional problems these algorithms can require hundreds of thousands of evaluations and run times can come prohibitively expensive. Mystic can enhance the performance of these algorithms by inserting a classifier between the algorithm and the PrimeSim HSPICE objective function that classifies candidates in the population based on their likelihood to have a low error versus the target data. High potential candidates are simulated while low potential candidates are discarded. This can help to reduce the overall number of evaluations required for convergence and reduce the runtime, while still achieving successful convergence. The following example demonstrates this capability on a low drain IdVg optimization problem.

```
Optimiser.set_method("differential_evolution")
Optimiser.SetOptimisationParameter("popsize", 30)
Optimiser.SetOptimisationParameter("maxiter", 50)
Optimiser.SetOptimisationParameter("ml_classifier", True)
Optimiser.SetOptimisationParameter("ml_kwargs", {
    "min_percentile": 80,
    "fill_multiplier": 10,
    "score_threshold": 0.8
})

Simulator.max_sims = 10

ExtractionUtils.DoStage(
    "Stage1", Model, Simulator, [PHIG,QMFACTOR,CDSC,U0,UA,EU],
    Optimiser, [IsVgld], parallel=True
)
```

The *ml_classifier* and *ml_kwargs* options can be applied to any population-based algorithm to apply the classifier. The output of the above strategy snippet is shown below.

```
Fitting Result Data
-------------------
name       = 14nm-FinFET-Mystic-nmos-global
parameters = PHIG : 4.263592676851327, QMFACTOR : 0.19948092734970402, CDSC : 0.09921803081762176,
             U0 : 0.020739693065431913, UA : 1.7408007756183679, EU : 1.3239571984081078
iterations = 9187
fit error  = 2.2325050627884786
converged  = False
method     = differential_evolution
wallclock  = 933.4s
message    = Maximum number of iterations has been exceeded.
score      = 0.8039895923677364
threshold  = 0.24717966412830766
run_ratio  = 62.806139109611415
```

The important number to take note of in the output is the *run_ratio*, which shows the percentage of evlauations which have been executed versus discarded. In the example above, almost 40% of total evaluations have been discarded, significantly reducing the runtime.

For more information on applying the ML classifier see Machine Learning Acceleration.

Copyright

Copyright and Proprietary Information Notice

© 2024 Synopsys, Inc. This Synopsys software and all associated documentation are proprietary to Synopsys, Inc. and may only be used pursuant to the terms and conditions of a written license agreement with Synopsys, Inc. All other use, reproduction, modification, or distribution of the Synopsys software or the associated documentation is strictly prohibited.

Destination Control Statement

All technical data contained in this publication is subject to the export control laws of the United States of America. Disclosure to nationals of other countries contrary to United States law is prohibited. It is the reader’s responsibility to determine the applicable regulations and to comply with them.

Disclaimer

SYNOPSYS, INC., AND ITS LICENSORS MAKE NO WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, WITH REGARD TO THIS MATERIAL, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE.

Trademarks

Synopsys and certain Synopsys product names are trademarks of Synopsys, as set forth at https://www.synopsys.com/company/legal/trademarks-brands.html. All other product or company names may be trademarks of their respective owners.

Free and Open-Source Licensing Notices

If applicable, Free and Open-Source Software (FOSS) licensing notices are available in the product installation.

Third-Party Links

Any links to third-party websites included in this document are for your convenience only. Synopsys does not endorse and is not responsible for such websites and their practices, including privacy practices, availability, and content.

https://www.synopsys.com